



The background features several hand-drawn diagrams illustrating frontend system concepts. At the top, a diagram shows a 'FeedItem' being passed to a 'Handler', which then updates a 'View'. Below this, a 'Viewport' is shown containing a list of items, with a 'more items' label indicating pagination. At the bottom left, a 'Database' is connected to a 'Service Layer' (represented by a gear icon), which in turn interacts with a 'Controller' (represented by a box with a play button icon). At the bottom right, a diagram shows a 'Loading' state with a progress bar, and a note 'update first & Rollback' with arrows pointing to two boxes, suggesting a transaction or state management process.

Frontend System Design Essentials

Juntao Qiu

Frontend System Design Essentials

A practical guide to designing scalable, reliable, and maintainable frontend systems.

Juntao Qiu

This book is available at

<https://leanpub.com/frontend-system-design-essentials>

This version was published on 2026-01-17



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2026 Juntao Qiu

Contents

- Preface 1**
 - A High-Level View of Frontend Work 1
 - Book Structure and Outline 2
 - How to Make the Most of This Book 3
 - Summary 4
- Chapter 1 – Evolving from Components to Systems 5**
 - Analyzing how simple requests turn complex 5
 - Part I – How a small component grows 5
 - Zooming out to the system perspective 10
 - Developing system design thinking 11
 - A note on CCDAO and the interview lens 12
 - Summary – The invisible work 13
- Chapter 2 – Introducing the CCDAO Framework 14**
 - Collecting information for requirement clarity 14
 - Designing modular component structures 15
 - Modeling data for predictability and efficiency 15
 - Designing stable API contracts 16
 - Implementing optimization strategies early 16
 - Applying CCDAO in an Interview 17
 - Why It Matters Beyond Interviews 17
- Chapter 3 – Applying CCDAO: Designing a Typeahead Search Box 19**
 - 1. Collect Information 19
 - 2. Component Structure 20
 - 3. Data Modeling 20
 - 4. API Design 21
 - 5. Optimization Strategies 22
 - Closing 22

Chapter 4 – Applying CCDAO: Designing a Scalable Feed List 24

1. Collect Information 24

2. Component Structure 25

3. Data Modeling 26

4. API Design 26

5. Optimization Strategies 27

Closing 28

Chapter 5 – Data Modeling: Understanding the Domain and the UI . . . 30

Defining the domain clearly through business rules 30

Designing models for specific UI consumption patterns 33

From Domain to Design 33

Chapter 6 – Case Study: Implementing Sidebar Entitlements 35

Step 1 – The Starting Point: Logic in the UI 35

Step 2 – When the Rules Multiply 35

Step 3 – Moving Business Logic to the Backend 36

Step 4 – Where Frontend Logic Still Belongs 36

Step 5 – Tailoring APIs with GraphQL or a BFF 37

Lessons from the Sidebar 38

Chapter 7 – Setting Up the Project Environment 39

Why We Use a Starter Project 40

Prerequisites 40

Getting the Project Running 41

Useful Scripts 41

Configuring the Mock API with MSW 41

Mock Endpoints Provided 42

Data Shapes You’ll Work With 43

Project Structure (High-Level) 44

Troubleshooting 44

What Comes Next 45

Chapter 8 – Implementing Data Normalization 46

Why Normalisation Matters 46

A Real Example: Inconsistent User Data 46

Establishing a consistent source of truth 48

Transforming nested board payloads into flat tables 49

Holding the normalized store in React Context 51

Hydrating data during the rendering process 53

Summary	54
Chapter 9 – Drawing Inspiration from Backend Databases	56
A Familiar Pattern From Databases	56
How Databases Reconstruct Data	57
The Frontend Equivalent	58
Why This Matters	60
A Good Place to Pause	60
Chapter 10 – Managing Requests and Data Fetching	62
Showing Real Assignees in the UI	62
Extending the Backend API	63
Updating the Board on Selection	64
Understanding Race Conditions in Search	65
Fixing Race Conditions with Request Cancellation	66
Reducing Request Volume with Debouncing and Throttling	67
Summary	68
Chapter 11 – Implementing Pagination Strategies	70
Demonstrating Pagination in UserSelect	70
The Pagination Strategies	74
A Simple Users Table	74
Offset Pagination	75
Cursor Pagination	76
Choosing the Right Strategy for UserSelect	76
UI Patterns for Pagination	77
Summary	78
Chapter 12 – Migrating to an Express Backend	80
A Brief Introduction to Express	80
Setting up a standalone mock API server	81
Proxying Frontend Requests to The Express Server	82
Summary	83
Chapter 13 – Implementing Server-Side Rendering	85
Rendering strategies	85
How SSR works	86
Coordinating hydration between server and client	86
Creating the client entry (hydration)	86
Creating the server entry (rendering on the server)	87

CONTENTS

Creating separate entry points	87
Configuring Vite to build dual entries	89
Generating HTML on the server	90
Bringing it together in the Express SSR route	91
Why consistency matters	92
From SSR to SPA behaviour	93
Summary	93
Case Study – Understanding Bundlers Through Code Splitting	94
What Problem Are We Solving?	94
What a Bundler Actually Does	95
Tree Shaking: Removing Unused Exports	96
Code Splitting: Deferring Code Until It’s Needed	97
A Subtle Caveat: Tree Shaking Stops at Dynamic Boundaries	98
What This Case Study Teaches Us	99
Summary	99
Chapter 14 – Implementing Code Splitting and Lazy Loading	100
Why Lazy Loading Helps	100
The Building Blocks of Lazy Loading	100
Implementing Lazy Loading in Our Board Application	102
Another Example: Lazy Loading the List View	104
Understanding the Build Output After Code Splitting	106
What Happens at Runtime	106
How Code Splitting Changes the Bundle	107
Summary	107
Chapter 15 – Implementing Data Prefetching	108
How Prefetch Works Conceptually	108
The Prefetch Function Inside QueryProvider	108
Exposing Prefetch Through a Hook	110
Prefetching Users in Card.tsx	111
How Prefetch and useQuery Work Together	113
The Full Runtime Experience	114
When Prefetch Helps	114
When Prefetch Doesn’t Help	114
Summary	115
Chapter 16 – Implementing Optimistic Updates	116
Creating a New Card (Server-First)	116

CONTENTS

Introducing Optimistic Updates	118
Adding a Card Optimistically	118
Deleting a Card Optimistically	120
Things to Keep in Mind	120
Summary	121
Chapter 17 – Exploring Real-time Update Strategies	122
Polling	122
Server-Sent Events (SSE)	123
WebSockets	124
Scaling Considerations	125
Summary	126
Chapter 18 – Implementing Real-time Updates with SSE	127
The pub-sub pattern	127
Using the pub-sub pattern on the server	128
Using the event emitter with SSE	129
Emitting events from the update API	131
Frontend: consuming SSE with EventSource	132
Demonstration and behaviour	134
Summary	134
Case Study – Implementing WebSockets for Real-time Updates	136
Server Side – Setting Up WebSockets	136
Client Side – Receiving WebSocket Events	140
A Real-World Bug: Duplicate Cards	142
Making the State Update Idempotent	143
Summary	143
Chapter 19 – Optimizing Performance with HTTP Caching	145
Why caching matters	145
Cache-Control – how long a response stays fresh	146
ETag – detecting whether content changed	148
Last-Modified – timestamp-based validation	150
Vary – preventing incorrect caching	151
Putting it all together: a realistic request flow	153
Summary	156
Chapter 20 – Handling Runtime Errors in React	157
A quick comparison to try/catch	157

Understanding Error Boundaries	158
Wrapping a fragile component	160
Cascading error boundaries in a column	161
Application-level fallback	163
Summary	163
Chapter 21 – Designing for Accessibility	165
Why we need to consider accessibility at all	165
Accessibility-first system design principles	166
Accessibility checks as part of the testing architecture	167
Semantic HTML in real components	169
Keyboard navigation as a system concern	171
A more complex example: accessible drag and drop	171
My typical accessibility workflow	174
Summary	177
Chapter 22 – Implementing Performance Monitoring	178
Why performance monitoring matters	178
Understanding bundle size in practice	178
Making performance limits explicit with budgets	180
Measuring real user experience with Web Vitals	183
Performance as a continuous system concern	186
Summary	186
The Architectural Roadmap: From Components to Systems	188
Module 1: The System Design Mindset (Chapters 1–4)	188
Module 2: Building the Data Foundation (Chapters 5–9)	188
Module 3: Managing Data Flow and Connectivity (Chapters 10–12)	189
Module 4: Rendering and Performance (Chapters 13–15)	189
Module 5: Mutations, Real-Time Sync, and Resilience (Chapters 16–22)	189
Designing System Step By Step	190

Preface

I've always enjoyed learning new concepts, and more importantly, I want to see abstract ideas in action, in a tangible and practical way. Concise and beautiful abstractions only make sense to me *after* I've seen the details and understood them clearly in my head.

Before I graduated from university, I had the chance to work as an intern at a company. During that internship, my mentor Dong gave me a few tasks, and one of them was to implement a thread pool in Java.

Until that moment, thread pools were still a mystery to me—something I only knew from textbooks. I recognised the diagrams, had a rough idea of how they behaved, and had seen some pseudo-code. But watching Dong implement a thread pool using a simple `List` in Java completely changed how I understood programming concepts. It was one of those moments that reshaped how I learn.

Later, while preparing for system design interviews, I realised how difficult it is to find practical, hands-on explanations—especially in the frontend space. Many tutorials and whiteboard explanations make sense on the surface, but I often walked away feeling unsure, sometimes with even more questions than before. If I couldn't see how something worked in a real environment, I knew I didn't fully understand it.

That's the gap I want to fill with this book (or course, or series—whatever this eventually becomes). I want to teach these concepts through concrete examples: runnable, interactive code you can play with. Examples where you can add a `console.log`, change a line, observe a network request, and immediately understand what's happening.

I truly believe this kind of tangible experience is what helps you internalise concepts, explain them confidently in a system design interview, apply them correctly in your projects, and contribute to complex systems at scale.

A High-Level View of Frontend Work

If we simplify everything, most frontend work falls into two categories: **reading** data and **writing** data. Frontend and backend systems must stay in sync. You might change some data and need to see your updates reflected, or you might need to react to changes made by someone else.

To deliver a smooth user experience around these two tasks, we have several opportunities. Some improvements happen at build time, some at the infrastructure level, and others directly at runtime in our application code.

At build time, for example, we can minify JavaScript and CSS into smaller bundles to reduce transfer time. On top of that, we can configure the server to use gzip or other compression methods, add ETags to improve caching, and keep assets on the client for longer so we don't fetch them on every page load.

At the code level, we can optimise rendering. In React, that might mean memoising components with `useCallback` or `useMemo` to prevent unnecessary re-renders. Or we can preload data when the user hovers over an important element, like a user avatar.

All of this contributes to a smoother, less janky experience. But building applications like this isn't easy. It requires understanding many subtle but important concepts. Even in the AI era, where LLMs can write code or tests for us, knowing *what* to ask for still matters. The quality of your prompt—or even the request you send to an API—makes a huge difference. Models can only help you when you understand what you're doing. Otherwise, the best you'll produce is something mediocre.

Book Structure and Outline

This book is designed as a practical journey—from high-level architectural thinking to concrete, production-ready implementation. Each section builds on the previous one, using the same running example to show how real frontend systems evolve over time.

The content is organized into several modules, each with a clear focus:

- **The CCDAO Framework (Chapters 2–4)** This module introduces a lightweight thinking framework—**Collect information, Component**

structure, Data modeling, API design, Optimization strategies. You can treat it as a “fast-start mode” for frontend system design: a way to organize your thoughts quickly under pressure, whether you’re in a design discussion, debugging a complex feature, or preparing for an interview.

- **Data Modeling (Chapters 5–9)** Here we focus on the foundation of most frontend complexity: data. You’ll explore domain-driven thinking and why **normalization** matters on the client side. By treating your frontend store more like a relational database, you establish a **single source of truth** that makes updates predictable and scalable.
- **Data Fetching and Requests (Chapters 10–11)** This module addresses the realities of asynchronous systems. You’ll look at request lifecycles, cancellation, debouncing, and pagination strategies, including the trade-offs between **offset-based** and **cursor-based** pagination.
- **Performance Optimization (Chapters 12–15)** With a real Express backend in place, we move beyond component-level optimizations. You’ll introduce **Server-Side Rendering (SSR)** to improve initial load performance, then layer in **code splitting** and **prefetching** to reduce perceived latency and keep interactions fast.
- **Mutation and Real-Time Sync (Chapters 16–18)** This section focuses on keeping the UI responsive while data changes. You’ll implement **optimistic updates** to eliminate unnecessary waiting, and explore real-time synchronization using **Server-Sent Events (SSE)** and **WebSockets**.
- **Productionization (Chapters 19–22)** The final module shifts attention to long-term reliability. Topics include **HTTP caching**, **error boundaries**, **accessibility-first design**, and **performance monitoring** with Core Web Vitals—concerns that often determine whether a system holds up in production.

How to Make the Most of This Book

This book is most effective when approached with a **hands-on mindset**. The goal is not just to understand the ideas, but to make them part of how you think and work.

1. **Work with the Running Project** The entire book is built around a Trello-style board application. Don’t treat it as a static example—clone the starter repository, run it locally, and use it as your playground.

2. **Use the Git Checkpoints** Each major step in the book is captured as a Git tag. You can jump to any checkpoint to compare implementations, recover from mistakes, or inspect a complete solution for a specific chapter.
3. **Break Things on Purpose** Because the project uses Mock Service Worker (MSW) or a local Express server, it's safe to experiment. Add `console.logs`, tweak request logic, introduce bugs, and observe how the system responds. This is where many of the insights become concrete.
4. **Apply the Mindset Beyond the Book** Treat the **CCDAO framework** as a reusable mental checklist. Use it when designing new features at work, reviewing pull requests, or reasoning about unfamiliar code—not just when following the examples here.

Summary

Frontend system design may sound abstract, but every concept in this book maps directly to your day-to-day work. Whether you're fetching data, updating state, handling errors, or improving performance, the ideas we cover will help you build smoother, more reliable experiences.

My goal is simple: to help you truly see these concepts by applying them, experimenting with them, and making them part of your toolkit. Once you understand how they work in practice, you can use them confidently—whether you're building complex features, reviewing designs, or explaining your decisions in an interview.

Chapter 1 — Evolving from Components to Systems

Modern frontend development isn't just about rendering pixels. It's about co-ordinating data, state, and behaviour across components that interact almost like small services. In this chapter, we'll start from something deceptively simple – a user selector dropdown – and use it to show why **system design thinking** matters when you want to build reliable applications (and explain your decisions clearly in interviews).



System design thinking Looking beyond a single component to consider data flow, state, constraints, and trade-offs across the whole application.

Analyzing how simple requests turn complex

You get a ticket:

“We need a dropdown to select a user when assigning a task.”

It feels straightforward. You give it a two-hour estimate. But when you finally ship it, two days have passed. What happened? Let's unpack it.

Part I — How a small component grows

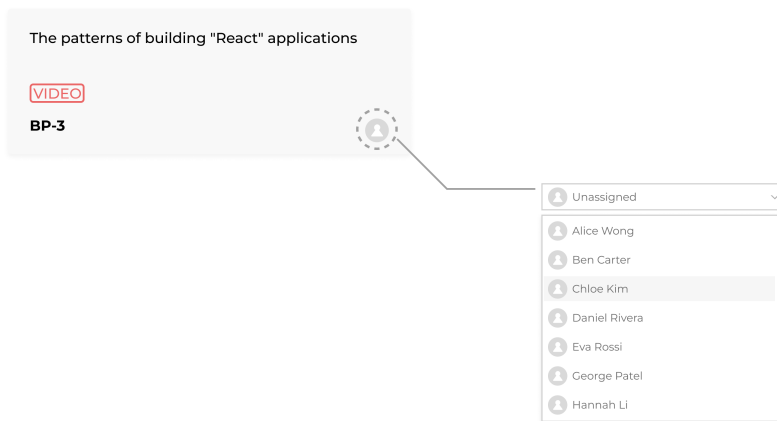


Figure 1. UserSelect component

Version 1 – The happy-path implementation

You might begin with the bare minimum: fetch some data and pass it into the dropdown.

```
1  const UserSelect = () => {
2    const [users, setUsers] = useState<User[]>([]);
3
4    useEffect(() => {
5      fetch("/api/users")
6        .then((r) => r.json())
7        .then((data) => setUsers(data));
8    }, []);
9
10   return <Select options={users} defaultValue="Unassigned" />;
11 };
```

This version works when everything goes right: the network is fast, the API behaves, and the data set is small. It's the version you might write in a coding exercise, or as a first pass.

Real production environments rarely stay that perfect for long.

Version 2 – Handling loading and errors

On a slower connection, the gaps start to show. When you open the dropdown, nothing happens for a moment. If the network fails, the whole component breaks. To make it robust, you add loading and error state:

```
1 const [isLoading, setLoading] = useState(false);  
2 const [error, setError] = useState<Error | null>(null);
```

You show a spinner when `isLoading` is true, and a friendly message when error is not null. The code isn't complicated, but the component now has more states to manage. Your “simple dropdown” is already tracking several transitions instead of just “data or no data”.

Version 3 – Scaling to large datasets

Everything seems fine until a customer with 2,000 employees reports that opening the dropdown freezes the UI. Rendering large lists and filtering them on the client side becomes noticeably expensive.

To make it scale, you introduce pagination and adjust the API:

```
1 const [page, setPage] = useState(0);  
2 fetch(`/api/users?page=${page}`);
```

This isn't just a local tweak anymore. You've changed the backend contract. A problem that started inside one React component now affects the API and possibly multiple teams.

Version 4 – Adding search

Next, customers ask for a search box so they can quickly find users.

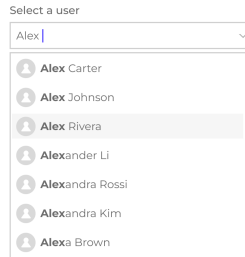


Figure 2. UserSelect component searchable

You add a controlled input and use its value in your request:

```
1 const [query, setQuery] = useState("");  
2 fetch(`/api/users?query=${query}&page=${page}`);
```

Typing “Alice” now sends a request on every keypress. That’s unnecessary load and can slow both the client and the server.

To fix it, you add debouncing.



Debounce Debouncing delays a function call until a certain time has passed without new input. It’s commonly used to avoid sending too many requests while the user is typing.

```
1 const debouncedQuery = useDebounce(query, 300);  
2 fetch(`/api/users?query=${debouncedQuery}&page=${page}`);
```

Now the request only fires after the user pauses typing for 300ms.

Version 5 – Avoiding race conditions

As the component becomes more dynamic, timing issues appear. QA finds a bug: they type “Alice”, then quickly change it to “Bob”. If the “Bob” request returns last, the UI is correct. But if the “Alice” response arrives later, it overwrites the results with stale data.

To avoid this, you cancel previous requests using `AbortController`:

```
1  useEffect(() => {
2    const controller = new AbortController();
3    const url = `/api/users?query=${debouncedQuery}&page=${page}`;
4
5    fetch(url, { signal: controller.signal })
6      .then((r) => r.json())
7      .then((data) => setUsers(data))
8      .catch((error) => {
9        if (error.name !== "AbortError") {
10          setError(error);
11        }
12      });
13
14    return () => controller.abort();
15  }, [debouncedQuery, page]);
```



AbortController A browser API that lets you cancel in-flight fetch requests, which is essential for avoiding race conditions in dynamic UIs.

This ensures only the latest request is allowed to update the UI. You’ve introduced another moving part, but it’s necessary to handle real-world user behaviour.

Version 6 – Accessibility considerations

An accessibility review reveals another gap. Keyboard users can’t reliably navigate the dropdown, and screen readers don’t announce options correctly. Fixing it means:

- Adding appropriate ARIA attributes
- Handling focus and blur events

- Supporting keyboard navigation (ArrowUp, ArrowDown, Enter, Escape)

Accessibility rarely means “just add a few attributes”. It often reshapes how the component is structured and how events are handled.

Version 7 – Internationalization

As the app expands to new regions, translations come into play. Labels need to be pulled from a translation system. Text might become longer, affecting layouts. Some languages require right-to-left layout support.

The same dropdown now has to respect locale, formatting, and text direction, on top of everything you’ve already added.

The real shape of the “simple dropdown”

If we pause here, the picture is clear. Our dropdown now deals with:

- Loading and error states
- Pagination and performance
- Search and debouncing
- Race condition prevention
- Accessibility
- Internationalization

What looked like a two-hour task quietly turned into something much larger. This isn’t over-engineering. It’s the natural shape of real-world requirements.

Zooming out to the system perspective

So far, we’ve only looked at one piece of UI. But in a real application, this user selector is only part of a bigger flow.

Changing the assignee of a task might:

- Update a card on a board and change its avatar
- Re-sort a list of tasks

- Append an entry to an activity log
- Trigger a notification
- Update a “My tasks” view on the assignee’s dashboard
- Push a real-time update to other connected clients

One tiny interaction ripples across many components and screens.

As soon as you zoom out, new questions appear:

- Should we update the UI immediately for responsiveness, or wait for server confirmation to avoid inconsistencies?
- If the request fails, do we roll back the UI or ask the user to retry?
- Where should we keep shared state: in a global store, React context, or local component state?
- How do we handle two users making conflicting updates at the same time?

These are not rare edge cases. They’re everyday concerns once you treat the frontend as a system with multiple sources of truth, asynchronous operations, and interconnected components.



Frontend as a system Once you zoom out, the frontend behaves like a distributed system: multiple views, shared state, network latency, partial failures, and concurrent updates.

This is where system design begins.

Developing system design thinking

Frontend has evolved from inline scripts to reusable components, and now to coordinated systems that must stay reliable under real pressure.

We deal with:

- Data modelling and shared state
- Data fetching and caching
- Data mutation and optimistic UI
- Performance and bundling strategies
- Accessibility and internationalization

- Error handling and resilience

None of these topics live in isolation. They shape each other and create trade-offs you need to reason about.

To make this easier to learn, this book is organised around a few core building blocks of frontend architecture:

- Data modelling and state management
- Data fetching
- Data mutation
- Performance optimisation
- Productionalisation (making things robust in real environments)



Optimisation strategies In this book, “optimisation” is not just about speed. It includes reducing complexity, improving reliability, and designing APIs and data flows that scale.

As you work through the chapters, you’ll build a mental model for how modern frontends are structured and how data moves through them. The goal is to help you reason about design decisions clearly – both in real projects and in interviews.

A note on CCDAO and the interview lens

Before we dive deep into each building block, the next two chapters take a slightly different angle.

If you’re preparing for system design interviews and don’t have time to absorb everything at once, it’s helpful to have a simple structure you can lean on under pressure. For that, we’ll use a lightweight framework called CCDAO.



CCDAO A practical framework for structuring frontend system design answers: Collect information, Component structure, Data modeling, API design, Optimisation strategies.

CCDAO is not the structure of this book, but it’s a useful lens for walking through open-ended questions. We’ll use two concrete examples and apply

CCDAO step by step, so you can see how to turn vague requirements into a clear, structured explanation.

Think of those chapters as a fast-start mode: a way to speak about frontend systems confidently, even before you've internalised all the details that follow.

Summary – The invisible work

Frontend complexity doesn't come from over-engineering. It comes from real-world conditions: slow and unreliable networks, large datasets, many users updating the same data, and the need to support different devices, languages, and accessibility requirements.

System design thinking gives you tools to understand these challenges, communicate your choices, and build software that holds up under pressure. When you learn to think in systems, you design more reliable products and explain your decisions more clearly.

In the rest of this book, we'll work from components up to systems, focusing on the key building blocks: modelling data, fetching it, mutating it, keeping it fast, and making it robust in production. Step by step, you'll see how all the pieces connect – and how to reason about them with confidence.

Chapter 2 — Introducing the CCDAO Framework

Preparing for a system design interview can feel daunting, even for experienced frontend engineers. Many developers spend most of their time refining features within a single product, not architecting systems from the ground up. That's why interview questions about designing scalable, maintainable, and performant applications can feel unfamiliar. But the gap isn't about intelligence or experience—it's about structure.

A structured framework gives you a way to reason through complex problems and communicate your thinking clearly. It ensures that you don't overlook key areas while also helping interviewers follow your logic. More importantly, it mirrors the way real frontend systems should be designed—methodically, with awareness of both technical trade-offs and user experience. That's where the CCDAO framework comes in.

CCDAO stands for **Collect Information, Component Structure, Data Modeling, API Design, and Optimization Strategies**. It's a practical, five-step framework you can use to approach any frontend system design challenge—whether in an interview or in your day-to-day work.

Collecting information for requirement clarity

Every strong design begins with understanding the problem. Before drawing diagrams or naming components, take a few minutes to clarify what you're building. Ask questions that reveal both the *functional* and *cross-functional* requirements.

Start with the **core functionality**. What exactly should the system do? What's the minimal set of features—often called the *steel thread*—that defines the essential user journey? Then move to **cross-functional needs**: performance goals, accessibility standards, security constraints, and expected scale. Will it handle thousands of concurrent users? Does it need to support real-time updates or offline access?

These clarifications shape every decision that follows. They help you focus on what matters most and demonstrate to interviewers that you're thinking beyond the happy path. In practice, developers who start by collecting information tend to make better architectural trade-offs later on.

When collecting information, focus first on the language of the domain, not the shape of the UI.

A “board app” that only talks about *columns* and *cards* is usually an oversimplification. Real systems quickly introduce richer concepts: users, filters, sprints, priorities, blockers, risk levels, tags, and more.

Your goal at this stage is not to design everything, but to extract enough domain vocabulary to have meaningful conversations. These terms will later influence your data model, APIs, and component boundaries.

This is where a **Domain-Driven Design** mindset helps: listen carefully to how the problem is described, reuse the same words in your design, and let the domain—not the UI—drive your structure.

Designing modular component structures

Once you understand the requirements, it's time to structure the interface. Imagine how the user interacts with the system and translate that into components. Even if you don't have a mockup, a quick sketch—on paper or mentally—is enough to identify the main building blocks.

Break the UI into logical parts: what belongs together, what can be reused, and how data should flow between components. Consider state ownership carefully—should a component manage its own state, or should it rely on a shared store or context? In interviews, describing these choices out loud helps the interviewer see how you reason about modularity and scalability.

In real projects, this step connects to architecture patterns like container-presentational separation or headless component design. Structuring components thoughtfully ensures flexibility as the system grows.

Modeling data for predictability and efficiency

After defining your component structure, you need to understand the data that powers it. What entities exist in your system, and how do they relate to each

other? For example, in a task management app, you might have users, tasks, and boards, each with relationships such as “a user owns many tasks.”

Good data modeling makes state management predictable and efficient. Decide whether to **normalize** or **denormalize** your data. Normalized structures avoid duplication and simplify updates, while denormalized ones can be faster to render but harder to maintain. Explain the trade-offs clearly if you’re in an interview.

Think about how the data flows: where it’s fetched, cached, and updated. Does your system support pagination for large datasets? Should it work offline with local storage or IndexedDB? These details not only show depth of understanding but also reflect real-world challenges frontend systems face daily.

Designing stable API contracts

Data modeling naturally leads to how your frontend communicates with the backend. This is where **API Design** comes into play. Define how the client retrieves, updates, and synchronizes data.

Consider whether the application would benefit more from **REST** or **GraphQL**. REST is simple and well-understood, but GraphQL allows for more flexibility in querying exactly what the UI needs. If the product requires real-time updates, discuss technologies like **WebSockets** or **Server-Sent Events** and how they integrate into your architecture.

Security is another critical aspect. Mention how you’ll handle authentication and authorization—using tokens like JWT or OAuth—and how you’ll protect the frontend from common vulnerabilities such as XSS or CSRF. Even brief comments on these areas show maturity and awareness of full-stack concerns.

Implementing optimization strategies early

Finally, you reach the stage where performance, resilience, and user experience come together. Optimization isn’t an afterthought—it’s an ongoing mindset that runs through the entire design process.

Think about **performance** at multiple levels. On the client side, techniques like **code splitting**, **lazy loading**, and **server-side rendering** can drastically

improve load times. On the network level, caching strategies—both client-side and server-side—reduce unnecessary requests. For large datasets, consider pagination or virtualization to keep the UI responsive.

Beyond raw performance, focus on **resilience and user experience**. How does your system handle network errors or server downtime? Are there loading states, skeleton screens, or retries in place? Accessibility and internationalization should also be treated as optimization problems: they expand your system's reach and usability.

In interviews, this is often where you can differentiate yourself by showing you've thought beyond functionality—considering how real users experience the application.

Applying CCDAO in an Interview

In a 45-minute system design interview, you won't have time to cover every detail. The goal is to demonstrate clarity of thought. Spend the first few minutes collecting information and clarifying requirements. Then move through component structure and data modeling, explaining how your design scales with complexity. Use the remaining time to discuss API design and optimization, showing awareness of performance and failure handling.

The exact time distribution doesn't matter as much as the flow. A good interview feels like a conversation guided by this framework, where you and the interviewer explore trade-offs naturally rather than following a rigid checklist.

Why It Matters Beyond Interviews

The CCDAO framework isn't just for interviews—it's a way of thinking that mirrors how robust frontend systems are built in practice. Each step represents a discipline that teams grapple with daily: gathering requirements, designing modular components, modeling data efficiently, defining stable APIs, and optimizing for performance and reliability.

If you invest time to study the concepts in this book—the patterns, trade-offs, and real-world examples—you'll go far beyond interview preparation. You'll develop the instincts of a true frontend architect: someone who can reason about complexity, communicate clearly, and design systems that stand the test of scale and time.

Before we move on, the next two chapters will show CCDAO in action. We'll walk through two concrete component designs step by step, applying the framework in real scenarios so you can see how each part translates into actual decisions in code.

Chapter 3 – Applying CCDAO:

Designing a Typeahead Search Box

To begin applying CCDAO to real features, I want to start with something compact but surprisingly deep: a typeahead search box. It’s a common interview question and a useful way to illustrate how a small UI element can reveal a whole set of design decisions.

At a glance, a typeahead feels simple – an input, a dropdown, and a request. But the moment you stop treating it as a toy component, the complexity appears. Different domains produce different result types. Network behaviour affects responsiveness. Keyboard navigation, accessibility, loading feedback, personalization – these all fall into the design space. This chapter is about tracing those ideas through the CCDAO lens, without restating what you’ve already read in earlier chapters.

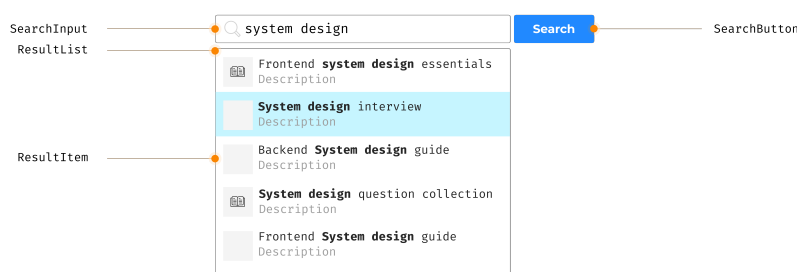


Figure 3. Typeahead component

1. Collect Information

For the typeahead, the first questions aren't about the UI at all – they're about understanding the environment you're designing for. The data domain matters: what is being searched, what counts as a valid suggestion, whether results come from a single index or multiple sources. Some products show trending or recent queries. Others merge personalized results with global ones.

Functional expectations also vary. Some typeaheads show suggestions on focus; others wait for input. Some support only mouse interaction; others require full keyboard semantics. In international products, locale-aware sorting can affect the order of results. In offline-capable apps, results may need to fall back to cached data.

These details change the shape of the feature, which is why collecting information upfront is essential. Later chapters will expand on domain-driven design and how early domain questions shape both data modeling and API boundaries.

2. Component Structure

Once I understand the problem space, I move to the UI design. A typeahead usually settles into a predictable component structure: a controlled input, a suggestion list that appears and disappears based on user interaction, individual suggestion items, and auxiliary states like loading, empty results, or errors.

The transitions between these states often matter more than the states themselves. When does the list open? When does it close? How do pointer interactions and keyboard navigation combine? What happens if a request finishes after the list has already closed?

When the logic grows beyond a couple of state variables, a dedicated hook helps keep responsibilities clear – handling request lifecycles, debouncing, highlighted index tracking, and visibility. This matches the philosophy of the CCDAO flow: establishing shape before solving details.

3. Data Modeling

For data, the typeahead relies on a simple unit – a suggestion – but the variations depend on the product. Suggestions may include icons, metadata,

or categories. Some results need to be grouped, which introduces a structural layer. Others merge local and remote results, which creates questions around weighting and ordering.

If the typeahead needs to remember prior queries or preloaded results, a normalized shape becomes useful. It reduces duplication, keeps the entities stable across renders, and makes caching naturally composable with the rest of the app’s state. These ideas will resurface in the chapters on state management and data mutation.

Pagination occasionally appears in typeahead systems with large result sets. Offset- or cursor-based strategies determine how the user progresses through results. Since pagination is a common interview topic, we’ll dedicate a full chapter to it soon.

4. API Design

The API surface for a typeahead can be as simple as a single query parameter:

```
1 GET /api/search?q=react&page=2&pageSize=8
```

Or it can return richer structures: grouped results, precomputed suggestions, personalized rankings. Real systems often mix multiple sources internally but present a unified response to the client.

Some products use GraphQL to give the frontend control over the data shape. For example:

```
1 query Search($q: String!) {  
2   search(query: $q) {  
3     id  
4     label  
5     url  
6     type  
7   }  
8 }
```

If this is your first time encountering GraphQL in the book, here’s the short explanation: a GraphQL query specifies the exact fields the frontend needs, and the server responds in that shape. The `$q` variable is supplied by the client.

This is especially helpful when the same search endpoint supports multiple use cases across the app.

We'll revisit pagination, cursor design, and request orchestration in the next chapters, since those topics apply more broadly to feed lists, notification panels, and search experiences.

5. Optimization Strategies

Once the logic works, the typeahead becomes a performance and user experience exercise.

Debouncing is essential for reducing noise. Cancelling stale requests prevents outdated results from flashing into view. Prefetching helps with responsiveness, especially when the user frequently types similar prefixes.

Predictability also matters. Caching recent queries reduces repeated work. Merging results consistently avoids reshuffling items as new data arrives. Normalized entities help reduce unnecessary rerenders when a single suggestion appears in multiple contexts.

Accessibility is a core part of the component. Arrow keys, Enter, Escape, ARIA roles, and focus management all contribute to making the typeahead usable by everyone. In international products, the ordering of results and the matching behaviour need to respect locale rules.

These ideas point forward to upcoming chapters on caching, request optimization, accessibility, and rendering predictability.

Closing

This typeahead example is the first practical application of CCDAO in the book. It's intentionally small, but it reveals how quickly a simple UI turns into a system design question once you consider domain rules, data flow, interactions, and performance guarantees.

This wraps up our first CCDAO example. You've now seen how the framework helps break a problem down, surface the real constraints, and guide the final design.

In the next chapter, we'll take the same approach with a different component – a `FeedList`. It introduces new considerations around pagination, data fetching, and real-time updates, and it's a good chance to see how CCDAO adapts when the problem shape changes.

Chapter 4 – Applying CCDAO:

Designing a Scalable Feed List

For this chapter, I want to apply the CCDAO framework to something much larger than the previous example. This time, we'll look at a pattern that almost every modern product relies on: the feed.

A feed sounds simple at first – show posts, let the user scroll. But the moment you treat it as a real feature rather than a toy example, the complexity reveals itself. Pagination, infinite loading, partial retries, real-time updates, scroll restoration, rendering cost – they all show up immediately.

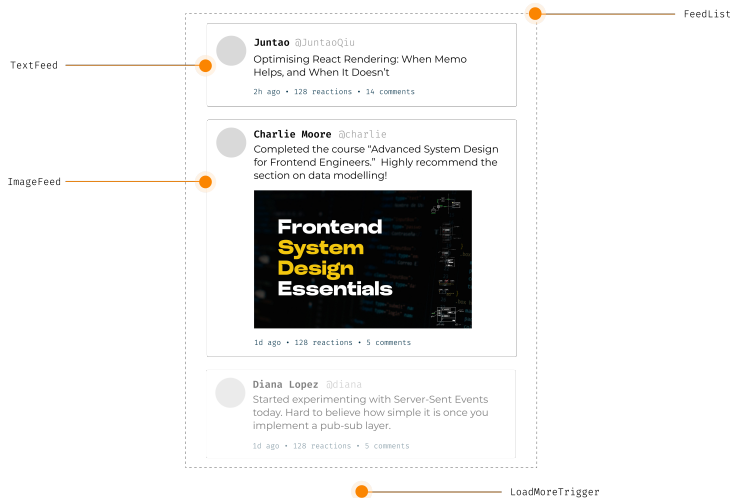


Figure 4. Feedlist component

Instead of going deep on every one of those topics right away, let's take a focused pass through the feature using CCDAO. Think of it as the first design meeting for your feed. Later chapters will pick up individual pieces – data fetching, caching, normalization, optimistic updates, virtualization – and go into detail.

1. Collect Information

With a feed, the first step is clarifying the shape of the experience. Before thinking about UI components or data flow, I want to understand what I’m designing for – not just the visible UI, but the domain it represents. We’ll talk about Domain-Driven Design properly in a later chapter, but we can already lean on the mindset here: surface the rules before committing to structure.

For example, mobile feeds often scroll continuously, while desktop versions might include sidebars or inline panels. Some feeds mix video, text, ads, polls, or inline forms. Others keep it simpler, with only one kind of item. Real-time updates may or may not be involved. Some feeds require offline behavior or degraded modes for poor network conditions.

And, importantly, interactions matter. A feed that supports likes and comments creates different data flows from one that is mostly static. A feed with pinned or sponsored content introduces ordering rules that the backend must respect. Even the first-page strategy influences loading behavior: whether to preload a few posts quickly or deliver a full batch.

None of these questions appear on the surface when you hear “design a feed list,” but they define the architecture you choose. This early stage is just about understanding the space you’re in.

2. Component Structure

Once I have some sense of the experience, I turn to the UI breakdown. For a feed, the structure tends to stabilize quickly around a few core pieces.

There’s the container responsible for orchestrating pagination and loading states. Inside it, each post becomes a dedicated unit – a `FeedItem` – responsible for layout and small local interactions. Loading placeholders fill in while the initial items appear. Error boundaries give the user a path to retry when a request fails. And near the bottom of the list, an invisible trigger component watches for scroll proximity and signals that the next page should load.

Different products add different layers around this core. Some have tabs for “Following” and “Recommended.” Others include contextual modules, like “People you may know.” Some products render author popovers, modals, or inline reply boxes. These variations change the visual composition but not

the underlying structure. What matters at this stage is establishing clear ownership: which part decides when to load, which part displays content, and which part handles user input.

The feed interface is mostly a long, incremental flow, and a clean structure helps keep the logic distributed in a predictable way.

3. Data Modeling

Feeds highlight an important modeling challenge: data that grows continuously while the user interacts with it.

A single item usually contains an ID, author information, content, timestamps, and engagement statistics. This is straightforward. The interesting part is how those items accumulate and change over time. Users scroll through multiple pages. New posts arrive at the top. Engagement numbers update unpredictably.

A raw array can represent this, but it leads to unnecessary work when items change. A normalized model helps the feed stay responsive as updates come in. Each post lives in a central store keyed by ID, while the feed itself only tracks the order of those IDs. This separates “what the item is” from “where it appears,” which becomes particularly useful when items appear in multiple feeds (highly likely in real-world systems) or when real-time data pushes updates into the existing list.

Pagination also fits into the model. Most real feeds use cursors rather than offsets because the ordering can shift. Cursors create stable continuation points even if new posts are inserted at the top. We’ll go deeper into cursor design in the data fetching chapter, but for the feed example, it’s enough to assume that items come in pages keyed by a cursor the API provides.

All of this contributes to a model that can support long-lived interaction – not just a momentary snapshot.

4. API Design

With the structure and model in place, the API becomes much easier to shape. A feed endpoint typically returns a slice of items along with a cursor that represents where to continue:

```
1 GET /feed?cursor=xyz
```

The response includes the items plus a new cursor. The frontend appends the new IDs to the list and requests the next slice when the user scrolls – a pattern that fits infinite scrolling well. Cursor-based pagination is the natural choice here because the content behaves more like a moving stream than fixed pages. We’ll cover pagination more deeply in its own chapter, so for now it’s enough to recognize that a cursor aligns with the user experience.

Some products also push updates from the server. New posts, edits, engagement count changes, or moderation actions may arrive through WebSockets or SSE. These changes drop neatly into the normalized model – no need to refetch the entire feed, just merge the updated entities.

In systems that mix different types of feed items – posts, ads, announcements – you may not want the frontend assembling results from multiple endpoints. This is where GraphQL or a BFF layer can simplify things by shaping the response exactly as the UI needs. A GraphQL feed query might look like:

```
1 query Feed($cursor: String) {  
2   feed(cursor: $cursor) {  
3     items {  
4       id  
5       content  
6       timestamp  
7       author {  
8         id  
9         name  
10        avatarUrl  
11      }  
12      stats {  
13        likeCount  
14        commentCount  
15        isLikedByMe  
16      }  
17    }  
18    nextCursor  
19  }  
20 }
```

We’ll explore pagination – including behaviours like prefetching, deduplication, and cancellation – in the next chapter. Here, the important idea is that infinite scrolling pairs naturally with cursor-based pagination and a normalized state model.

5. Optimization Strategies

Feeds live in the intersection of large datasets, constant updates, and long interaction sessions. This is where optimization makes the biggest difference to the experience.

Rendering is the most visible constraint. Rendering a large number of posts is expensive, especially when images or videos are involved. Virtualization helps by only rendering what the user sees. Combined with stable keys and memoization, this keeps scrolling smooth. The goal isn't to micro-optimize each component but to reduce the total work required as the list grows.

Fetching strategy also matters. Prefetching the next page before the user reaches the end prevents visible delays. Recovering from failures with retries is important on mobile networks. And in some products, caching the initial pages improves startup time.

For real-time interactions, the goal is to make changes feel immediate. Optimistic updates help likes or comments feel instant. If the server later reports a failure, the UI rolls back. Incoming real-time posts need to merge smoothly into the existing list without disrupting the scroll. A good model and a predictable merging strategy make this manageable.

There are also subtler concerns. When items resize – for example, when images finish loading – scroll position can jump. Preserving scroll stability requires coordination between measurement and rendering. These kinds of details define the quality of the feed more than any single API choice.

Later chapters on performance optimization, virtualisation, and real-time updates will unpack these techniques more deeply. Here, the aim is simply to trace the decision points through the feature using the CCDAO lens.

Closing

Designing a feed with CCDAO is a good example of how a familiar UI can turn into an involved system design problem. Pagination, real-time updates, state changes, rendering cost – they all show up quickly. But with a structured approach, the problem stays manageable.

Try applying CCDAO to another piece of your system – a notification panel, a message thread, or an activity timeline. The more you practice, the more

these structures become second nature.

With the framework in place, we're ready to dive into the core of the book. The next chapters explore the fundamental techniques that power scalable frontend systems, starting with one of the most important pieces: data modeling. How you shape your data influences everything else – your APIs, your state management, your cache behavior, and your rendering strategy. So that's where we'll begin.

Chapter 5 — Data Modeling:

Understanding the Domain and the UI

When we first learn frontend development, **data modeling doesn't seem that important**. You fetch some JSON, drop it into state, and render it on the screen. For a small demo app, that works fine. But as the application grows, cracks begin to show: state gets duplicated, updates fail to propagate across components, and new features become unexpectedly hard to represent.

That's when you realize: **data modeling isn't just a backend concern – it's fundamental to frontend architecture**. A good model shapes how you store, retrieve, and synchronize data. A poor one makes even simple changes painful.

Defining the domain clearly through business rules

Every solid model begins with understanding the **domain** – the world your software represents. The more you learn about that world, the more precise your model becomes.

Example 1 — Chat Application

A basic chat app seems simple: a *User*, a *Message*, maybe an *Attachment*. But real systems evolve quickly. You soon discover *Threads* that group messages, *Private Groups* that restrict membership, *Broadcasts* that can't be replied to, and *Archived Messages* that alter how history is displayed.

None of these appear in your first API response. You uncover them only by exploring how people actually use the product – by learning the domain's rules and exceptions.

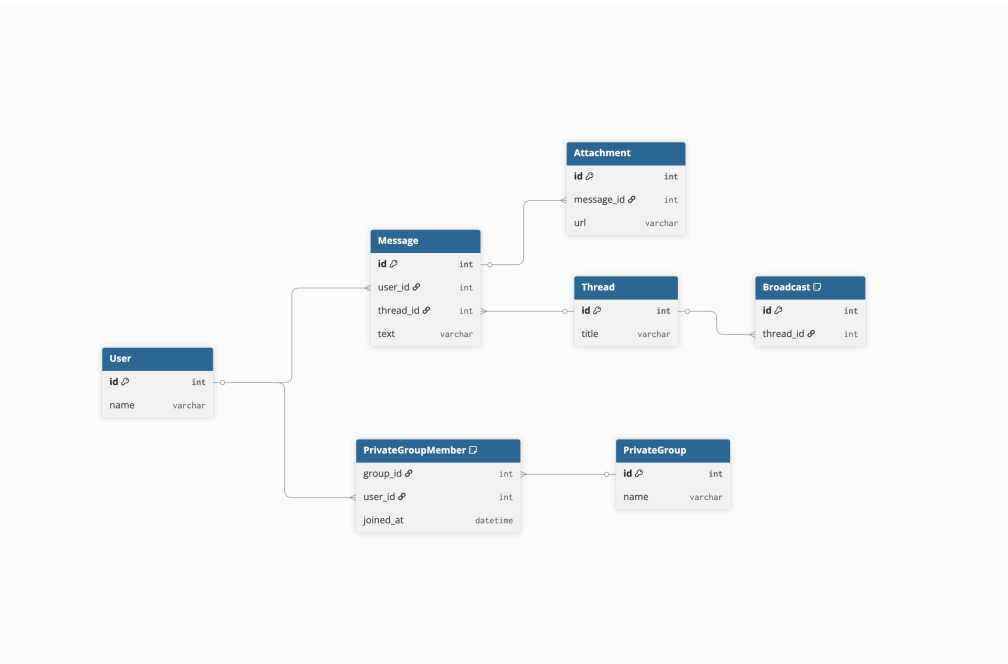


Figure 5. Chat application ER diagram

Example 2 – Online Course Platform

An online learning platform starts with obvious entities: *Course*, *Module*, *Lesson*, *Student*, *Instructor*. But even that’s incomplete.

A *Student* isn’t just a generic user – they’re *enrolled* in specific courses. Courses may have *Coupons*, *Expiry Dates*, or a *Drip Schedule* that unlocks lessons over time. You may need to track *Progress* or issue a *Certificate* when the course is finished.

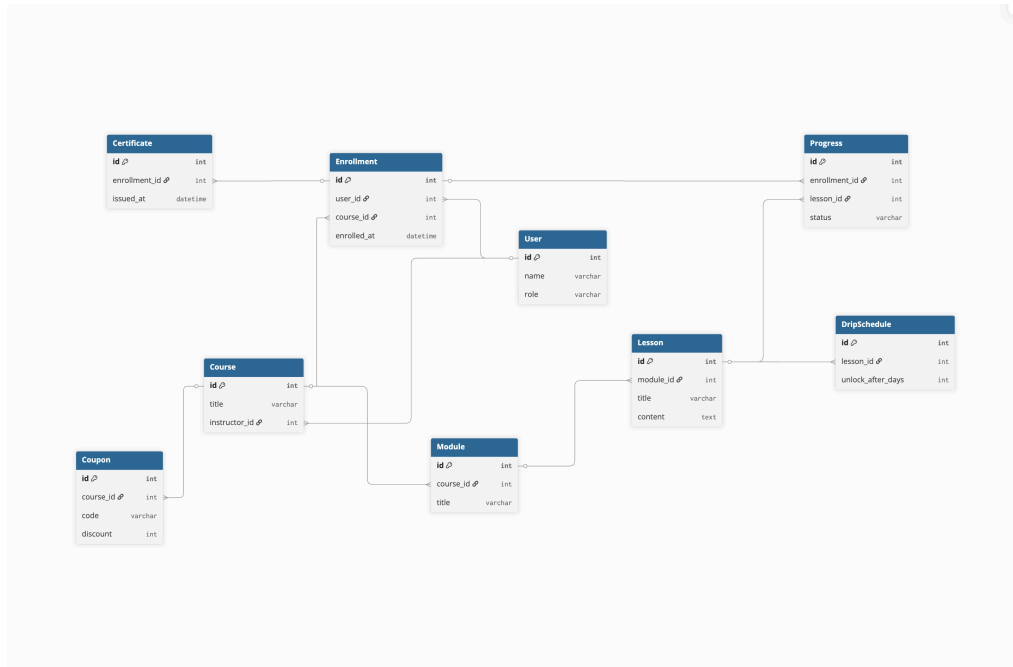


Figure 6. Course application ER diagram

The deeper you go, the more the model mirrors the business logic behind it.

Example 3 – Board Application

In our running example – a **board app** similar to Trello – the first design might look like *Board*, *Column*, *Card*. But soon you add *Assignees*, *Permissions*, *Workflow Stages*, *Comments*, and *Attachments*.

If you simply nest everything inside arrays, the code quickly becomes brittle. A small update in one part of the UI can break another. Instead, you need a model that reflects relationships clearly:

- A Board has many Columns.
- A Column has many Cards.
- A Card belongs to a User.

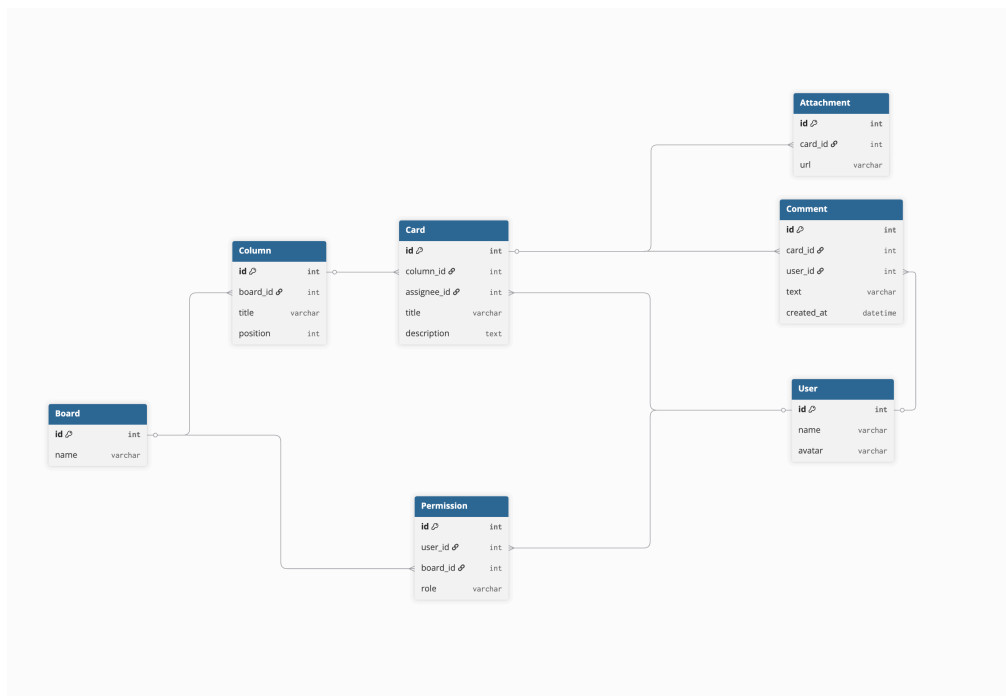


Figure 7. Board application ER diagram

Once you define that shared language, your codebase becomes easier to reason about – logic is clearer, bugs rarer, and collaboration smoother.

Designing models for specific UI consumption patterns

Understanding the domain is only half the story. As frontend engineers, our models must also reflect **how the UI consumes data**. APIs shouldn't dump everything at once; they should serve data in shapes that match the application's access patterns.

For example, rendering an entire board view might require a nested structure where each column contains its cards. Updating a single card, however, benefits from a **normalized** representation – storing entities by ID in separate tables so changes propagate consistently.

In other words, the *right* shape of data depends on **how the UI uses it**. The backend can remain consistent, but the frontend model adapts to optimize rendering, updates, and caching.

From Domain to Design

Before writing code, step back and ask a few guiding questions:

- What is the **end-to-end process** in this system?
- Who performs each action, and under what **rules or constraints**?
- Which relationships shape how data flows between features?

These questions shift your thinking from surface-level entities (*User*, *Message*, *Course*) toward concepts that better reflect real behavior (*Enrollment*, *Thread*, *Drip Schedule*). Once these domain ideas are clear, implementation details become much easier to reason about.



This style of thinking is closely related to ideas in **Domain-Driven Design (DDD)**. You don't need deep DDD knowledge here, but the core principle is helpful: let the *domain* guide how data and behavior are modeled.

To make this concrete, the next chapter introduces a small **case study**. It's intentionally unrelated to the board application – a compact example that's easier to examine on its own. In that version, a frontend sidebar decides which features to display by checking a `planType` directly in the UI. It works for a demo, but it doesn't scale well.

We'll look at how shifting those rules to the API – treating entitlements as data rather than UI logic – leads to a cleaner and more consistent model.

After that, we'll explore a few more design pieces before moving on to the project setup chapter. By the end, you'll see how thoughtful **data modeling** connects domain understanding with UI needs, creating systems that are predictable, scalable, and easier to extend.

Chapter 6 — Case Study: Implementing Sidebar Entitlements

In the last chapter, we saw how **data modeling connects the domain with the UI**. Now let's see that idea in action with a small but telling example – a **sidebar navigation**.

At first glance, it's just a list of links. But once we look closer, it becomes a great example of how data modeling decisions affect maintainability, scalability, and where logic should live in a system.

Step 1 — The Starting Point: Logic in the UI

Imagine our API returns a simple user object with a `planType`. In the frontend, we use that information to decide which navigation items to show:

```
1 const showFancyFeature = user.planType === "premium";
2
3 <Section title="Features">
4   <NavItem>Common Feature</NavItem>
5   {showFancyFeature && <NavItem emphasis>Fancy Feature ✨</NavItem>}
6 </Section>
```

This works fine – until it doesn't. If the product manager changes the rule (“now gold users get access too”), every frontend – web, mobile, and admin – must update the same condition and redeploy. That's a clear sign our **business logic has leaked into the UI**.

Step 2 — When the Rules Multiply

Let's make the example more realistic. Suppose the product team adds a new rule: *a premium user must also have been registered for more than three years* to see the feature.

```

1  const THREE_YEARS = 1000 * 60 * 60 * 24 * 365 * 3;
2
3  const showFancyFeature =
4    user.planType === "premium" &&
5    Date.now() - new Date(user.registeredAt).getTime() > THREE_YEARS;

```

Now our UI is doing more than presentation – it’s calculating business eligibility. That’s brittle and error-prone. If three different clients (web, mobile, admin) all reimplement these rules, it’s only a matter of time before they drift apart.

Step 3 — Moving Business Logic to the Backend

The real fix is to push these **entitlement rules** to the backend. The backend already knows which users qualify, so it can return the allowed features directly. That way, the UI becomes purely declarative – it just renders what the API provides.

```

1  {
2    "id": "u123",
3    "planType": "premium",
4    "features": {
5      "showCommonFeature": true,
6      "showFancyFeature": false
7    }
8  }

```

Our component then simplifies beautifully:

```

1  <Section title="Features">
2    {user.features.showCommonFeature && <NavItem>Common Feature</NavItem>}
3    {user.features.showFancyFeature && <NavItem emphasis>Fancy Feature
4      ↳ ✨</NavItem>}
5  </Section>

```

Now, if the business changes the rules, the backend updates them once and every client stays in sync. The frontend no longer “knows” the rules – it simply **renders the state of the world** as defined by the API.

This shift embodies a key data modeling idea: **move logic to the layer that owns the knowledge**. The backend owns business rules; the frontend owns presentation.

Step 4 – Where Frontend Logic Still Belongs

That doesn't mean every condition should live on the server. Some concerns are purely visual or contextual – for example, highlighting a new feature or showing a badge to draw attention.

```
1 <NavItem emphasis>
2   Fancy Feature ✨
3   {isNew && <span className="ml-2 text-xs text-orange-500">New</span>}
4 </NavItem>
```

Here, `isNew` is derived in the frontend – maybe based on a launch date or rollout flag:

```
1 const launchedAt = new Date("2025-08-01");
2 const isNew = Date.now() - launchedAt.getTime() < 7 * 24 * 60 * 60 * 1000;
```

This is **presentation logic**, not a business rule. The backend decides *who* can access the feature; the frontend decides *how* to display it. Separating the two keeps responsibilities clear and codebases cleaner.

Step 5 – Tailoring APIs with GraphQL or a BFF

In real-world systems, the API serving your frontend often supports multiple clients – mobile apps, partner integrations, or internal tools. You can't always reshape the main API just for your UI needs.

That's where **GraphQL** or a **Backend-for-Frontend (BFF)** architecture helps. They sit between your core services and your UI, exposing data in a shape that matches how the frontend consumes it.

For example, a GraphQL query might look like this:

```
1 query MenuFeatures {  
2   viewer {  
3     features {  
4       showCommonFeature  
5       showFancyFeature  
6     }  
7   }  
8 }
```

The business rules still live in the backend, but the frontend can query exactly what it needs – nothing more, nothing less. This approach maintains a single source of truth while allowing flexibility for each client.

Lessons from the Sidebar

This simple sidebar reveals a deep truth about data modeling: where you place logic shapes how maintainable your system becomes.

- When rules live in the UI, duplication and drift are inevitable.
- When the backend defines entitlements, every client stays consistent.
- When presentation logic stays in the frontend, the experience remains flexible.
- When APIs are shaped for consumption, both sides evolve independently.

By modeling data and responsibilities thoughtfully, you reduce coupling, improve scalability, and make both backend and frontend teams more autonomous.

In the next chapter, we'll extend this idea further. We'll explore how **API design and data contracts** evolve once you start aligning them with real UI access patterns – a crucial step in making frontend systems resilient and future-proof.

We'll use a board application as an example throughout this book, and all the theory, patterns and techniques will be based on the project. So as we go you will get the core ideas in these techniques and can apply them easily into your projects. Surely before that we'll need to setup the project so we can move forward.

Chapter 7 — Setting Up the Project Environment

Before we explore system design ideas, we need a common playground to experiment with. Throughout this book, we'll use a small **board application**—similar to a simplified Trello—as our running example. Every concept we cover later (pagination, normalization, caching, optimistic updates, prefetching, etc.) will build on this foundation.

Think of this chapter as preparing your workbench. Once the environment is ready, you'll be able to follow along, run each pattern locally, and see how the ideas behave in a real UI.

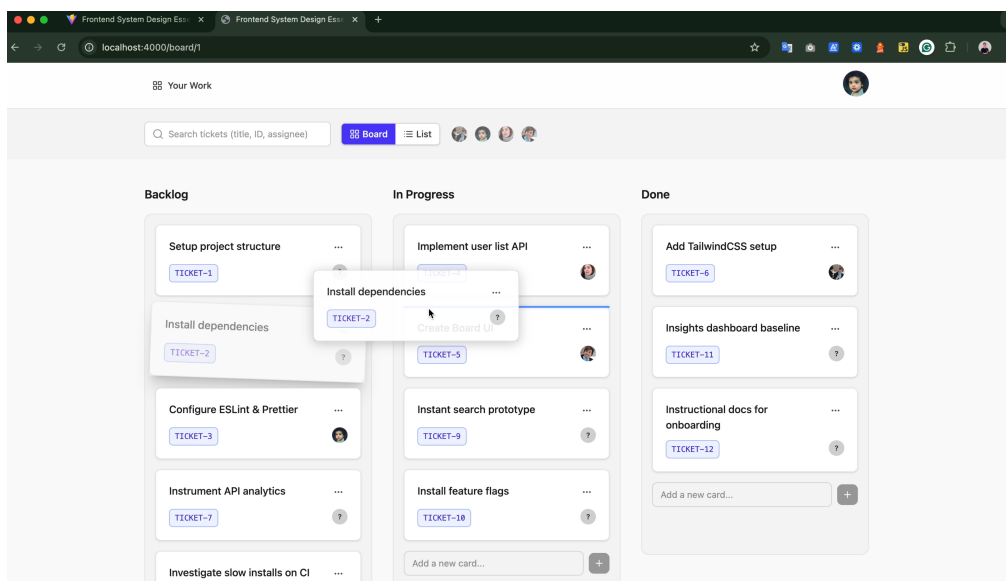


Figure 8. The complete app

You can find the starter project here: <https://github.com/abruzzi/frontend-system-design-essentials-starter>

- The default branch (starter) contains the minimal setup you'll use while reading through the book.

- The `main` branch shows the full implementation for reference.
- The repository also includes **tags** for each checkpoint so you can jump to specific moments in the project and compare changes.

With your environment ready, we can begin exploring the patterns that shape scalable frontend systems.

Why We Use a Starter Project

The goal of this book is to learn **frontend system design through practice**, not just through diagrams and abstract rules. Having a live application lets us:

- refactor real features instead of contrived examples
- see where performance issues actually come from
- compare different data shapes in the UI
- test behaviors like debounce, throttling, cancellation, mocking, and caching
- measure before-and-after improvements

The starter project is intentionally simple, but not trivial. It has enough structure to support real features, and enough imperfections that we can improve it together.

Prerequisites

You don't need much to get started—just a standard frontend setup.

You'll need:

- **Node.js** ≥ 20 (LTS recommended; this book uses 20.19.4)
- A package manager: **npm**, **pnpm**, or **yarn**
- Git

If you like keeping your system clean, you can use `nvm` to install and switch Node versions:

```
1 nvm install 20.19.4
2 nvm use 20.19.4
```

If you've run a Vite + React app before, you're already set. If not, the repository includes a small `SETUP.md` with optional links.

Getting the Project Running

Let's clone the repository and spin up the dev server.

```
1 # 1. Clone the repo
2 git clone
  → https://github.com/abruzzi/frontend-system-design-essentials-starter.git
3 cd frontend-system-design-essentials-starter
4
5 # 2. Install dependencies
6 npm install
7 # or: pnpm install / yarn install
8
9 # 3. Start the dev server
10 npm run dev
11
12 # 4. Open the URL printed by Vite
13 http://localhost:5173
```

You should see a basic board UI with a few columns and cards. We'll use this as our baseline for every chapter that follows.

Useful Scripts

The project keeps the scripts simple and predictable:

```
1 npm run dev      # Start Vite dev server
2 npm run build    # Production build
3 npm run preview  # Preview the production build locally
4 npm test         # Run tests
```

You won't need anything beyond these throughout the book.

Configuring the Mock API with MSW

Real APIs aren't required for this book. To keep the development loop fast and deterministic, the starter project uses **MSW v2** to mock backend endpoints directly in the browser.

This means:

- you can work entirely offline
- responses are consistent for every run
- we can introduce delays, errors, and edge cases easily
- tests use the exact same handlers as dev mode

The service worker

The worker file must exist here (already set up in the repo):

```
1 public/mockServiceWorker.js
```

If you ever delete the `public` folder by accident or see MIME-type errors, just recreate the worker:

```
1 npx msw init public --save
```

The worker is automatically started in development through `main.tsx`:

```
1 if (import.meta.env.DEV) {  
2   const { worker } = await import('./mocks/browser');  
3   await worker.start({  
4     onUnhandledRequest: 'bypass',  
5     serviceWorker: { url: `${import.meta.env.BASE_URL}mockServiceWorker.js` },  
6   });  
7 }
```

You don't need to modify this unless you want to customize behavior later.

Mock Endpoints Provided

The project includes several mocked endpoints we'll build on:

GET /api/users

Returns a static list of users.

GET /api/board/:id?q=<text>

Returns board data: columns + cards. Supports a simple search query:

- title
- card ID
- assignee name

Empty columns are hidden during searches.

GET /api/cards/:id

Fetches a single card and its parent column. Returns 404 if not found.

Handlers live in:

```
1 src/mocks/handlers.ts
```

Static mock data is in `users.json` and `board.json`.

Data Shapes You'll Work With

We keep the data intentionally lightweight—enough to support interesting behaviors without overwhelming you.

Board Payload

```
1 type UserLite = { id: number; name: string; avatar_url: string };
2 type Card = { id: string; title: string; assignee?: UserLite };
3 type Column = { id: string; title: string; cards: Card[] };
4 type BoardPayload = { columns: Column[] };
```

Users

```
1 type User = {
2   id: number;
3   name: string;
4   description: string;
5   avatar_url: string
6 };
```

Later chapters will refactor this shape, normalize it, cache it, prefetch it, and stream it in different ways.

Project Structure (High-Level)

Here's the folder layout you'll work with:

```
1 src/
2   components/
3     Board.tsx
4     BoardColumn.tsx
5     BoardControl.tsx
6     ListView.tsx
7     AssigneeList.tsx
8
9   mocks/
10    handlers.ts      # MSW API handlers
11    browser.ts       # setupWorker
12    users.json       # sample users
13    board.json       # sample board
14
15 public/
16   mockServiceWorker.js
```

As we progress, we'll extract utilities, introduce custom hooks, reorganize data-fetching, and reshape this structure step by step.

Troubleshooting

Here are a few common issues you might run into during setup:

1. “unsupported MIME type ‘text/html’ for mockServiceWorker.js”

Cause: The worker file is missing. **Fix:**

```
1 npx msw init public --save
```

Then refresh the page (or clear site data). Visiting `/mockServiceWorker.js` should show JavaScript—not HTML.

2. 404 errors for `/api/*` during development

- Ensure the worker is starting in dev (see `main.tsx`)
- Check the console for MSW startup messages

3. Port conflicts

Vite defaults to **5173**. If something else is using that port, Vite will pick the next available one. Check the terminal output for the correct URL.

What Comes Next

Now that your environment is ready, we can begin exploring the real focus of this book: **how to design reliable frontend systems**.

Starting in the next chapter, we’ll use this starter project to build:

- request control (debounce, throttle, race cancellation)
- pagination and infinite lists
- data normalization and cache strategies
- code splitting, lazy loading, prefetching
- mocking and testing techniques
- optimistic updates and rollback
- HTTP caching behavior

You’ll modify the project as you read, so by the end of the book you’ll not only understand the concepts—you’ll have applied each of them hands-on.

Chapter 8 — Implementing Data Normalization

Why Normalisation Matters

As frontend applications grow, we often end up holding data structures that look convenient at first but become difficult to maintain. A common pattern is **deeply nested backend responses** that duplicate the same entities in multiple places. This feels harmless early on – until you update the data in one part of the UI and discover the rest of the interface is now out of sync.

In this chapter, we'll walk through a concrete example from our Trello-style board. You'll learn why duplication becomes a maintenance problem, how normalising the data solves it, and how to build a simple shared store using React context to keep everything consistent.

A Real Example: Inconsistent User Data

To explore the problem, check out the example tag:

```
1 git checkout update-user-name
```

Then start the app:

```
1 nvm use
2 npm install
3 npm run dev
```

You'll see a small UI that lets you update a user's name. That same user also appears inside cards on the board.

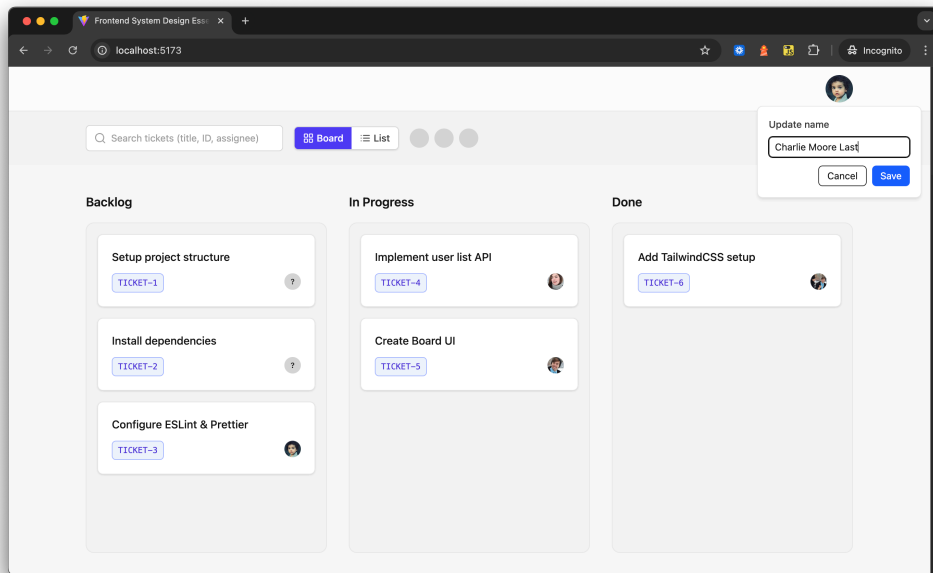


Figure 9. TopBar update user name

Here's the update logic inside `TopBar.tsx`:

```

1  const handleSaveName = async () => {
2    if (!user) return;
3
4    const res = await fetch(`/api/users/${user.id}`, {
5      method: "PATCH",
6      headers: { "Content-Type": "application/json" },
7      body: JSON.stringify({ name: newName }),
8    });
9
10   if (res.ok) {
11     const updated = await res.json();
12     setUser(updated);
13   } else {
14     console.error("Failed to update user");
15   }
16 };

```

The API returns a complete user object:

```

1 {
2   "id": 2,
3   "name": "Charlie Moore",
4   "description": "Charlie Moore is a sample user in the starter project user
   ↪ list.",
5   "avatar_url": "https://i.pravatar.cc/150?img=2"
6 }

```

Meanwhile, the board fetches its own payload:

```

1 {
2   "columns": [
3     {
4       "id": "col-1",
5       "title": "Backlog",
6       "cards": [
7         { "id": "TICKET-1", "title": "Setup project structure" },
8         { "id": "TICKET-2", "title": "Install dependencies" },
9         {
10          "id": "TICKET-3",
11          "title": "Configure ESLint & Prettier",
12          "assignee": {
13            "id": 2,
14            "name": "Charlie Moore",
15            "avatar_url": "https://i.pravatar.cc/150?img=2"
16          }
17        }
18      ]
19    }
20  ]
21 }

```

Here's the issue: **the same user is duplicated in multiple places** – one copy in the top bar, another copy inside the board payload.

When you update the user name in the top bar, the board still displays the old name. It holds a separate, outdated copy.



Hidden duplication Anytime the same entity appears in more than one place in your state tree, you're one update away from an inconsistent UI.

This is a real-world problem you'll encounter in any app that joins data from different sources or where multiple teams shape their own payloads.

Establishing a consistent source of truth

To prevent duplication, we use **normalisation** – the process of transforming nested, tree-shaped payloads into **flat dictionaries keyed by ID**.



What normalisation does

- Extracts each type of entity into its own dictionary
- Stores every entity *once*
- Replaces nested objects with ID references
- Makes updates predictable and consistent

Normalisation has several benefits:

- **One canonical copy** of each entity
- **Easy updates** – modify one place and the entire UI stays in sync
- **Predictable data flow**
- **Less redundant data** in memory
- **Cleaner rendering logic** (components hydrate data by ID)

There is some overhead: you do one shaping step upfront, and components perform simple lookups when rendering. But this small cost is quickly outweighed by the consistency and clarity it brings.

Transforming nested board payloads into flat tables

Let's start by converting the nested board structure into four flat “tables”:

- `usersById`
- `cardsById`
- `columnsById`
- `columnOrder`

```

1  export function normalizeBoard(payload: BoardPayload): NormalizedBoard {
2    const usersById: Record<number, User> = {};
3    const cardsById: Record<string, NormalizedCard> = {};
4    const columnsById: Record<string, NormalizedColumn> = {};
5    const columnOrder: string[] = [];
6
7    for (const col of payload.columns) {
8      const cardIds: string[] = [];
9
10     for (const c of col.cards) {
11       let assigneeId: number | undefined;
12
13       if (c.assignee) {
14         usersById[c.assignee.id] = c.assignee;
15         assigneeId = c.assignee.id;
16       }
17
18       cardsById[c.id] = {
19         id: c.id,
20         title: c.title,
21         assigneeId,
22       };
23
24       cardIds.push(c.id);
25     }
26
27     columnsById[col.id] = {
28       id: col.id,
29       title: col.title,
30       cardIds,
31     };
32
33     columnOrder.push(col.id);
34   }
35
36   return { usersById, cardsById, columnsById, columnOrder };
37 }

```

What this function achieves

- **Users** are extracted once and keyed by ID
- **Cards** reference an assigneeId instead of embedding the user
- **Columns** hold arrays of cardIds, not full objects
- **Column order** is preserved separately

This removes duplication and creates a clean, relational-style structure.

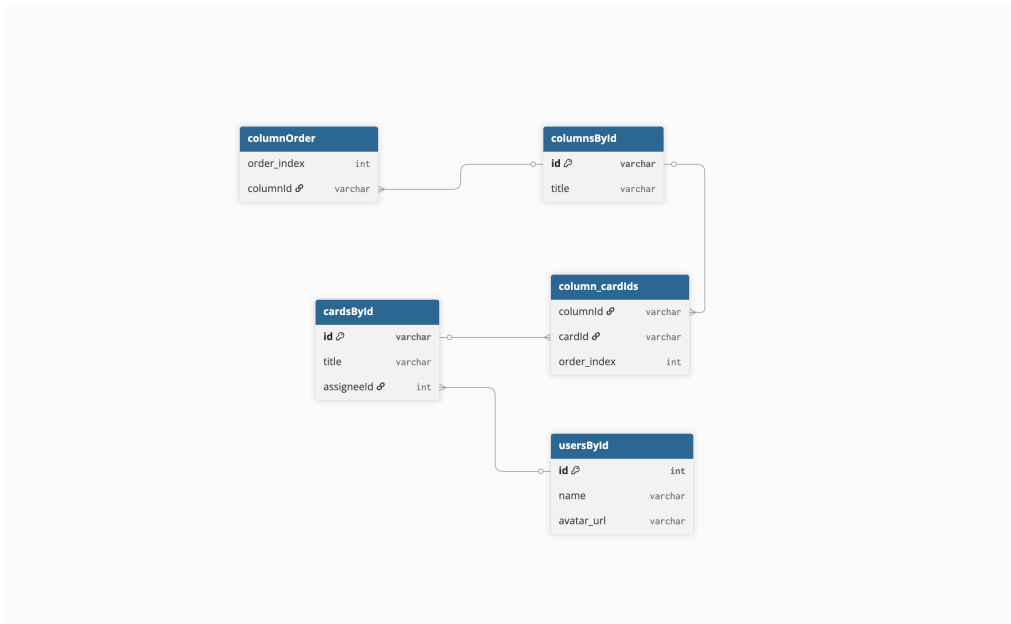


Figure 10. Normalised store

Example output

```

1 {
2   "usersById": {
3     "1": { "id": 1, "name": "Quentin Davis", "avatar_url": "..."},
4     "2": { "id": 2, "name": "Charlie Moore", "avatar_url": "..."}
5   },
6   "cardsById": {
7     "TICKET-1": { "id": "TICKET-1", "title": "Setup project structure"},
8     "TICKET-3": { "id": "TICKET-3", "title": "Configure ESLint & Prettier",
9       ↪ "assigneeId": 2 }
10  },
11  "columnsById": {
12    "col-1": { "id": "col-1", "title": "Backlog", "cardIds": ["TICKET-1",
13      ↪ "TICKET-3"] }
14  },
15  "columnOrder": ["col-1"]
16 }

```

Now there is **one source of truth** for the user with ID 2.

Holding the normalized store in React Context

To make this data available throughout the app, we create a context that exposes:

- The current state
- A method to ingest a full board payload
- A method to update or insert a user

```
1  const BoardContext = createContext<BoardContextType>({
2    state: initialState,
3    ingestBoard: () => {},
4    upsertUser: () => {},
5  });
```

The provider

```
1  export const BoardProvider = ({ children }) => {
2    const [state, setState] = useState<NormalizedBoard>(initialState);
3
4    const ingestBoard = (data: BoardPayload) => {
5      setState(normalizeBoard(data));
6    };
7
8    const upsertUser = (user: User) =>
9      setState((state) => ({
10        ...state,
11        usersById: {
12          ...state.usersById,
13          [user.id]: { ...state.usersById[user.id], ...user },
14        },
15      }));
16
17    return (
18      <BoardContext.Provider value={{ state, ingestBoard, upsertUser }}>
19        {children}
20      </BoardContext.Provider>
21    );
22  };
```



Why upsertUser? API responses often deliver partial user data. Merging updates ensures we preserve existing fields while applying new ones.

Ingesting data on load

```
1  const { ingestBoard } = useBoardContext();
2
3  useEffect(() => {
4    const url = `/api/board/${id}${search ? `?q=${encodeURIComponent(search)}` :
      ↪  ""}`;
5    fetch(url)
6      .then((r) => r.json())
7      .then((data) => ingestBoard(data));
8  }, [id, search]);
```

Updating a user

```
1  const { upsertUser } = useBoardContext();
2
3  const handleSaveName = async () => {
4    if (!user) return;
5
6    const res = await fetch(`/api/users/${user.id}`, { /* ... */ });
7
8    if (res.ok) {
9      const updated = await res.json();
10     upsertUser(updated);
11   }
12 };
```

Now every place that uses the user with ID 2 will see the updated name automatically.

Hydrating data during the rendering process

When rendering the board, we “join” the normalised data back into UI-friendly shapes:

```

1 export const BoardView = () => {
2   const { state } = useBoardContext();
3
4   return (
5     <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
6       {state.columnOrder.map((colId) => {
7         const column = state.columnsById[colId];
8
9         const hydratedCards = column.cardIds.map((cardId) => {
10           const card = state.cardsById[cardId];
11           const assignee = card.assigneeId
12             ? state.usersById[card.assigneeId]
13             : undefined;
14           return { ...card, assignee };
15         })};
16
17       return (
18         <section key={column.id}>
19           <h2 className="text-lg font-semibold">{column.title}</h2>
20           <BoardColumn cards={hydratedCards} />
21         </section>
22       );
23     })}
24   </div>
25 );
26 };

```

Notice that the assignee is always looked up from `usersById`. That means:

- **TopBar** writes user updates once
- **BoardView** reads the latest data everywhere

The UI stays consistent without duplicating logic.

Summary

In this chapter, we converted a nested backend response into a clean, normalised structure that eliminates duplication and keeps the UI consistent. We built a simple shared store using React context and used hydration at render time to assemble the final UI shape.

Normalisation adds a bit of upfront work, but it pays off quickly:

- No stale copies

- Predictable updates
- Clean data flow
- Easier reasoning as the app grows

As your application becomes more interconnected, normalisation becomes a quiet but essential foundation for a reliable UI.

Chapter 9 — Drawing Inspiration from Backend Databases

When we talk about *normalizing data* in the frontend, it can feel like a modern trick – something we picked up to make state management libraries behave. But normalization isn’t new at all. In fact, we’re borrowing a technique that backend databases have relied on for decades.

Understanding that connection helps the idea “click,” because we’re not inventing structure for the sake of structure – we’re aligning with a proven, well-understood model.

A Familiar Pattern From Databases

Let’s look back at our Jira-like board app. We’ve been working with four core entities:

- **Board** – has many columns
- **Column** – belongs to a board, has many cards
- **Card** – belongs to a column, may have one assignee
- **User** – can be assigned to many cards

If you were designing this for a backend database, you’d naturally break these apart into separate tables. A `Card` would store an `assignee_id`, not the full user object. A `Column` would reference its cards by ID instead of embedding every field over and over.

That’s normalization: **each piece of data is stored once, and relationships connect them.**

But to see why this matters, let’s have a look at the following illustration – the denormalized structure.

Ticket ID	Title	Column	Assignee	Assignee avatar
1	Setup project structure	Backlog		
2	Install dependencies	Backlog		
3	Configure ESLint & Prettier	In Progress		https://i.pravatar.cc/1507img+2
4	Instrument API analytics	In Progress		
5	Investigate slow installs on CI	Done		https://i.pravatar.cc/1507img+2
6	Instant search prototype	In Progress		

Board ID	Name	Type	Owner	Owner avatar
1	Frontend System Design	Kanban		
2	Travel (M)	GTD		https://i.pravatar.cc/1507img+2
3	I Code It - nextgen	Kanban		
4	AI Wrapper	Software		

Figure 11. Denormalised

In that model, a card doesn't just reference a user – it contains a full copy of the user's name and avatar. Now imagine the user changes their name. To apply that change, you must visit every card that embeds this user and update the duplicated fields one by one. It's easy to miss one, easy to create inconsistencies, and easy to break the UI in subtle ways.

The same problem appears across the system. If a board name changes, or a column label changes, you have to walk through nested objects hoping nothing gets out of sync. The more features you add, the more expensive every update becomes.

This pain is exactly why backend databases avoid embedding data in multiple places. They store each piece once and let relationships do the rest.

How Databases Reconstruct Data

When a backend needs to display a richer view – for example, cards together with their assignee details – it doesn't denormalize the tables permanently. It combines them on demand using SQL:

```

1 SELECT
2   c.id AS card_id,
3   c.title AS card_title,
4   u.id AS assignee_id,
5   u.name AS assignee_name,
6   u.avatar_url AS assignee_avatar_url
7 FROM cards AS c
8 LEFT JOIN users AS u ON u.id = c.assignee_id;

```

The database stays clean and efficient. The *result* of the query is the denormalized shape the UI needs.



Figure 12. Normalised database

Frontend normalization works exactly the same way.

The Frontend Equivalent

Here's a normalized store in the browser, following the same structure as relational tables:

```

1  {
2    "columnOrder": ["col-1", "col-2", "col-3"],
3
4    "columnsById": {
5      "col-1": {
6        "id": "col-1",
7        "title": "Backlog",
8        "cardIds": ["TICKET-1", "TICKET-2", "TICKET-3"]
9      }
10   },
11
12   "cardsById": {
13     "TICKET-3": { "id": "TICKET-3", "title": "Configure ESLint & Prettier",
14       ↪ "assigneeId": 2 },
15     "TICKET-4": { "id": "TICKET-4", "title": "Implement user list API",
16       ↪ "assigneeId": 5 }
17   },
18
19   "usersById": {
20     "1": { "id": 1, "name": "Quentin Davis", "avatar_url":
21       ↪ "https://i.pravatar.cc/150?img=1" },
22     "2": { "id": 2, "name": "Charlie Moore", "avatar_url":
23       ↪ "https://i.pravatar.cc/150?img=2" }
24   }
25 }

```

The parallels map one-to-one:

- `usersById` ⇨ the **users** table
- `cardsById` ⇨ the **cards** table
- `columnsById` ⇨ the **columns** table
- IDs ⇨ **foreign keys**
- selectors ⇨ the frontend version of **SQL joins**

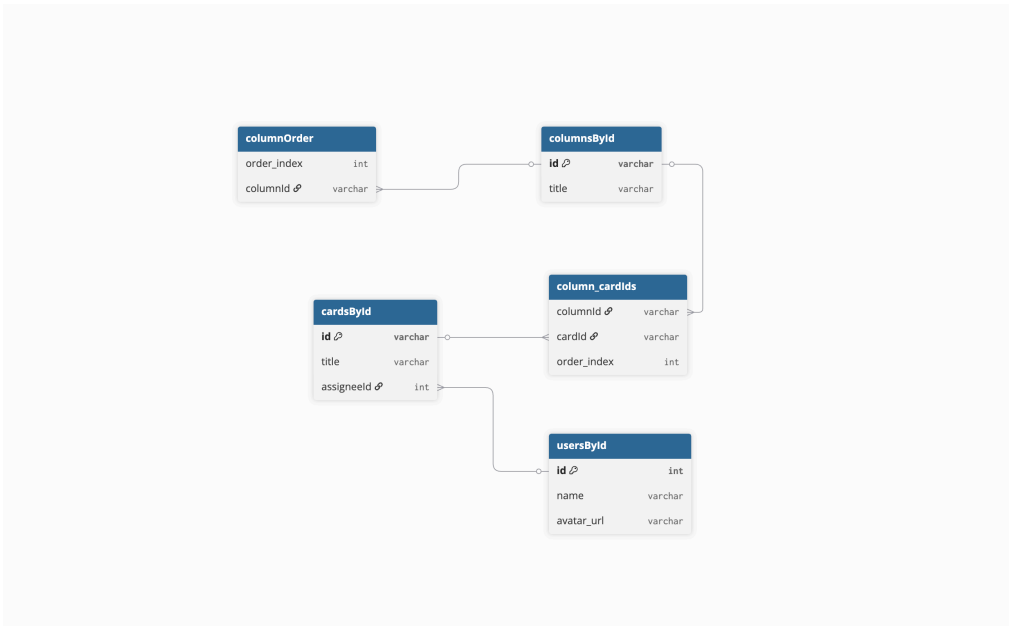


Figure 13. Normalised store

Normalized storage. Denormalized *views*. The same pattern, just applied in the browser.

Why This Matters

The important idea isn't that "frontend should copy databases." It's that both worlds converged on normalization because it solves the same core problems:

- Avoiding duplicated state
- Keeping updates consistent and predictable
- Making relationships easy to traverse
- Supporting efficient reads and writes

Backend databases reconstruct data using **JOINS**. Frontend apps reconstruct data using selectors, memoization, or cached queries.

Either way, the model stays clean – and the UI stays fast.

A Good Place to Pause

This closes our data modeling module. At this point, you've learned not just *how* to normalize frontend data, but also *why* the pattern works so well. In the next chapters, we'll build on this foundation and explore how normalized data unlocks powerful behaviors: efficient updates, optimistic UI, caching, pagination, and more.

The better your data model, the easier every upcoming system design technique becomes.

Chapter 10 — Managing Requests and Data Fetching

In this chapter, we shift our focus from data modeling to **how data actually travels through your application**. Fetching sounds simple – make a request, get a response – but the moment you add user interactions, dynamic queries, filters, and real-world network behaviour, the system becomes far more nuanced.

We'll begin with a straightforward example—adding assignee filtering to our board view—and use it to introduce essential request-management patterns such as **cancellation**, **debouncing**, and handling **race conditions**. These same principles show up in search boxes, filtering panels, dashboards, and any UI that reacts quickly to user input.

Showing Real Assignees in the UI

Our current AssigneeList component displays placeholder circles. To make it useful, we want it to show the real users from the store. The first version is simple:

```
1 export const AssigneeList = () => (  
2   <div className="flex items-center gap-2">  
3     <span className="inline-block h-7 w-7 rounded-full bg-neutral-300"  
4       ↪ aria-label={`assignee-1`} />  
5     <span className="inline-block h-7 w-7 rounded-full bg-neutral-300"  
6       ↪ aria-label={`assignee-2`} />  
7     <span className="inline-block h-7 w-7 rounded-full bg-neutral-300"  
8       ↪ aria-label={`assignee-3`} />  
9   </div>  
10 );
```

Replacing these placeholders is straightforward because our BoardContext already keeps track of user data:

```

1  import { useBoardContext } from "../BoardContext";
2
3  export const AssigneeList = () => {
4    const { state } = useBoardContext();
5    const users = Object.values(state.usersById).slice(0, 3);
6
7    return (
8      <div className="flex items-center gap-2">
9        {users.map((u) => (
10          <img
11            key={u.id}
12            src={u.avatar_url}
13            alt={u.name}
14            title={u.name}
15            aria-label={`assignee-${u.id}`}
16            className="inline-block h-7 w-7 rounded-full object-cover"
17            loading="lazy"
18          />
19        ))}
20      </div>
21    );
22  };

```

This gives us a real list of assignees. The next step is making each avatar clickable so that selecting a user triggers a board reload filtered by that assignee.

Extending the Backend API

To support filtering, we expand the `/api/board/:id` endpoint to accept an optional `assigneeIds` query parameter. Our MSW handler now parses that parameter and feeds it into the existing filter logic:

```

1  http.get("/api/board/:id", ({ request }) => {
2    const url = new URL(request.url);
3    const q = url.searchParams.get("q") ?? "";
4
5    // Parse assigneeIds parameter – comma-separated list of IDs
6    const assigneeIdsParam = url.searchParams.get("assigneeIds");
7    const assigneeIds: number[] = assigneeIdsParam
8      ? assigneeIdsParam
9        .split(",")
10       .map((id) => parseInt(id.trim(), 10))
11       .filter((id) => !isNaN(id))
12      : [];

```

```

13
14   const filtered = filterBoard(board as BoardPayload, q, assigneeIds);
15   return HttpResponse.json(filtered);
16 });

```

This handler does a few things worth highlighting. It extracts search keywords as before, but now it also reads `assigneeIds`, which may contain a single ID or a comma-separated list. It normalizes them into an array of numbers and hands those values to `filterBoard`, making the backend responsible for determining which cards match the selected users.

Updating the Board on Selection

With the backend ready, we adjust the board component to include the selected assignees in the query:

```

1  const { ingestBoard, state } = useBoardContext();
2  const selectedAssigneeIds = state.selectedAssigneeIds;
3
4  useEffect(( ) => {
5    const params = new URLSearchParams();
6    if (search) params.append("q", search);
7    if (selectedAssigneeIds.length > 0) {
8      params.append("assigneeIds", selectedAssigneeIds.join(","));
9    }
10
11   const url = `/api/board/${id}${params.toString()} ? `?`${params.toString()}` :
12     ↳ " "`;
13   fetch(url)
14     .then((r) => r.json())
15     .then((data) => ingestBoard(data));
16 }, [id, search, selectedAssigneeIds, ingestBoard]);

```

The store now keeps track of `selectedAssigneeIds`. Whenever the user toggles an avatar, this array updates and triggers a new board request. For simple interactions like clicking avatars, this approach works well—the user cannot click rapidly enough to overwhelm the system.

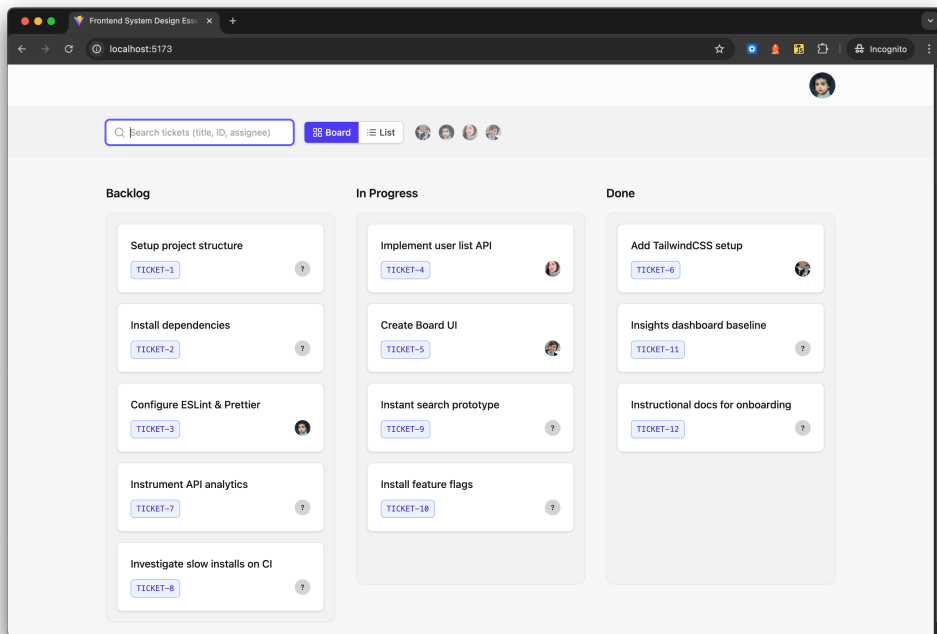


Figure 14. Search by assignee

Typing, however, is a different story.

Understanding Race Conditions in Search

Searching introduces a new dimension: speed. Users can type much faster than they click, and each keystroke can fire a request. Even worse, network latency is unpredictable, meaning **responses may arrive in the wrong order**.

To demonstrate, we inject artificial delays into the MSW handler:

```
1 const ms = variableDelay(q);  
2 await delay(ms);
```

The delay function exaggerates the timing difference:

```

1 function variableDelay(q: string): number {
2   const base = q.startsWith("inst") ? 150 : q.startsWith("ins") ? 650 : 300;
3   const jitter = Math.floor(Math.random() * 120);
4   return base + jitter;
5 }

```

This means queries for "ins" return significantly slower than queries for "inst". So if the user types quickly:

1. The request for "ins" is sent.
2. Immediately after, the request for "inst" is sent.
3. "inst" responds first.
4. Then "ins" responds later and **incorrectly overwrites** the UI.

This is the classic race condition: earlier requests finishing later and overwriting newer results. The UI ends up showing stale data even though logically the user already moved on.

Fixing Race Conditions with Request Cancellation

To solve this, every search request needs a cancellation mechanism. Abort-Controller is designed exactly for this scenario.

Inside `useEffect`, we create a controller and pass its signal into `fetch`:

```

1 useEffect(() => {
2   const ctrl = new AbortController();
3
4   const params = new URLSearchParams();
5   if (search) params.append("q", search);
6   if (selectedAssigneeIds.length > 0) {
7     params.append("assigneeIds", selectedAssigneeIds.join(","));
8   }
9
10  const url = `/api/board/${id}${params.toString()} ? `?${params.toString()}` :
    ↪  " "`;
11  fetch(url, { signal: ctrl.signal })
12    .then((r) => r.json())
13    .then((data) => ingestBoard(data))
14    .catch((e) => {
15      // Abort errors are expected; we only log actual failures.
16      if (e.name !== "AbortError") console.error(e);
17    });

```

```
18
19 // Cleanup: cancel the inflight request when dependencies change
20 return () => ctrl.abort("query changed");
21 }, [id, search, selectedAssigneeIds, ingestBoard]);
```

A few important ideas here:

AbortController A browser API that cancels ongoing operations. When `ctrl.abort()` is called, any fetch using `signal: ctrl.signal` stops immediately and throws an `AbortError`.

Cleanup function in `useEffect` React calls the cleanup whenever dependencies change. This ensures we cancel the previous request before sending the next one. With this, slow responses from earlier queries never overwrite newer data.

Silent abort handling Because aborting is expected behaviour, we avoid treating it as a real error.

This pattern ensures the UI stays consistent even under unpredictable network conditions.

Reducing Request Volume with Debouncing and Throttling

Cancellation solves correctness, but it doesn't solve efficiency. A user typing quickly can still fire many requests in a short time, which can put unnecessary pressure on both the client and the backend.

To smooth out this behaviour, we often rely on two simple techniques:



Debouncing waits until the user pauses before running the action. It's ideal for search boxes, auto-save behaviour, and input validation.



Throttling limits how often an action can run within a given period. It works well for scroll handlers, resize events, and drag interactions.

These techniques keep the UI responsive while avoiding a flood of unnecessary network requests.

For search boxes, debouncing is typically the right choice. Here’s a simple custom hook:

```
1 function useDebounced<I>(value: T, delay = 300) {  
2   const [v, setV] = useState(value);  
3  
4   useEffect(() => {  
5     const id = setTimeout(() => setV(value), delay);  
6     return () => clearTimeout(id);  
7   }, [value, delay]);  
8  
9   return v;  
10 }
```

Using it in `Board.tsx` is straightforward:

```
1 const debouncedKeyword = useDebounced(search);
```

Now your effect listens to `debouncedKeyword` instead of `search`. The UI updates instantly, but network requests only fire once the user pauses typing.

Together with cancellation, debouncing gives us:

- fewer unnecessary requests
- consistent results despite latency
- a smoother experience for fast typists

You can check out the `debounce-query` tag in the repo to experiment with this behaviour yourself. Use Chrome DevTools to observe how the request stream becomes far calmer and more predictable.

Summary

In this chapter, we explored request management through a practical feature: filtering the board by assignees and search keywords. A simple UI interaction quickly uncovered deeper concerns—race conditions, cancellation, and request volume.

By introducing **AbortController**, we ensured correctness: newer queries always win. By adding **debouncing**, we improved efficiency: fewer requests without sacrificing responsiveness.

These techniques are foundational. You will apply them again when building typeahead components, real-time dashboards, infinite scrolling feeds, inline editing, and any UI that responds dynamically to user intent.

In the next chapters, we'll build on this foundation as we explore pagination, caching strategies, infinite loading, and other patterns that make modern frontend applications feel fast and dependable.

Chapter 11 — Implementing Pagination Strategies

As your application grows, you eventually reach a point where loading and rendering “everything” simply stops working. A list that once held a dozen users might grow to hundreds or thousands, and if your frontend tries to render that entire dataset at once, you pay the price twice: the backend sends too much data, and the browser creates too many DOM nodes. Both slow down your app and make the UI feel heavy.

Pagination solves this by letting us fetch data in smaller chunks. Instead of downloading the whole list at once, we load just enough to keep the interface responsive and only request more when the user needs it. In this chapter, we’ll add pagination to the `UserSelect` component, and then explore the two major strategies used in real systems – offset and cursor pagination – and how they shape your UI patterns.

Demonstrating Pagination in `UserSelect`

Our current `UserSelect` fetches from `/api/users`, which returns the entire list. That works with a small mock dataset, but imagine there are a thousand users. Showing that many items inside a dropdown would make it almost unusable.

```
1  [
2    {
3      "id": 1,
4      "name": "Quentin Davis",
5      "description": "Quentin Davis is a sample user in the starter project
6      ↪ user list.",
7      "avatar_url": "https://i.pravatar.cc/150?img=1"
8    },
9    {
10     "id": 2,
11     "name": "Charlie Moore",
12     "description": "Charlie Moore is a sample user in the starter project
13     ↪ user list.",
```

```

12     "avatar_url": "https://i.pravatar.cc/150?img=2"
13 },
14 {
15     "id": 3,
16     "name": "Alice Gonzalez",
17     "description": "Alice Gonzalez is a sample user in the starter project
    ↪ user list.",
18     "avatar_url": "https://i.pravatar.cc/150?img=3"
19 },
20 //...
21 ]

```

To make the component scalable, we'll extend the backend so the endpoint accepts two new parameters: `page` and `pageSize`. The `page` tells us which chunk of data we want, and the `pageSize` defines the size of each chunk.

```

1 http.get('/api/users', async ({ request }) => {
2   const url = new URL(request.url);
3   const q = (url.searchParams.get('query') || '').trim().toLowerCase();
4   const page = Number(url.searchParams.get('page') ?? '0');
5   const pageSize = Number(url.searchParams.get('pageSize') ?? '5');
6
7   const source: User[] = q
8     ? (users as User[]).filter(u => u.name.toLowerCase().includes(q))
9     : (users as User[]);
10
11   const total = source.length;
12   const start = page * pageSize;
13   const end = start + pageSize;
14   const items = source.slice(start, end);
15
16   return HttpResponse.json({
17     items,
18     pageInfo: {
19       total,
20       page,
21       pageSize,
22       hasMore: end < total,
23     }
24   });
25 });

```

The response now returns two fields:

- `items`, which contains the slice of users for the current page

- `pageInfo`, which tells the frontend where we are in the dataset. It includes the current page number, the requested page size, the total number of users, and a simple `hasMore` flag so the frontend knows whether it should keep loading more data.

From the UI's perspective, this structure is enough to build both traditional pagination controls and infinite scrolling lists.

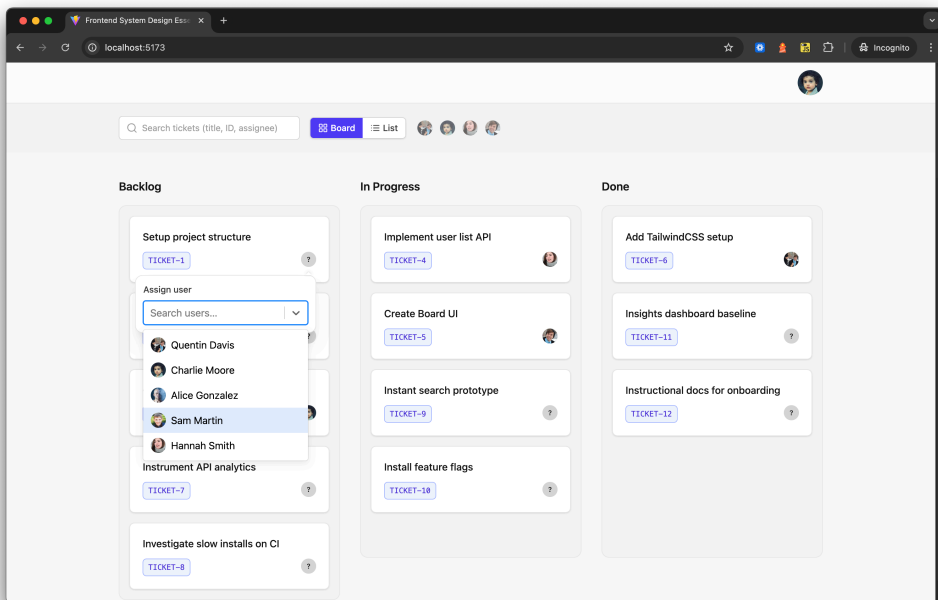


Figure 15. UserSelect with pagination

Inside `UserSelect.tsx`, we update the component so it fetches a new page whenever the page number changes:

```

1  const isFetchingRef = useRef(false);
2
3  async function fetchPage(nextPage: number, q: string) {
4    if (isFetchingRef.current) return;
5    isFetchingRef.current = true;
6    setIsLoading(true);
7    try {
8      const url = `/api/users?page=${nextPage}&pageSize=${pageSize}${q ?
      ↪ `&query=${encodeURIComponent(q)}` : ""}`;
9      const res = await fetch(url);
10     const data: {
11       items: User[];
12       pageInfo: { hasMore: boolean; page: number };
13     } = await res.json();
14
15     setOptions(prev =>
16       nextPage === 0 ? data.items : [...prev, ...data.items],
17     );
18     setHasMore(data.pageInfo.hasMore);
19     setPage(data.pageInfo.page);
20   } finally {
21     setIsLoading(false);
22     isFetchingRef.current = false;
23   }
24 }

```

Each time a new page loads, we unpack the `pageInfo` to know whether there are more users to load and what page we're currently showing. If we're on the first page, we replace the list; otherwise, we append to it.

We use a ref `isFetchingRef` here as a simple internal lock to prevent overlapping fetches. A state variable (like `isLoading`) wouldn't work as well because state updates are asynchronous and trigger re-renders we don't need. A ref updates immediately and doesn't cause re-renders, so it reliably guards the request flow even if the component re-renders or the user scrolls quickly.

The final step is triggering this function. When users scroll to the bottom of the dropdown, we fetch the next page. Since we're using `react-select`, we don't need to wire up our own scroll logic or an `IntersectionObserver` – the library gives us an `onMenuScrollToBottom` callback. In a custom list component, you could implement this yourself by placing a sentinel element at the bottom and observing it with the browser's `IntersectionObserver` (which fires when an element enters the viewport), but here the library handles that for us.



A quick note on IntersectionObserver:

This browser API lets you watch when an element becomes visible inside a scrollable container or the viewport. It’s often used to implement infinite scrolling: you place a small “sentinel” element at the bottom of the list, and when that element enters view, you trigger the next data fetch. Libraries like `react-select` expose this as a callback (`onMenuScrollToBottom`), so you don’t have to wire up the observer yourself.

With the mock API working as expected, let’s shift our focus to how pagination works behind the scenes in a real database.

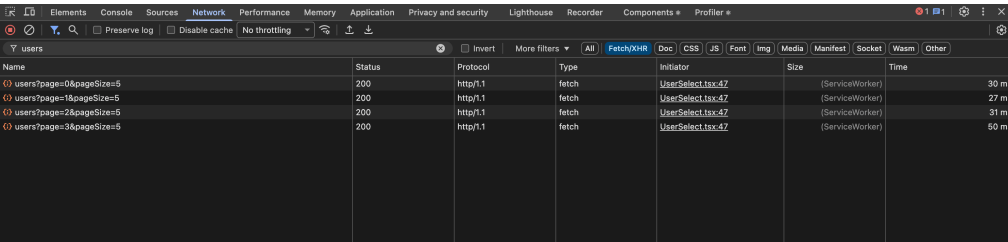


Figure 16. Pagination requests

You can also switch to the `pagination-user-list` tag to see the full example in context:

```
1 git checkout pagination-user-list
```

The Pagination Strategies

Pagination sounds simple, but how you implement it has a direct impact on performance, correctness, and even the UI patterns you can offer. A paginated table with numbered buttons, for example, expects stable pages – page 3 today should still be page 3 tomorrow. An infinite scroll feed, however, cares less about page numbers and more about loading continuously and efficiently.

To explore the trade-offs, we’ll look at the two main pagination strategies: **offset-based** pagination and **cursor-based** pagination. Both can serve the same `/api/users` endpoint, but they behave differently and come with different strengths.

A Simple Users Table

To ground the discussion, here's a basic SQLite users table:

```
1 CREATE TABLE users (  
2   id INTEGER PRIMARY KEY,  
3   name TEXT,  
4   description TEXT,  
5   avatar_url TEXT,  
6   created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
7 );
```

Imagine a sequence of rows ordered by `created_at`. Pagination queries typically operate over a column like this – unique, sortable, and indexed.

Offset Pagination

Offset pagination is the approach we just implemented in the mock API. It works by skipping a number of rows and then taking a slice. For example, the first page simply fetches the first 5 items:

```
1 SELECT id, name, created_at  
2 FROM users  
3 ORDER BY created_at DESC  
4 LIMIT 5 OFFSET 0;
```

The next page starts at row 6:

```
1 SELECT id, name, created_at  
2 FROM users  
3 ORDER BY created_at DESC  
4 LIMIT 5 OFFSET 5;
```

This is why offset pagination works well with UI patterns like:

- numbered pages (“1, 2, 3...”)
- a “go to page” control
- predictable table navigation

Those controls assume you can jump anywhere, and offset maps perfectly to that expectation.

However, offset pagination does degrade as your dataset grows. `OFFSET 10,000` still makes the database walk past those 10,000 rows before returning the next five. And when new users are inserted, the rows shift, so page 3 today might show a slightly different set tomorrow. This instability is usually acceptable for tables but becomes a problem for infinite scrolling components, which expect continuity.

Cursor Pagination

Cursor pagination approaches the problem differently. Instead of skipping rows, it uses a pointer – usually a unique, indexed field like `created_at` or `id` – and asks for “items after this cursor.” The first page looks identical:

```
1 SELECT id, name, created_at
2 FROM users
3 ORDER BY created_at DESC
4 LIMIT 5;
```

But for page 2, instead of calculating an offset, we pass the last item’s `created_at` as a cursor:

```
1 SELECT id, name, created_at
2 FROM users
3 WHERE created_at < '2025-08-05 09:05:00'
4 ORDER BY created_at DESC
5 LIMIT 5;
```

Cursor pagination keeps the dataset stable even when new users are inserted. A new row appearing at the top won’t shift all following rows; instead, it naturally appears as a fresh item on the next reload. Infinite scrolling UIs behave more smoothly with this approach, because the “load more” action always picks up right where the previous page ended.

The trade-off is that cursor-based systems don’t have a natural page number. You can move forward or backward, but you can’t jump directly to “page 5” because the concept no longer exists. This makes cursor pagination excellent for continuous lists (feeds, activity streams, chats) but less suited for spreadsheet-style interfaces with numbered controls.

Choosing the Right Strategy for UserSelect

In our UserSelect scenario, the pagination pattern sits somewhere between a dropdown list and a lightweight infinite scroll experience. The component doesn't show page numbers and doesn't let users jump around arbitrarily. Instead, it loads more users when the bottom is reached, which is conceptually closer to **infinite loading**.

This kind of UI works, but it also puts pressure on the backend. As the list grows, you want each “load more” request to be stable and efficient. Cursor pagination naturally fits this shape because each request always picks up exactly where the previous batch ended, even if new users are being added to the database.

Offset pagination still works fine in small datasets or mock environments like ours, but it's not ideal at scale or under high write traffic. Infinite loading patterns tend to reveal offset pagination's weaknesses much more quickly because duplicates, missing items, or unstable ordering are easier for users to notice.

More importantly, each pagination strategy aligns with a corresponding UI pattern, and this relationship helps you choose the right tool for the job.

UI Patterns for Pagination

When thinking about pagination, we're really thinking about how users *move through* data. The backend decides how to slice the dataset, but the UI decides how users experience that slice. Here are three common UI patterns that map to the strategies we've discussed.

1. Numbered Pagination

This is the traditional pattern: users see “1, 2, 3...” at the bottom of a table, can jump to any page, and often get a clear sense of “there are N total pages.”

This pattern aligns naturally with **offset pagination** because offset lets you move to any part of the dataset directly. It's a good fit for spreadsheets, admin consoles, analytics dashboards, and other UIs that emphasise structure and predictability.

2. Infinite Scroll

Infinite scroll loads more content as the user moves downward, creating the feeling of a continuous stream. Social media timelines use this model because the flow matters more than knowing which “page” you’re on.

This pattern aligns best with **cursor pagination**. A cursor ensures that each new chunk picks up exactly where the previous one left off. It also helps the backend avoid expensive large offsets, which is important for systems with massive datasets.

3. Infinite Loading

Infinite loading is similar to infinite scroll, but it’s not tied to the viewport. Instead, the user explicitly triggers the next fetch – usually via a “Load more” button, a dropdown reaching the bottom, or a controlled list that loads data as part of an interaction rather than direct scrolling.

Many real-world components use infinite loading, even when they aren’t full scrolling experiences. For example:

- A **dropdown** that loads more users each time you scroll near the bottom
- A **search results page** that shows the first 20 results and loads the next 20 only when the user clicks “Load more”
- A **chat sidebar** showing members of a group, loading more only when needed
- GitHub’s “Load more issues” button
- LinkedIn’s “Show more comments” interaction
- Slack’s channel member list when you scroll

Infinite loading also aligns naturally with **cursor pagination**, even though it’s not visually identical to infinite scroll. The interaction is controlled and predictable: whenever the component indicates it wants more data, the API returns the next slice using the stored cursor. The user’s mental model becomes “give me the next chunk,” which fits exactly how cursor-based pagination works.

Summary

Pagination isn't just a backend concern – each approach creates a different user experience. Numbered pages feel structured and predictable, infinite scroll feels fluid, and infinite loading offers a controlled middle ground. Cursor pagination pairs well with both infinite scroll and infinite loading because it keeps the stream of data stable and efficient. Offset pagination still has a place, especially when users need direct access to any page. Understanding these patterns helps you design a pagination experience that matches how users actually navigate your data.

Chapter 12 — Migrating to an Express Backend

Up to this point, we've relied on **MSW** to mock backend behaviour during local development. MSW is great for simulating API calls, testing components in isolation, and keeping the entire project self-contained. But as the examples in this book get more realistic – especially around update actions, server-side rendering, and more complex data flows – MSW starts to feel limiting.

For the next few chapters, we want a setup that behaves more like a real backend. So in this chapter, we'll migrate our mock API from MSW to a small **Express** server running alongside the frontend. This gives us full control over request/response behaviour, lets us simulate real mutation flows, and prepares us for topics like code splitting and server-side rendering in the upcoming chapters.

If you're already familiar with Express, you can skim this chapter and jump straight to Chapter 13. For everyone else, this is a short walkthrough to get the environment ready.

A Brief Introduction to Express

Express is a minimal web framework for Node.js. It provides a clean way to define routes, handle requests, send responses, and plug in useful middleware. You can think of it as a lightweight HTTP server where you decide exactly how each endpoint behaves.

A simple Express server looks like this:

```
1  const app = express();
2
3  app.get("/hello", (req, res) => {
4    res.json({ message: "Hello world" });
5  });
6
7  app.listen(4000, () => {
8    console.log("Server running on http://localhost:4000");
9  });
```

Express doesn't impose many conventions. You build your routes directly, add middleware as needed, and keep everything in plain JavaScript or TypeScript. That simplicity makes it perfect for a small mock backend like ours.

Setting up a standalone mock API server

Right now, all of our mock handlers live inside `src/mocks/handlers.ts`. When we run `vite dev`, MSW intercepts `/api/...` requests inside the browser using a service worker. For what we've done so far, this is enough.

To move beyond MSW, we'll set up a standalone server that listens on port **4000**:

```
1  const app = express();
2
3  // define all the handlers here
4
5  const PORT = process.env.PORT || 4000;
6  app.listen(PORT, () => {
7    console.log(`Mock API listening on http://localhost:${PORT}`);
8  });
```

Before defining our routes, we'll enable two common pieces of middleware:

```
1  app.use(cors());
2  app.use(express.json());
```

Middleware in Express runs **before** your route handlers. It lets you transform the request, run validations, add headers, or modify behaviour. Here, `cors()` enables cross-origin requests so the browser can call our server from

a different port, and `express.json()` lets Express automatically parse JSON request bodies – something we need for PATCH and POST endpoints.

Next, we migrate the handlers themselves. The code can move over almost unchanged. Here's the PATCH `/api/users/:id` handler:

```

1 app.patch("/api/users/:id", async (req, res) => {
2   const id = String(req.params.id);
3   const payload = req.body as Partial<User>;
4
5   const index = (users as User[]).findIndex((u) => String(u.id) === id);
6   if (index === -1) {
7     return res.status(404).json({ error: `User ${id} not found` });
8   }
9
10  (users as User[])[index] = { ...(users as User[])[index], ...payload };
11  return res.json((users as User[])[index]);
12 });

```

After moving all handlers into the new server, the folder structure looks like this:

```

1 |— index.ts
2 |— MockEventEmitter.ts
3 |— mocks
4 |   |— board.json
5 |   |— users.json
6 |— types.ts
7 |— utils.ts

```

For now, all route handlers live in `index.ts`. We'll cover `MockEventEmitter.ts` when we discuss data mutation in a later chapter.

To run the server:

```

1 cd server
2 npm run server

```

You can test it using `curl`:

```

1 curl http://localhost:4000/api/users

```

Or just open that URL in a browser to confirm the JSON response.

Proxying Frontend Requests to The Express Server

The last step is telling Vite to forward all `/api` requests to our new Express server. We do this through a proxy:

```
1 export default defineConfig({
2   server: {
3     proxy: {
4       "/api": {
5         target: "http://localhost:4000",
6         changeOrigin: true,
7       },
8     },
9   },
10 });
```

The proxy means:

- requests like `/api/users` never hit the frontend dev server
- instead, Vite forwards them to `http://localhost:4000/api/users`
- `changeOrigin` adjusts the request headers so the Express server sees them as coming from the correct host

With this in place, your frontend code doesn't need to change – it still calls `/api/...` exactly as before. The difference is that MSW is no longer handling those requests; our Express server is.

To run the full setup locally:

Terminal 1

```
1 cd server
2 npm run server
```

Terminal 2

```
1 npm run dev
```

You now have a real backend running alongside the frontend, ready for more advanced examples.

Summary

MSW has been great for bootstrapping our examples, but moving to an Express server opens the door to more realistic behaviours: updates, mutations, streaming, server-side rendering, and more control over how the backend responds. With this migration complete, the next chapter introduces several new data-fetching techniques that will make our application feel faster and more responsive.

Let's jump into it.

Chapter 13 — Implementing Server-Side Rendering

Rendering is one of the most important architectural choices we make in a frontend application. Up until now, our entire UI has been built in the browser. The server sends an HTML shell, the JavaScript bundle loads, and React renders everything after that. It works, but it delays the first meaningful paint and makes content invisible to crawlers until JavaScript runs.

In this chapter, we introduce Server-Side Rendering (SSR). Instead of waiting for the browser to assemble the UI, we generate the initial HTML on the server and send it to the user ready to display. Once the JavaScript bundle arrives, React hydrates that HTML so it becomes interactive. Before we implement anything, let's build a shared understanding of how different rendering strategies work.

Rendering strategies

Client-side rendering (CSR) is the familiar single-page app model: the browser receives a minimal HTML file and a JavaScript bundle that renders the entire UI. This gives smooth in-app navigation and simple deployment, but the first load can feel slow because nothing meaningful appears until the JavaScript runs. SEO is also limited.

Server-side rendering (SSR) reverses the sequence. The server renders the UI into HTML and sends it to the browser. Users can see content immediately, and search engines can index it. When the JavaScript bundle loads, React hydrates the page so it becomes interactive. SSR improves perceived performance and SEO, but requires more coordination between server and client.

Static site generation (SSG) goes one step further by building pages at compile time. The result is extremely fast to serve, but less suitable for pages that depend on constantly changing data.

Most modern applications mix these strategies. Some pages are static, some server-rendered, and some fully client-side. React 18 adds another tool:

streaming SSR, where the server sends HTML in chunks as the page renders. This allows faster first paint and smoother delivery of complex layouts.

How SSR works



Hydration is the process where React takes server-rendered HTML and “wires it up” with event listeners so the page becomes interactive. It’s not a re-render – it’s attaching behaviour to existing HTML.

Here is the high-level flow when a user visits `/board/1`:

1. The browser sends a request to our Express server.
2. The server fetches the data needed for that page.
3. React runs on the server and generates HTML.
4. The server sends that HTML to the browser.
5. The browser paints the content immediately.
6. The client bundle loads and hydrates the existing markup.

The important idea is that server and client must render the same component tree using the same data. If the server shows one thing and the client shows another, hydration warnings appear and React may re-render the whole page.

Coordinating hydration between server and client

Now that we have a mental model for how SSR works, let’s walk through how we bring it into our Vite + Express setup. The idea is simple: run the same React component tree in two different environments – once on the server for HTML, and once in the browser for hydration.

To make this happen, we introduce two entry points, one for each environment, and update the server to stream HTML instead of sending a blank shell.

Creating the client entry (hydration)

When the browser loads a server-rendered page, it already contains meaningful HTML. Our job on the client side is not to re-render everything, but to “attach” the component logic to that HTML.

That’s why the entry point changes from this:

```
1 createRoot(document.getElementById("root")).render(<App />);
```

to this:

```
1 hydrateRoot(  
2   document.getElementById("root")!,  
3   <BoardProvider initialState={initial}>  
4     <BoardPage id={boardId} />  
5   </BoardProvider>  
6 );
```

Hydration tells React to reuse the server-rendered HTML instead of creating new elements. The initial paint stays exactly as the server rendered it.

Hydration only works when the server and client render the same component tree. This is why we pass the initial data from the server through `window.__INITIAL_DATA__`.

Creating the server entry (rendering on the server)

Creating separate entry points

To support SSR, we split our application bootstrap into two files:

1. **entry-client.tsx** – runs in the browser and hydrates the page
2. **entry-server.tsx** – runs in Node and renders the initial HTML

Both files import the same application root, the same providers, and the same page component. The only real difference is *how* the tree is rendered.

The client entry uses `hydrateRoot`:

```

1 // src/entry-client.tsx
2 hydrateRoot(
3   document.getElementById("root")!,
4   <BoardProvider initialState={initial}>
5     <BoardPage id={boardId} />
6   </BoardProvider>
7 );

```

The server entry exports a function that returns a React stream:

```

1 // src/entry-server.tsx
2 export function renderApp(initialData: NormalizedBoard, boardId: string) {
3   return new Promise((resolve) => {
4     const { pipe } = renderToPipeableStream(
5       <BoardProvider initialState={initialData}>
6         <BoardPage id={boardId} />
7       </BoardProvider>,
8       {
9         onShellReady() {
10           resolve({ pipe });
11         }
12       }
13     );
14   });
15 }

```



These two entry points are the core of SSR: the server renders HTML, and the client hydrates the markup into an interactive React app.

Once we have these in place, Vite needs to know how to build both.

The server entry mirrors the client one. Instead of running in the browser, it runs in Node and generates HTML using React's streaming SSR API:

```

1 const { pipe } = renderToPipeableStream(element, {
2   onShellReady() {
3     resolve({ pipe });
4   }
5 });

```

Here, `element` is the same component tree as in the client entry:

```

1  const element = (
2    <QueryProvider>
3      <BoardProvider initialState={initialData}>
4        <BoardPage id={boardId} />
5      </BoardProvider>
6    </QueryProvider>
7  );

```



A helpful way to think about SSR is that your React app becomes “portable.” The same component tree can run in the browser or on the server – only the rendering mechanism changes.

Configuring Vite to build dual entries

Vite normally builds a single client bundle. With SSR, we ask it to build two separate bundles:

- a **client build** that includes JavaScript, CSS, and a manifest
- a **server build** that bundles React and our server entry into a Node-compatible module

Here is the full Vite configuration:

```

1  import { defineConfig } from "vite";
2  import react from "@vitejs/plugin-react";
3  import tailwindcss from "@tailwindcss/vite";
4
5  export default defineConfig({
6    plugins: [react(), tailwindcss()],
7    server: {
8      proxy: {
9        "/api": {
10          target: "http://localhost:4000",
11          changeOrigin: true,
12        },
13      },
14    },
15    build: {
16      outDir: "dist",
17      manifest: true,
18      rollupOptions: {

```

// needed so we know which JS/CSS files to load

```
19     input: "/src/entry-client.tsx", // client entry for hydration
20   },
21 },
22 ssr: {
23   noExternal: ["react", "react-dom"], // bundle React in the server build
24 },
25 });
```

Two things matter here:

1. `manifest: true` enables Vite's manifest file (`manifest.json`), which lists the final hashed filenames.
2. The `input` option tells Vite the browser entry point is `entry-client.tsx`.

For the server build, we use a separate npm script:

```
1 "scripts": {
2   "dev": "vite",
3   "build": "npm run build:client && npm run build:server",
4   "build:client": "vite build",
5   "build:server": "vite build --ssr src/entry-server.tsx --outDir dist/server"
6 }
```

This produces the following folder structure:

```
1 dist/
2   index.html
3   assets/...
4   .vite/manifest.json
5 dist/server/
6   entry-server.js
```



Think of the client bundle as “what the browser runs,” and the server bundle as “what Express uses to pre-render HTML.”

With these two outputs, the Express server now has everything it needs: the ability to **render HTML on the server** and the ability to **serve the correct client-side assets for hydration**.

Generating HTML on the server

When a request such as `/board/1` arrives, the server constructs a full HTML document, not just a React string. We use small helper functions to keep this readable:

```
1 startHTML(head);  
2 endHTML(serializedData, scripts);
```

These produce the opening and closing HTML tags, plus the script block that injects initial data into `window.__INITIAL_DATA__`.

This ensures that both server and client start from the same state.

Bringing it together in the Express SSR route

The SSR route is where everything comes together. The server follows a clear sequence.

1. Fetch initial data

This mirrors the client's `useEffect`, except it runs before rendering:

```
1 const initialData = await fetchBoardData(boardId);
```

2. Begin streaming HTML

The server entry returns a pipe function that represents the stream of generated HTML.

3. Send the HTML shell immediately

As soon as React signals that the shell is ready, we write:

```
1 <!doctype html>
2 <html>
3   ...
4   <div id="root">
```

This gives the browser something to paint right away.



Early shell delivery is the core performance benefit of streaming SSR. Users see meaningful content sooner, even if some components take longer to render.

4. Pipe the stream into the response

React produces HTML in chunks. The server forwards each chunk directly to the browser:

```
1 streamObj.pipe(responseWriter);
```

5. When rendering completes, inject data and scripts

We close the HTML and inject the hydration script:

```
1 <script>
2   window.__INITIAL_DATA__ = { ... };
3   window.__BOARD_ID__ = "1";
4 </script>
5 <script type="module" src="/assets/entry-client.js"></script>
```

This prepares the browser to hydrate the page once the client bundle loads.

Why consistency matters

Hydration only works when the HTML sent by the server and the tree rendered by the client are identical. Any discrepancy – timestamps, random numbers, layout variations – can break hydration.



The safest rule is: anything dynamic must come from the server and be passed to the client, never recomputed during hydration.

This is why the server serializes initial data into the HTML and the client reads it from `window.__INITIAL_DATA__`.

From SSR to SPA behaviour

Once hydration completes, the app behaves like a normal React SPA. Navigation is handled client-side, component updates happen in the browser, and no further server rendering occurs unless the user reloads the page.

SSR improves the first paint and first interaction, but everything after that remains the client-side application you already built.

Summary

In this chapter, we introduced Server-Side Rendering and React's streaming SSR model. We explored how the server renders HTML, how the browser hydrates it, and how this fits into our existing Vite + Express setup. SSR improves first-paint performance and SEO, while streaming SSR refines the experience by sending HTML progressively instead of waiting for the full page to render.

The important concepts – hydration, consistency between server and client, and serializing initial data – form the foundation for more advanced techniques like code splitting and interactivity on demand.

In the next chapter, we'll build on this rendering pipeline and look at how code splitting and lazy loading influence both server-rendered and client-rendered flows.

Case Study – Understanding Bundlers Through Code Splitting



Case study repository This chapter is based on a small, standalone repository: <https://github.com/abruzzi/code-splitting-case-study>

The code is intentionally minimal. Its only purpose is to make bundling behaviour visible and easy to reason about.

If your application feels slow to load, the cause is not always your component structure or rendering logic. Very often, the issue lies *before* the browser even starts executing your code – in how that code is bundled.

In this chapter, we'll use a concrete case study to understand how modern frontend bundlers work, and how design decisions in your codebase affect:

- what ends up in the initial bundle
- what gets removed entirely
- what is deferred until later

We'll focus on three core ideas: **dependency graphs**, **tree shaking**, and **code splitting** – not in theory, but as they appear in a real build.

What Problem Are We Solving?

Before looking at tools or optimisations, it's worth stating the problem clearly.

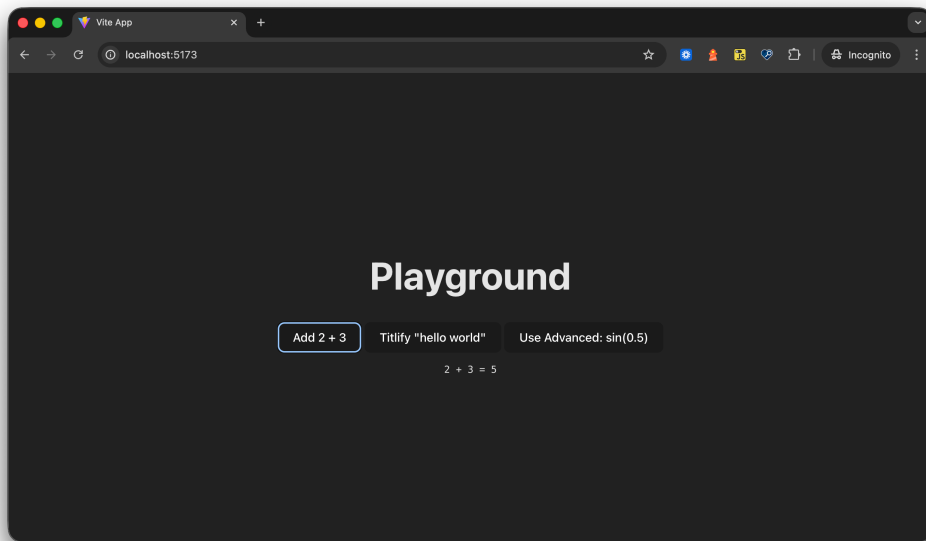


Figure 17. Playground

In this repo, the code is organised in a way most frontend developers would consider “clean”:

- small files
- focused utility modules
- explicit imports

You can see this under `src/utils/`:

- `math.js`
- `string.js`
- `advanced.js`
- `unused.js`

This structure is good for maintainability – but it raises an important question:

How does the bundler decide which of these files actually belong in the final output?

To answer that, we need to start with what a bundler really does.

What a Bundler Actually Does

When you run a production build, the bundler starts from a single place: the **entry file**.

In this repo, that file is:

```
1 src/main.js
```

This file is the root of the entire application. From here, the bundler:

1. Reads every `import` statement
2. Recursively follows those imports
3. Builds a dependency graph
4. Turns that graph into one or more JavaScript bundles



Context Although we're using Vite, the production build is handled by Rollup internally. In development mode, Vite uses native ES modules and does not bundle at all.

At this point, nothing has been optimised yet. The bundler simply knows *what depends on what*.

Now we can talk about what gets removed.

Tree Shaking: Removing Unused Exports

Tree shaking is the first major optimisation applied to that dependency graph.

In `src/utils/math.js`, several functions are exported:

```
1 export function add(a, b) { ... }
2 export function subtract(a, b) { ... }
3 export function multiply(a, b) { ... }
4 export function divide(a, b) { ... }
```

Now look at how this module is used in `main.js`.

Only **one** of these functions is imported and called.

Because the file uses **ES module syntax**, the bundler can statically analyse usage and conclude:

- `add` is required
- `subtract`, `multiply`, and `divide` are not

During the production build, those unused exports are removed entirely.



Key point Tree shaking works at the *export level*, not the file level – but only when imports and exports are static and explicit.

Where We Are Now

So far, we've established that:

- the bundler builds a dependency graph
- tree shaking removes unused exports from that graph

Next, we'll look at code that isn't removed – but still doesn't belong in the initial bundle.

Code Splitting: Deferring Code Until It's Needed

Tree shaking removes code you *never* use. Code splitting handles code you *do* use – just not immediately.

In this repo, that role is played by:

```
1 src/utils/advanced.js
```

This file exports more advanced functionality, but it is not imported at the top of `main.js`.

Instead, it is loaded using a **dynamic import**, triggered by user interaction.

Conceptually, the flow looks like this:

1. The app loads with a small initial bundle
2. The user clicks a button
3. Only then is `advanced.js` fetched and executed

From the bundler's point of view, a `dynamic import()` means:

“This module should live in a separate chunk.”

The result is two bundles:

- a main bundle for initial load
- a secondary bundle for advanced functionality



Design insight Code splitting is not about optimisation alone – it is a way to encode *user behaviour* into your bundle structure.

A Subtle Caveat: Tree Shaking Stops at Dynamic Boundaries

There is an important limitation hidden in this example.

Even though `advanced.js` exports multiple functions, and the code only calls one of them, **the entire module ends up in the split chunk.**

Why?

Because dynamic imports return a **module namespace object**. At build time, the bundler cannot know which properties you will access at runtime.

So it includes everything.



Important limitation Tree shaking does not remove unused exports *inside* dynamically imported modules.

If you want both:

- fine-grained tree shaking

- and deferred loading

you need to split the code further – for example, one file per advanced function.

This is not a tooling problem. It is a **module design decision**.

What This Case Study Teaches Us

Let's pause and connect the dots.

In this small repo, we've seen that:

- the entry file defines the root of the dependency graph
- static imports enable tree shaking
- unused exports are removed automatically
- dynamic imports create new bundles
- dynamic boundaries prevent export-level tree shaking

None of this required configuration tweaks. All of it emerged from **how the code was structured**.

Summary

This case study shows why bundling should be treated as a **system design concern**, not just a build step.

- Bundlers follow your imports, not your intentions
- Tree shaking rewards explicit, static module boundaries
- Code splitting maps well to user-driven interactions
- Dynamic imports introduce both power and trade-offs

Understanding these mechanics helps you design frontend systems that load faster *without* sacrificing clarity or maintainability.

In the next chapter, we'll build on this foundation and look at how bundling interacts with **lazy loading, prefetching, and perceived performance** – where timing becomes just as important as size.

Chapter 14 – Implementing Code Splitting and Lazy Loading

Modern frontend applications ship a surprising amount of JavaScript. Even if you optimise server-side rendering, a large initial bundle can still slow down how quickly the page becomes interactive. Users might only need a small part of your UI at first, yet they pay the cost of downloading and parsing everything upfront.

In this chapter, we look at a practical technique to ease that pressure: code splitting and lazy loading. The idea is simple—load only the code the user needs during the initial render, and defer the rest until the moment it’s actually required. We’ll apply this concept to the user profile popover in the top navigation bar. It’s useful, but not essential for the first paint, and therefore a strong candidate for lazy loading.

Why Lazy Loading Helps

Lazy loading improves the first impression of your application. By shipping less JavaScript at the beginning, the browser spends less time downloading, parsing, and executing code. This usually leads to a faster first paint and smoother hydration, especially on slower devices.

Many UI features aren’t needed during the initial render. Menus, drawers, modals, secondary navigation, and other interaction-triggered components can be loaded later without affecting the user experience. Lazy loading doesn’t change how the feature behaves; it simply removes unnecessary work from the critical path.

To make lazy loading work in React, a few concepts come together: dynamic import, `React.lazy`, and `Suspense`.

The Building Blocks of Lazy Loading

Lazy loading in React relies on a small sequence: a dynamic import signals to the bundler that the code can be split, `React.lazy` wraps that import into a component, and `Suspense` displays a fallback while the new code is being downloaded.

Dynamic import



Dynamic import A dynamic import is a runtime call to `import()` that returns a promise. Bundlers detect this pattern and create a separate chunk, letting the code load on demand.

Static imports are always included in the initial bundle. A dynamic import runs only at runtime. Because the bundler knows this code won't be needed immediately, it emits an extra JavaScript file that can be requested later.

`React.lazy`



React.lazy `React.lazy` converts a dynamic import into a component that React loads only when it tries to render it.

For example:

```
1 const UserProfilePopover = lazy() =>
2   import("./lazy/UserProfilePopover").then((module) => ({
3     default: module.UserProfilePopover,
4   }))
5 );
```

React won't load this code until the component is first rendered. If the module uses a default export, the mapping becomes even simpler.

Suspense



Suspense Suspense defines what to show while a lazy-loaded component is still downloading.

When React attempts to render a lazy component, it suspends that part of the tree until the code arrives. The Suspense fallback appears during this time. Once the module finishes loading, React swaps the fallback for the real component.

Implementing Lazy Loading in Our Board Application

Now that we understand the pieces, let's apply them to the user profile popover.

Step 1: Extract the component

Here is a trimmed-down version of the popover component:

```

1  import * as Popover from "@radix-ui/react-popover";
2  import type { User } from "../../types";
3
4  export const UserProfilePopover = ({ user, onUpdate }: {
5    user: User;
6    onUpdate: (user: User) => void;
7  }) => {
8    return (
9      <Popover.Root>
10        <Popover.Trigger asChild>
11          {/* avatar */}
12        </Popover.Trigger>
13
14        <Popover.Content
15          className="z-50 w-64 rounded-md border border-neutral-200 bg-white p-4
16            ↳ shadow-md"
17          side="bottom"
18          align="end"
19          >
20          {/* popover content */}
21        </Popover.Content>
22      </Popover.Root>
23    );
24  };

```

Because this component depends on Radix and handles interactive behaviour, it has a noticeable footprint. Since it's not needed immediately, we can safely load it later.

Step 2: Lazy load the component

In the top bar:

```
1  const UserProfilePopover = lazy(() =>
2    import("../lazy/UserProfilePopover").then((module) => ({
3      default: module.UserProfilePopover,
4    })))
5  );
```

React will now download this code only when needed.

Step 3: Wrap with Suspense

We provide a fallback that resembles the avatar itself:

```
1  <Suspense fallback={
2    <div className="h-10 w-10 rounded-full overflow-hidden">
3      <img
4        src={user.avatar_url}
5        alt={user.name}
6        className="h-10 w-10 rounded-full object-cover"
7      />
8    </div>
9  }>
10  <UserProfilePopover user={user} onUpdate={upsertUser} />
11 </Suspense>
```

A good fallback is small, visually stable, and close to the final layout.

Step 4: Coordinate SSR and hydration using useHydrated

React cannot server-render interactive popovers. To avoid hydration mismatches, we only render the lazy popover after the client has hydrated.

Here's the hook:

```
1 import { useEffect, useState } from "react";
2
3 export function useHydrated() {
4   const [hydrated, setHydrated] = useState(false);
5
6   useEffect(() => {
7     setHydrated(true);
8   }, []);
9
10  return hydrated;
11 }
```

And here is how it comes together:

```
1 const isHydrated = useHydrated();
2
3 {isHydrated ? (
4   <Suspense fallback={/* avatar */}>
5     <UserProfilePopover user={user} onUpdate={upsertUser} />
6   </Suspense>
7 ) : (
8   /* simple avatar for SSR */
9 )}
```

Before hydration, only the static avatar appears. Once hydration completes, React switches to the interactive version and triggers the lazy load.

Another Example: Lazy Loading the List View

The same approach also works well when you have multiple views and only one of them is needed at a time. In the board page, we originally imported both the board view and the list view statically:

```
1 import { BoardView } from "../BoardView.tsx";
2 import { ListView } from "../ListView.tsx";
```

Even though users typically start on the board view, the list view code was still included in the initial bundle. We can improve this by applying the same three steps from earlier: extract the component, lazy load it, and wrap it in `Suspense`.

Step 1: Move `ListView` into its own file

Place it under an `async` folder to make the intention clear:

```
1 src/  
2   async/  
3     ListView.tsx
```

The content of the component itself does not need to change:

```
1 export const ListView = () => {  
2   return (  
3     <div>  
4       {/* ... list view content ... */}  
5     </div>  
6   );  
7 };
```

By isolating the component in its own file, we give bundlers a clean split point.

Step 2: Convert the import to `React.lazy`

In the main `Board` component:

```
1 const ListView = lazy((=>  
2   import("./async/ListView.tsx").then((mod) => ({  
3     default: mod.ListView,  
4   })))  
5 );
```

React will now download the list view only when the user switches to it.

Step 3: Wrap the lazy component with `Suspense`

Inside the returned JSX:

```
1 {view === "board" ? (  
2   <BoardView />  
3 ) : (  
4   <Suspense fallback={<div>Loading...</div>}>  
5     <ListView />  
6   </Suspense>  
7 )}
```

When the user toggles the control, the list view component is rendered for the first time. That triggers the dynamic import. While the async chunk downloads, the fallback appears. Once the download completes, the list view takes its place.

This change keeps the initial bundle focused on the board view—the default experience—and moves the list view into a separate chunk that loads only when users explicitly switch to it. It follows the same pattern as the popover example but demonstrates how lazy loading also applies to larger sections of the UI, such as entire screens or alternate layouts.

Understanding the Build Output After Code Splitting

Once we add lazy loading to the popover and the list view, the production build output changes in a noticeable way. Before the change, Vite produced a single large client bundle:

```
1 dist/assets/entry-client-Be45_0Is.js    357.73 kB | gzip: 116.02 kB
```

This file contained **everything**: the board view, the list view, the popover, and all their shared dependencies. Even users who never switched to the list view or opened the popover had to download the code for both.

After enabling lazy loading, the build output splits into several files:

```
1 dist/assets/UserProfilePopover-DGu60DMQ.js    1.36 kB  
2 dist/assets/ListView-D_vFb7rc.js             1.79 kB  
3 dist/assets/entry-client-DH42TKWj.js         356.34 kB
```

The main entry file becomes slightly smaller, and Vite now emits two additional chunks. These new files represent the code for the lazy-loaded components. Importantly, they are not part of the initial page load. The browser will fetch them **only when the popover or list view is rendered for the first time**.

What Happens at Runtime

Putting these pieces together gives us a clear runtime sequence:

1. The server renders a simple avatar image, keeping the initial HTML light.
2. The client hydrates the application; `useHydrated()` becomes true.
3. React attempts to render the lazy component, triggering the dynamic import.
4. The Suspense fallback remains visible while the chunk downloads.
5. Once the async chunk arrives, React renders the full popover.

This approach keeps the initial load small while still providing the intended interactive experience.

How Code Splitting Changes the Bundle

Originally, the popover was bundled directly into the initial JavaScript file along with its library dependency. Every user downloaded it on page load.

After introducing lazy loading, the bundler produces an additional async chunk for the popover. The initial bundle becomes smaller, and the additional chunk downloads only when React needs it. The browser also caches this file separately, which benefits other pages that reuse the same navigation bar.

Summary

In this chapter, we explored code splitting and lazy loading, then applied them to the user profile popover in our board application. Dynamic imports, `React.lazy`, and `Suspense` work together to defer downloading optional UI. The `useHydrated` hook ensures that server-rendered HTML remains consistent with what the client expects, avoiding hydration issues.

We followed the complete runtime flow and looked at variations such as loading on user intent. Lazy loading reduces the initial bundle size, improves perceived performance, and helps keep your application scalable as it grows. Components behind interactions—popover menus, drawers, and similar patterns—are excellent candidates for this technique.

Chapter 15 – Implementing Data Prefetching

Even after we optimise JavaScript bundles and use lazy loading to defer code, the user interface can still feel slow for a simple reason: the data is not ready. A component may load instantly, but its data still needs to travel from the server, and that delay often appears as a spinner. Many interactions in a modern UI follow this pattern: the user clicks something, the UI opens a panel or dropdown, and then everyone waits.

Prefetching is a simple technique that helps you remove that wait. Instead of fetching data only when the user requests it, we fetch it a little earlier – usually when the user shows intent, such as hovering over a button or focusing an input. If the data arrives before the user takes the final step, the UI feels instant.

In this chapter, we'll build a small prefetch mechanism into our custom `QueryProvider` and use it inside `Card.tsx` to preload a list of users before the user opens the assignee picker.

How Prefetch Works Conceptually

Before looking at the code, it helps to clarify what “prefetch” means.



Prefetch Fetching data *before* the component that needs it is actually rendered, based on a reasonable guess of user intent.

The idea is simple. If we think the user is *likely* to open a panel, we can fetch the panel's data ahead of time. When the user finally interacts, there's no waiting.

Prefetching only works when the results are stored somewhere. Our `QueryProvider` already maintains a cache of results keyed by strings. Prefetch simply reuses the same cache and writes into it early.

The Prefetch Function Inside QueryProvider

Let's start with the prefetch function. It lives inside the QueryProvider and has access to the cache, which is just a Map<string, CacheEntry>.

Here is the simplified version:

```

1  const prefetch: PrefetchFn = useCallback(
2    async (key, queryFn, staleTimeMs = 60000) => {
3      const entry = cache.get(key) || {
4        data: null,
5        timestamp: 0,
6        isLoading: false,
7        error: null,
8      };
9
10     const now = Date.now();
11     const isStale = now - entry.timestamp > staleTimeMs;
12     const shouldFetch = (!entry.data && !entry.isLoading) || (isStale &&
13       ↪ !entry.isLoading);
14
15     if (!shouldFetch) return;
16     if (inFlightRequests.has(key)) return;
17
18     inFlightRequests.add(key);
19     updateCache(key, { isLoading: true, error: null });
20
21     try {
22       const data = await queryFn();
23       updateCache(key, {
24         data,
25         timestamp: Date.now(),
26         isLoading: false,
27         error: null,
28       });
29     } catch (error) {
30       updateCache(key, {
31         isLoading: false,
32         error: error instanceof Error ? error : new Error(String(error)),
33       });
34     } finally {
35       inFlightRequests.delete(key);
36     }
37   },
38   [updateCache, cache],
39 );

```

Let's go through the logic slowly.

Key	Value	Config	
<i>users::0:5</i>	{ ... }	60 seconds	⌚
<i>boards/1</i>	{ ... }	120 seconds	⌚
<i>users/2/settings</i>	{ ... }	60 seconds	⌚
...			

Figure 18. Simple Cache

The function starts by checking whether the cache already has an entry for the key. Each cache entry stores the data, a timestamp, whether it is loading, and any error.

```
1 const entry = cache.get(key) || { ...default values... };
```

Next, we check whether the existing entry is still fresh. If it is not stale, or if there is already an inflight request for this key, we skip fetching entirely. This is important because prefetch is often triggered repeatedly (for example, on every mouse enter). We do not want to launch duplicated network requests.

If the data is missing or stale, and nothing is currently fetching, we begin the request. We immediately mark the cache entry as loading and store a reference in `inFlightRequests` to prevent race conditions. We then call `queryFn()`, which is the function responsible for actually fetching the data.

Once the request succeeds, the data is placed directly into the cache. If it fails, we store the error in the cache instead. Either way, the entry for this key is updated and ready for `useQuery()` to consume.

Exposing Prefetch Through a Hook

We provide access to the prefetch function using a simple hook:

```
1 export function usePrefetch() {  
2   const context = useContext(QueryContext);  
3   if (!context) {  
4     throw new Error("usePrefetch must be used within QueryProvider");  
5   }  
6   return context.prefetch;  
7 }
```

Components can now call `usePrefetch()` and trigger prefetching whenever they choose.

Prefetching Users in Card.tsx

Let's see this work inside a real component. In `Card.tsx`, the user may open an assignee picker. This picker displays a list of users fetched from the backend. The default experience is that the user clicks the button, the panel opens, and then a spinner appears until the user list arrives.

Instead, we want to preload a small list of users as soon as the user hovers over the “assign” button. If the user actually clicks the button, the panel opens instantly.

Here is the code:

```
1 const prefetch = usePrefetch();  
2  
3 const prefetchUsers = () => {  
4   prefetch(`users::5:0`, () => fetchUsers(0, 5, ""), 60_000);  
5 };
```

Explaining the Cache Key

The cache key here is:

```
1 users::5:0
```

This is simply a string that uniquely identifies the request. We encode:

- the resource name (users)

- the page size (5)
- the page number (0)

This means “the first page of 5 users.” The exact format is not important so long as it is consistent. `useQuery()` and `prefetch()` just treat it as a lookup key.



Figure 19. Cache key

When we call:

```
1 prefetch(`users::5:0`, () => fetchUsers(0, 5, ""))
```

we are telling the `QueryProvider`:

- “Please fetch the first 5 users.”
- “Store the result into the cache under the key `users::5:0`.”
- “Consider it fresh for 60 seconds.”

The Fetch Function

The request itself is defined as:

```
1 export async function fetchUsers(page, pageSize, q) {
2   const url =
3     `/api/users?page=${page}&pageSize=${pageSize}` +
4     (q ? `&query=${encodeURIComponent(q)}` : "");
5   const res = await fetch(url);
6   if (!res.ok) throw new Error(`Failed to load users: ${res.status}`);
7   return res.json();
8 }
```

The only thing that matters for prefetch is that this returns a promise that resolves to the user list.

Triggering Prefetch on Hover

Here is how we tie everything together in the UI:

```
1 <button
2   type="button"
3   aria-label="Open assignee picker"
4   disabled={isUpdating}
5   onMouseEnter={prefetchUsers}
6   className="ml-2 inline-flex h-6 w-6 items-center justify-center rounded-full
   ↪   bg-neutral-300"
7 >
```

When the user moves their mouse toward the button, we prefetch the user list. The user might not click it – and that’s fine. Prefetch should be cheap, and the cache prevents unnecessary re-fetching.

How Prefetch and useQuery Work Together

Now let’s see what happens when the user actually opens the assignee picker. Inside the picker component, we use `useQuery` like this:

```
1  const { data, isLoading } = useQuery<PageResult>(  
2    `users:${debounced}:${pageSize}:${page}`,  
3    () => fetchUsers(page, pageSize ?? 5, debounced),  
4    60_000,  
5  );
```

The logic inside `useQuery` begins by checking the cache. If prefetch has already filled this entry, `useQuery` simply returns the cached data. No network request is made, and `isLoading` is false from the start. The UI appears instantly.

If the user opens the panel too quickly (before prefetch resolves), then `useQuery` performs the fetch as usual. Prefetch is an optimisation, not a guarantee.

The Full Runtime Experience

The whole flow becomes much clearer when we step through it:

1. The user hovers over the assignee button.
2. `prefetchUsers()` runs and starts loading the first page of users.
3. The result arrives and is stored in the cache.
4. The user clicks the button to open the picker.
5. `useQuery()` checks the cache, finds the preloaded data, and returns it immediately.
6. The picker appears without any loading spinner.

This is the essence of prefetch: shift the waiting to a moment when the user is not blocked.

When Prefetch Helps

Prefetch is most useful when you can reasonably predict the user's next interaction and when the data is small or inexpensive. It is especially effective for UI elements like drawers, popovers, tabs, or modals, where any delay is immediately noticeable to the user.

When Prefetch Doesn't Help

Prefetching can backfire if the data is large or unlikely to be used. It can waste bandwidth on slow networks, increase memory usage, or even compete with more important network requests. As with all performance techniques, use it intentionally.

Summary

Prefetch is a simple but powerful tool for removing perceived latency. Instead of waiting for data only after the user clicks, we begin the work earlier based on user intent. With a small addition to our query system and a single event handler on the UI, we can make interactions feel significantly faster.

In this chapter, we built a prefetch mechanism, used it to preload user data, and saw how it seamlessly integrates with our existing `useQuery` system. As your application grows, this technique becomes an effective way to make your UI feel responsive without increasing backend complexity.

Chapter 16 – Implementing Optimistic Updates

Fetching data makes an application come alive, but it only covers half of what users expect. Real applications let users *change* things: adding new items, editing details, moving tasks, deleting entries, and so on. These interactions fall under the umbrella of **data mutation**.

Sending a mutation to the backend is usually straightforward – we make a POST or PATCH request and the server updates its state. But deciding *when* the UI should update is where things get interesting. If we wait for the server before showing any changes, the UI often feels slow. If we update too early, we risk showing incorrect information.

In this chapter, we'll start with a simple example – creating a new card in the board application – then use that to introduce one of the most effective strategies for making mutations feel fast: **optimistic updates**.

Creating a New Card (Server-First)

Our backend defines an endpoint for creating a card. It accepts a `title` and a `columnId`, finds the correct column, generates a unique ID for the new card, appends it, and returns the created card:

```
1 app.post("/api/cards", async (req, res) => {
2   const payload = req.body as { title: string; columnId: string };
3
4   if (!payload.title || !payload.columnId) {
5     return res.status(400).json({ error: "Title and columnId are required" });
6   }
7
8   const boardData = board as BoardPayload;
9   const column = boardData.columns.find((col) => col.id === payload.columnId);
10
11  if (!column) {
12    return res.status(404).json({ error: `Column ${payload.columnId} not found`
    ↪  });
13  }
```

```
13   }
14
15   const totalCards = boardData.columns.reduce(
16     (sum, col) => sum + col.cards.length,
17     0,
18   );
19   const newCardId = `TICKET-${totalCards + 1}`;
20
21   const newCard: Card = {
22     id: newCardId,
23     title: payload.title.trim(),
24   };
25
26   column.cards.push(newCard);
27
28   return res.status(201).json(newCard);
29 };
```

On the frontend, `BoardColumn.tsx` includes a small form to create a card:

```
1  const handleCreateCard = async () => {
2    const title = newCardTitle.trim();
3    if (!title) return;
4
5    setIsCreating(true);
6    try {
7      const response = await fetch("/api/cards", {
8        method: "POST",
9        headers: { "Content-Type": "application/json" },
10       body: JSON.stringify({ title, columnId }),
11     });
12
13     if (response.ok) {
14       const newCard = await response.json();
15       addCard(columnId, newCard);
16       setNewCardTitle("");
17     } else {
18       console.error("Failed to create card");
19     }
20   } catch (error) {
21     console.error("Error creating card:", error);
22   } finally {
23     setIsCreating(false);
24   }
25 };
```

This approach works, but the user doesn't see the new card until the server replies. Even a small delay feels noticeable when you're adding many cards

quickly. This is where optimistic updates help.

Introducing Optimistic Updates



Optimistic update Update the UI immediately *before* the server responds, assuming the request will succeed. If it fails, roll back to the previous state.

With the server-first approach, the sequence looks like this:

1. User types a title.
2. User clicks “Add”.
3. UI waits for the server.
4. Card appears.

This keeps the UI correct, but it feels slow.

Optimistic updates flip the order:

1. Add the new card to the UI immediately.
2. Send the request to the server in the background.
3. If the server succeeds, do nothing further.
4. If the server fails, remove the card and show an error.

This creates instant feedback and a much smoother experience.

Let’s apply this technique to card creation.

Adding a Card Optimistically

To perform an optimistic creation, we need two things:

- A temporary ID for the card (so React can render it immediately).
- A way to roll back if the request fails.

Let’s assume we have a card context with an `addCard` function and a `removeCard` function. Here is one way to implement optimistic creation:

```
1  const handleCreateCard = async () => {
2    const title = newCardTitle.trim();
3    if (!title) return;
4
5    // 1. Create a temporary card
6    const tempId = `temp-${Date.now()}`;
7    const optimisticCard = { id: tempId, title };
8
9    // Add it to the UI immediately
10   addCard(columnId, optimisticCard);
11   setNewCardTitle("");
12
13   try {
14     // 2. Send the real request
15     const response = await fetch("/api/cards", {
16       method: "POST",
17       headers: { "Content-Type": "application/json" },
18       body: JSON.stringify({ title, columnId }),
19     });
20
21     if (!response.ok) {
22       throw new Error("Failed to create card");
23     }
24
25     const savedCard = await response.json();
26
27     // 3. Replace the temporary card with the server card
28     replaceCard(columnId, tempId, savedCard.id, savedCard.title);
29   } catch (error) {
30     // 4. Roll back on failure
31     removeCard(columnId, tempId);
32     console.error("Error creating card:", error);
33   }
34   };
```

Let's step through this slowly.

Step-by-step

1. We create a temporary card with a fake ID. This makes it possible for React to render it immediately.
2. We insert it into the UI using `addCard`.
3. Meanwhile, the server request runs in the background.
4. When the server returns the real card, we replace the temporary one with the real card.
5. If the server fails, we remove the temporary card from the UI.

The most important part is the rollback path. Errors are rare in normal use, but they must be handled. Rolling back ensures your UI never drifts away from the server's actual state.

This technique makes adding cards feel instant. Users no longer wait for the round trip in order to see what they just created.

Deleting a Card Optimistically

Deletion benefits even more from optimistic updates, because waiting for the server before disappearing the card feels particularly awkward.

Here is a typical optimistic delete:

```
1  <button
2    onClick={async () => {
3      const backup = card;          // save a copy for rollback
4      removeCardLocally(card.id);    // optimistic delete
5
6      try {
7        await deleteCard(card.id);
8      } catch {
9        restoreCard(backup);         // rollback if server rejects
10       alert("Could not delete card");
11     }
12   }}
13 >
14   Delete
15 </button>
```

As before:

- We update the UI immediately.
- We save a backup so we can recover the previous state.
- If deletion fails, we restore the backup and inform the user.

This pattern keeps destructive actions fast while still respecting correctness.

Things to Keep in Mind

Optimistic updates dramatically improve responsiveness, but they come with responsibilities:



Rollback is essential Always store enough information to undo the optimistic change.



Consistency across the app If multiple components rely on the same data, make sure they all update correctly when the optimistic update applies or rolls back.



Revalidation When appropriate, re-fetch or revalidate after a mutation succeeds. This prevents subtle inconsistencies.



Clear feedback on errors If something fails, the UI should clearly inform the user and revert gracefully.

Summary

Mutations are a core part of building real, interactive applications. The traditional “server-first” approach updates the UI only after the server responds, but this often feels slow. Optimistic updates reverse the order: the UI updates immediately, and only rolls back if the server sends an error.

In this chapter, we walked through:

- Creating a card in the traditional server-first manner
- Improving the experience with an optimistic approach
- Using optimistic updates for deletion, including rollback strategies

Optimistic updates can make your UI feel dramatically faster without compromising correctness. They are a powerful tool to have in your performance toolkit.

In the next chapter, we’ll go the other direction: what happens when the server updates *us*? We’ll introduce real-time updates using Server-Sent Events.

Chapter 17 – Exploring Real-time Update Strategies

In the last chapter, we looked at optimistic updates – a way to make the UI feel fast even though the server still performs the actual mutation. But optimistic updates only solve *one* side of the experience. If another user makes a change, or if the server updates an item in the background, our UI won't know unless we explicitly fetch again.

A real application needs a way for the server to *push* updates to the client. This is where real-time communication comes in.

In this chapter, we'll explore the three common approaches:

- Polling
- Server-Sent Events (SSE)
- WebSockets

Each has different trade-offs, different use cases, and different levels of complexity. We'll begin with the simplest and work our way up.

Polling

Polling is the easiest technique to understand: the client simply asks the server for updates at regular intervals.

```
1 // Short polling - ask every 5 seconds
2 const pollForUpdates = () => {
3   setInterval(async () => {
4     try {
5       const response = await fetch('/api/board/1');
6       const data = await response.json();
7       updateUI(data);
8     } catch (error) {
9       console.error('Polling failed:', error);
10    }
11  }, 5000);
12 };
```

Every five seconds the client asks, “*Anything new?*” If the server has updated data, the UI reflects it. If not, we still burned a request. This waste is tolerable on small apps, but becomes expensive as traffic grows.

A slightly more efficient version is **long polling**. The client sends a request that the server holds open until something changes.

```
1 // Long polling - server replies only when something changes
2 const longPoll = async () => {
3   while (true) {
4     try {
5       const response = await fetch('/api/board/1/poll?timeout=30');
6       const data = await response.json();
7       updateUI(data);
8     } catch (error) {
9       console.error('Long polling failed:', error);
10      await new Promise(r => setTimeout(r, 1000));
11    }
12  }
13 };
```

Here the server responds only when:

- An update occurs, or
- The timeout expires

This reduces waste, but the flow is still request-driven. The client is always the one initiating the communication.

Server-Sent Events (SSE)

With SSE, we flip the direction. The client opens a single connection and listens for updates that the server pushes as soon as they happen.

```
1 // Client-side SSE
2 const eventSource = new EventSource('/api/board/1/events');
3
4 eventSource.addEventListener('card-assigned', (event) => {
5   const data = JSON.parse(event.data);
6   updateCardAssignment(data);
7 });
8
9 eventSource.onerror = (error) => {
10  console.error('SSE error:', error);
11 };
```

Instead of checking periodically, the client simply listens. When a card is assigned or updated, the server pushes an event:

```
1 // Server-side SSE
2 app.get('/api/board/:id/events', (req, res) => {
3   res.writeHead(200, {
4     'Content-Type': 'text/event-stream',
5     'Cache-Control': 'no-cache',
6     'Connection': 'keep-alive',
7   });
8
9   res.write(`event: connected\ndata: ${JSON.stringify({ boardId: req.params.id
10     ↪ })}\n\n`);
11
12   const handleCardAssigned = (data) => {
13     res.write(`event: card-assigned\ndata: ${JSON.stringify(data)}\n\n`);
14   };
15   eventEmitter.on('card-assigned', handleCardAssigned);
16
17   req.on('close', () => {
18     eventEmitter.off('card-assigned', handleCardAssigned);
19   });
20 });
```

SSE is ideal for **one-way** updates – news feeds, notifications, live dashboards, presence updates, task status changes, and so on. It's simple, lightweight, and supported natively by browsers.

WebSockets

WebSockets provide a **full-duplex** connection. Both client and server can send messages at any time. This makes them suitable for scenarios where *both sides* need to communicate frequently.

```
1 // Client
2 const ws = new WebSocket('ws://localhost:3000/board/1');
3
4 ws.onopen = () => {
5   ws.send(JSON.stringify({ type: 'join-board', boardId: '1' }));
6 };
7
8 ws.onmessage = (event) => {
9   const data = JSON.parse(event.data);
10  if (data.type === 'card-dragging') {
11    updateCardPosition(data);
12  }
13 };
```

And the server:

```
1 // WebSocket server
2 const WebSocket = require('ws');
3 const wss = new WebSocket.Server({ port: 3000 });
4
5 wss.on('connection', (ws) => {
6   ws.on('message', (msg) => {
7     const data = JSON.parse(msg);
8     if (data.type === 'card-dragging') {
9       ws.send(JSON.stringify(data)); // broadcast or process
10    }
11  });
12 });
```

WebSockets shine in interactive, real-time scenarios like collaborative editing, live typing indicators, chat systems, or dragging cards in a shared interface.

Scaling Considerations

As your application grows, real-time communication introduces new design considerations.

Keeping many users connected means maintaining many open connections – and each connection consumes resources. Polling becomes expensive because it generates constant requests; SSE and WebSockets reduce the request count, but require your backend to track open streams or sockets for every user.

If your backend runs on multiple instances, updates must be coordinated across them. For example, if *Instance A* receives a card update, but the connected client is talking to *Instance B*, you need a way to deliver that update consistently. Systems like **Redis Pub/Sub**, **Kafka**, or similar message brokers are commonly used to broadcast events internally.

On the client side, real-time connections are not always stable. Tabs sleep, network conditions change, and mobile devices frequently disconnect. Your UI needs to handle reconnection and resubscription gracefully.

There's also the question of **data freshness**. If users can update the same data at the same time, your application must decide how to resolve conflicts. Some systems rely on timestamps or version numbers; others simply re-fetch after receiving a server event to guarantee the UI reflects the latest state.

Finally, not every feature needs a persistent subscription. For example, in a pizza order tracking app, a simple polling logic would fit perfectly. But if the backend supports pushing events, technologies like SSE or GraphQL subscriptions often scale better over time because they avoid sending repeated requests. Polling is simple and reliable, but real-time push mechanisms tend to become more efficient as traffic increases.

Summary

Real-time communication is essential for keeping modern applications in sync. Polling is easy to implement and works everywhere, but becomes inefficient as load grows. SSE provides a lightweight, server-to-client push mechanism and is ideal for unidirectional updates. WebSockets offer a full-duplex channel for rich, interactive workflows.

Each tool fits a different set of use cases. The most important part is choosing the right level of complexity for the problem you're solving.

In the next chapter, we'll build a real-time feature directly into our board application, using SSE to keep multiple browser sessions in sync without manual refresh.

Chapter 18 – Implementing Real-time Updates with SSE

In the last chapter, we looked at different approaches for real-time updates: polling, Server-Sent Events (SSE), and WebSockets. In this chapter, we'll make things concrete and implement real-time updates in our board application using SSE.

Our goal is simple: when a card is updated on the server (for example, its assignee changes), all open board pages should reflect that update automatically, without a manual refresh.

To do that, we will:

1. Introduce a simple publish–subscribe (pub-sub) mechanism on the server
2. Use it to push card updates over an SSE endpoint
3. Listen for those events on the frontend and update local state

Along the way, we'll see how this pattern is just a practical version of the Observer pattern you may have seen in design patterns books.

The pub-sub pattern

Pub-sub (publish–subscribe) is a simple way to decouple **producers** of events from **consumers** of those events.



Publish–subscribe In pub-sub, code that produces events “publishes” them, and code that cares about those events “subscribes” to them. Publishers don’t need to know who is listening.

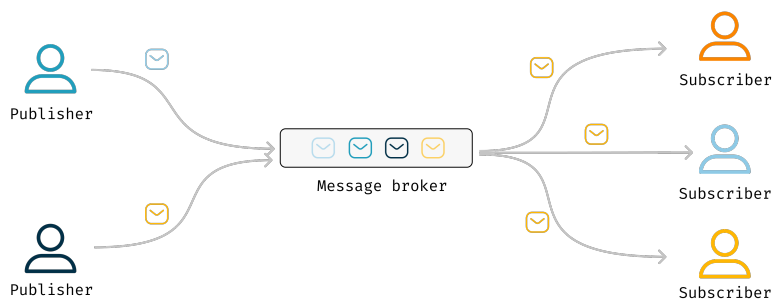


Figure 20. Pub-sub pattern

In our case:

- The code that updates a card is the publisher.
- The SSE endpoint is the subscriber.
- When the card changes, the update code publishes an event.
- The SSE endpoint forwards that event to all connected clients.

This lets us keep the update logic focused on business rules, while the SSE layer focuses on pushing updates out to the clients.

Using the pub-sub pattern on the server

To keep things lightweight, we can implement a minimal event emitter instead of pulling in a full library. Here is a small `MockEventEmitter`:

```
1 class MockEventEmitter {
2   private listeners: Map<any, any>;
3
4   constructor() {
5     this.listeners = new Map();
6   }
7
8   on(event, callback) {
9     if (!this.listeners.has(event)) {
10      this.listeners.set(event, new Set());
11    }
12    this.listeners.get(event).add(callback);
13  }
14
15  off(event, callback) {
16    if (this.listeners.has(event)) {
17      this.listeners.get(event).delete(callback);
18    }
19  }
20
21  emit(event, data) {
22    if (this.listeners.has(event)) {
23      this.listeners.get(event).forEach((callback) => callback(data));
24    }
25  }
26 }
27
28 export { MockEventEmitter };
```

The `MockEventEmitter` has three basic operations:

- `on(event, callback)` registers a listener for a given event name.
- `off(event, callback)` removes that listener.
- `emit(event, data)` calls all listeners that subscribed to that event.

Internally, it uses a `Map` keyed by event name, where each value is a `Set` of callback functions. This is enough for our needs: multiple parts of the server can subscribe to the same event, and any part of the code can emit it.

Using the event emitter with SSE

With the pub-sub piece in place, we can now wire it into an SSE endpoint. The idea is:

- Each connected client opens `/api/board/:id/events`.
- The server subscribes that connection to the relevant events.
- Whenever a card changes, we emit an event and send it over the SSE stream.

Here is the server endpoint:

```

1  // GET /api/board/:id/events (SSE)
2  app.get("/api/board/:id/events", async (req, res) => {
3    const boardId = req.params.id;
4
5    // SSE headers
6    res.writeHead(200, {
7      "Content-Type": "text/event-stream",
8      "Cache-Control": "no-cache",
9      "Connection": "keep-alive",
10     "Access-Control-Allow-Origin": "*",
11     "Access-Control-Allow-Headers": "Cache-Control",
12   });
13
14   // helper to send one event
15   const sendEvent = (eventType: string, data: any) => {
16     res.write(`event: ${eventType}\n`);
17     res.write(`data: ${JSON.stringify(data)}\n\n`);
18   };
19
20   // initial connection event
21   sendEvent("connected", { boardId });
22
23   // listeners (same event name as your emitter usage)
24   const handleCardAssigned = (data: any) => {
25     sendEvent("card-assigned", data);
26   };
27
28   mockEventEmitter.on("card-assigned", handleCardAssigned);
29
30   // heartbeats (keeps proxies happy)
31   const heartbeat = setInterval(() => res.write(`: keep-alive\n\n`), 15_000);
32
33   // cleanup on disconnect
34   req.on("close", () => {
35     clearInterval(heartbeat);
36     mockEventEmitter.off("card-assigned", handleCardAssigned);
37     res.end();
38   });
39 });

```

Let's unpack this.

First, we set the response headers. SSE is just HTTP, but we need to tell the client and any intermediaries that:

- The response is a stream of events (`Content-Type: text/event-stream`).
- It should not be cached.
- The connection should be kept alive.

Then we define a small helper `sendEvent`, which writes a single SSE event. The format is a sequence of lines:

- One line for the event name: `event: card-assigned`
- One line for the data: `data: { ... }` as JSON
- A blank line to separate events

Next, we send an initial `connected` event, which can be useful for debugging or for confirming that the client is listening on the right board.

We then subscribe this connection to our `mockEventEmitter`. Any time the emitter receives a "card-assigned" event, our `handleCardAssigned` function will be called, and we forward that payload to the client via `sendEvent`.

The heartbeat (a comment line starting with `:`) is written every 15 seconds. This keeps intermediaries (like proxies and load balancers) from timing out the connection due to inactivity.

Finally, when the client closes the connection, we:

- Stop the heartbeat
- Unsubscribe from "card-assigned"
- End the response

This avoids leaking listeners on the server.

Emitting events from the update API

Now we need to actually emit events when a card changes. In the card update endpoint, after we find and update the card, we publish an event:

```

1 app.patch("/api/cards/:id", async (req, res) => {
2   const id = String(req.params.id);
3   const payload = req.body as Partial<Card>;
4   const data = board as BoardPayload;
5
6   // Find and update the card (simplified)
7   const cardFound = /* ... */;
8
9   if (cardFound) {
10    // for example, cardFound now has updated assignee
11    mockEventEmitter.emit("card-assigned", cardFound);
12    return res.json(cardFound);
13  }
14
15  return res.status(404).json({ error: `Card ${id} not found` });
16 });

```

The important line is:

```

1 mockEventEmitter.emit("card-assigned", cardFound);

```

This sends the updated card to all subscribers of "card-assigned", which includes any open SSE connections created by `/api/board/:id/events`. Those connections will, in turn, push events down to the browser.

The result is a simple data flow:

1. The user updates a card (for example, changes its assignee).
2. The server updates the card in memory.
3. The server emits "card-assigned" with the updated card.
4. Any clients listening on `/events` receive a card-assigned event.
5. The frontend updates its local state.

Frontend: consuming SSE with EventSource

On the frontend, we hook into SSE in `BoardPage.tsx`. We want each board page to open a connection and react to "card-assigned":

```

1  useEffect(() => {
2    if (typeof window === "undefined") {
3      return;
4    }
5
6    const es = new EventSource(`/api/board/${id}/events`);
7
8    es.onopen = () => {
9      console.log("sse connection opened");
10   };
11
12   es.addEventListener("card-assigned", (event) => {
13     const data = JSON.parse((event as MessageEvent).data);
14
15     const { id: cardId, assignee: user } = data;
16     if (user) {
17       upsertUser(user); // Ensure user is in our local state
18       updateCard(cardId, { assigneeId: user.id });
19     } else {
20       updateCard(cardId, { assigneeId: undefined });
21     }
22   });
23
24   es.onerror = (error) => {
25     console.log(`sse error: ${error}`);
26   };
27
28   return () => {
29     console.log("closing");
30     es.close();
31   };
32 }, [id]);

```



EventSource EventSource is a browser API for consuming Server-Sent Events. You give it a URL, and it keeps a long-lived connection open to receive events from the server.

The effect does a few things:

- It guards against running on the server by checking `typeof window`.
- It creates a new `EventSource` for the current board ID.
- It logs when the connection opens.
- It listens for "card-assigned" events and updates local state.
- It logs errors.
- It closes the connection on cleanup to avoid leaks.

The "card-assigned" handler is where the real work happens. Each event contains the updated card data. From that, we extract:

- The `cardId`
- The `assignee` (if any)

If there is a user, we ensure they exist in our local user store via `upsertUser`. Then we call `updateCard` to set the `assigneeId` for that card. If the assignee is missing (for example, if the card was unassigned), we clear the `assigneeId`.

This means that when **any** user assigns or unassigns a card, all open board pages will reflect that change automatically.

Demonstration and behaviour

Putting it all together, the behaviour looks like this:

- You open the board in two browser tabs.
- In one tab, you change the assignee of a card.
- The server updates the card and emits "card-assigned".
- Both tabs, through their SSE connections, receive the event and update their local state.

No manual refresh, no explicit “refresh” button, and no extra polling logic. The UI simply stays in sync with the server.

Summary

In this chapter, we turned the abstract idea of real-time updates into a concrete implementation using Server-Sent Events:

- We introduced a simple pub-sub mechanism with `MockEventEmitter`.
- We used SSE on the server to stream events to connected clients.
- We emitted events from the card update API whenever a card changed.
- We connected the frontend using `EventSource` and wired incoming events into our local state.

This pattern is powerful because it keeps responsibilities separate: mutation logic updates the data and emits events, while the SSE layer focuses on broadcasting those events to interested clients.

In the broader picture of frontend system design, this is another example of how we can design for **data flow**: mutations, optimistic updates on the client, and real-time notifications from the server all work together to keep the UI fast and consistent.

Case Study — Implementing WebSockets for Real-time Updates

In the previous chapter, we implemented real-time updates using **Server-Sent Events (SSE)**. With that approach, when a user assigns a card in one tab, other open tabs can see the change immediately.

SSE works well for many scenarios, and we could continue using it for other mutations too. However, SSE is not the only option. Another common approach is **WebSockets**, which allow full-duplex communication between the client and the server.

We've already discussed the trade-offs between SSE and WebSockets in the previous chapter. In this chapter, we'll focus on **how to implement the same real-time behaviour using WebSockets**, and—more importantly—what design problems appear once we do.



Reference implementation All the code shown in this chapter comes from the `websockets` branch. If you want to explore the full, working example end-to-end, check out that branch before reading further.

Server Side — Setting Up WebSockets

To support WebSockets, we first need to make a few changes on the backend.

At a high level, we'll:

- install the `ws` package
- attach a WebSocket server to our existing HTTP server
- track active WebSocket connections per board
- broadcast domain events (for example, “card created”) to connected clients

Installing the WebSocket Library

In the server/ folder, install the ws package:

```
1 npm install ws --save
```

We'll use two APIs from this package:

```
1 import { WebSocketServer, WebSocket } from "ws";
```

Tracking Connections per Board

Even though our application currently has only one board, it's important to design this properly from the start.

We'll keep a collection of WebSocket connections **per board**:

```
1 // WebSocket connections per board
2 const wsConnections = new Map<string, Set<WebSocket>>();
```

This allows us to broadcast updates only to clients that care about a particular board.



Grouping connections by board may look unnecessary right now, but it prevents the real-time layer from becoming tightly coupled to “a single global board” assumption.

Attaching the WebSocket Server

Next, we attach the WebSocket server to the existing HTTP server:

```
1  const PORT = process.env.PORT || 4000;
2  const server = app.listen(PORT, () => {
3    console.log(`API running at http://localhost:${PORT}`);
4  });
5
6  const wss = new WebSocketServer({ server });
```

By doing this, WebSocket and HTTP traffic share the same port and lifecycle.

Handling WebSocket Connections

Now we can listen for incoming WebSocket connections:

```
1  wss.on("connection", (ws, req) => {
2    const url = new URL(req.url || "", `http://${req.headers.host}`);
3    const boardId = url.pathname.split("/").pop();
4
5    if (!boardId) {
6      ws.close(1008, "Board ID required");
7      return;
8    }
9
10   if (!wsConnections.has(boardId)) {
11     wsConnections.set(boardId, new Set());
12   }
13
14   wsConnections.get(boardId)!.add(ws);
15
16   ws.send(JSON.stringify({ type: "connected", boardId }));
17
18   ws.on("close", () => {
19     wsConnections.get(boardId)?.delete(ws);
20     if (wsConnections.get(boardId)?.size === 0) {
21       wsConnections.delete(boardId);
22     }
23     console.log(`WS disconnected: board ${boardId}`);
24   });
25 });
```

What's happening here:

- Each connection extracts the `boardId` from the URL
- The connection is stored in a `Set` for that board
- A small “connected” message confirms the handshake

- Cleanup happens when the client disconnects

At this point, the server can **track who is connected**, but it still doesn't broadcast any meaningful events.

Emitting a Domain Event

Next, we connect WebSockets to our domain logic.

In the `POST /api/cards` endpoint, once a new card is created, we emit a domain event:

```
1 mockEventEmitter.emit("card-created", {
2   boardId: "1",
3   card: newCard,
4   columnId,
5 });
```



The API endpoint does not know anything about WebSockets. It emits a domain-level event and moves on. This separation is intentional.

Broadcasting to WebSocket Clients

At the global level of `server/index.ts`, we listen for that event and broadcast it:

```
1 mockEventEmitter.on(
2   "card-created",
3   (data: { boardId: string; card: Card; columnId: string }) => {
4     const message = JSON.stringify({
5       type: "card-created",
6       data: {
7         card: data.card,
8         columnId: data.columnId,
9       },
10    });
11  });
12  const wsSet = wsConnections.get(data.boardId);
13  if (wsSet) {
14    for (const ws of wsSet) {
```

```

15         if (ws.readyState === WebSocket.OPEN) {
16             ws.send(message);
17         }
18     }
19 },
20 },
21 );

```



A single domain event results in a single broadcast. The number of connected clients does not multiply the event.

Client Side — Receiving WebSocket Events

Now let's switch to the client.

Our goal is simple: when something changes on the board, the UI should update automatically.

Encapsulating WebSocket Logic in a Hook

We encapsulate the WebSocket logic in a custom hook:

```

1  export function useBoardWebSocket(
2      boardId: string,
3      onCardCreated?: (data: {
4          card: { id: string; title: string };
5          columnId: string;
6      }) => void,
7  ) {
8      const wsRef = useRef<WebSocket | null>(null);
9      const onCardCreatedRef = useRef(onCardCreated);
10
11      useEffect(() => {
12          onCardCreatedRef.current = onCardCreated;
13      }, [onCardCreated]);
14
15      useEffect(() => {
16          if (!boardId) return;
17
18          const ws = new WebSocket(`ws://localhost:4000/ws/board/${boardId}`);
19          wsRef.current = ws;
20

```

```

21     ws.onopen = () => {
22         console.log("WS connected:", boardId);
23     };
24
25     ws.onmessage = (event) => {
26         const message: WebSocketMessage = JSON.parse(event.data);
27
28         if (message.type === "card-created") {
29             onCardCreatedRef.current?.(message.data);
30         }
31     };
32
33     ws.onerror = (err) => {
34         console.error("WS error", err);
35     };
36
37     ws.onclose = () => {
38         console.log("WS closed:", boardId);
39     };
40
41     return () => {
42         ws.close(1000, "cleanup");
43         wsRef.current = null;
44     };
45 }, [boardId]);
46 }

```



The connection lifecycle is tied to `boardId`, not to re-renders. Updating the callback does not cause the socket to reconnect.

Wiring It into the Board

In the board page, we register the handler:

```

1  useBoardWebSocket(id, ({ card, columnId }) => {
2      addCard(columnId, {
3          id: card.id,
4          title: card.title,
5      });
6  });

```

Whenever a `card-created` event arrives, we update the global board state via `addCard`.

Observing the Behaviour

Open the board in two tabs – Tab A and Tab B.

When you add a card in one tab, the other tab updates immediately:

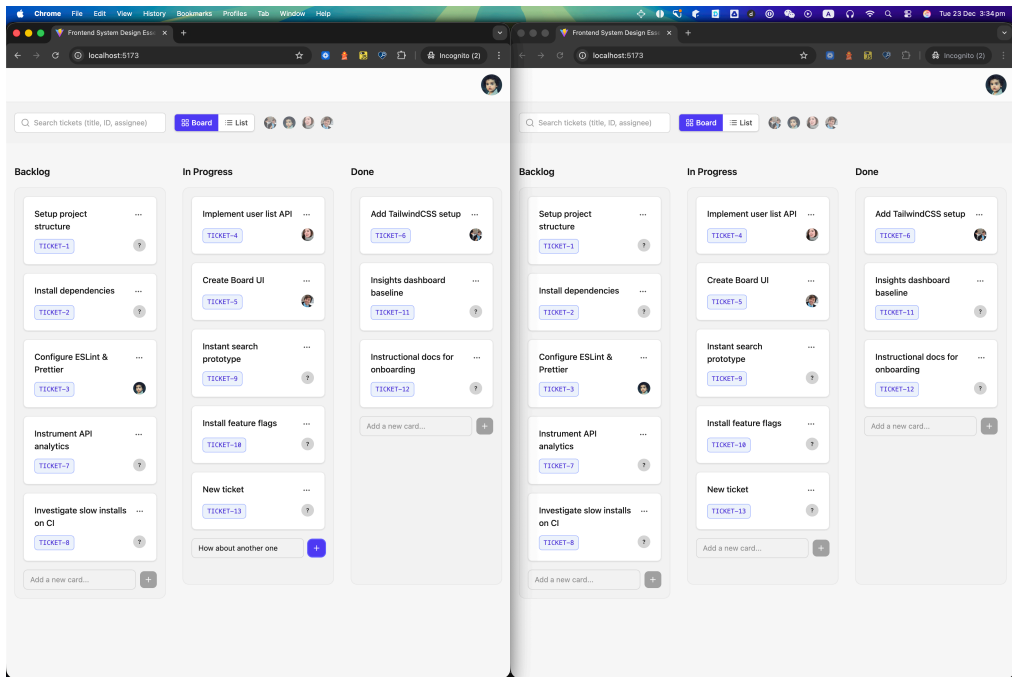


Figure 21. websockets add card

So far, everything looks correct.

A Real-World Bug: Duplicate Cards

When you add a card in **Tab A**, it appears **twice**.

This happens because:

1. Tab A adds the card locally after submitting the form
2. The backend broadcasts card-created
3. Tab A receives the broadcast as well
4. The same card is added again



Broadcasts are not acknowledgements. The sender receives the event too.

Making the State Update Idempotent

The fix belongs in the state update logic, not in the WebSocket layer.

In BoardContext, we update addCard so it becomes **idempotent**:

```
1  const addCard = useCallback(  
2    (columnId: string, card: { id: string; title: string }) => {  
3      setState((prevState: NormalizedBoard) => {  
4        if (prevState.cardsById[card.id]) {  
5          return prevState;  
6        }  
7  
8        // existing logic  
9      });  
10   },  
11   [],  
12 );
```

Now:

- optimistic updates are safe
- WebSocket echoes are safe
- retries and replays are safe

Calling addCard multiple times with the same ID produces the same state.

Summary

In this chapter, we:

- implemented real-time updates using WebSockets
- integrated WebSockets cleanly into the backend
- encapsulated connection logic in a stable client-side hook
- uncovered a subtle duplication bug caused by broadcasts

- fixed it by making state updates idempotent



Key takeaway In real-time systems, the same event can arrive through multiple paths – so state updates must be idempotent.

With this in place, we can build more advanced features—optimistic UI, retries, and failure handling—without sacrificing correctness.

Chapter 19 — Optimizing Performance with HTTP Caching

Up to this point, most of our performance improvements have focused on what happens inside the frontend: normalising data, breaking up bundles, prefetching resources, using optimistic updates, and wiring up real-time events. These all matter, but a big part of perceived performance is determined long before React even runs.

A large portion of latency comes from **how often the browser must talk to your server, how much data it needs to download, and how quickly the server can confirm that the data hasn't changed.**

This is where **HTTP caching** becomes a major frontend performance tool. These caching rules apply across all browsers, require no React code, and work purely by returning the right HTTP headers from your backend. When used well, they improve page load times, reduce bandwidth, and significantly lower your backend's load – all without changing components or state.

In this chapter, we'll focus on four core caching headers and apply them to our Kanban board endpoint:

- Cache-Control
- ETag
- Last-Modified
- Vary

By the end, you'll understand what each header does, how to implement it, and how they work together to create a fast and scalable frontend.

Why caching matters

Opening a board in our Kanban app typically returns:

- column structure

- lists of cards
- card details and metadata
- some related user information

This payload is often large, and it does not change every second. Yet without caching, the browser downloads it every time:

- the page loads
- the user refreshes
- the user reopens the tab after a short break

All of that extra traffic increases latency and adds unnecessary load on the backend – especially when you have many concurrent users or mobile users on unstable networks.



Caching reduces round trips, speeds up rendering, and makes your system more resilient during peak load. It's one of the simplest, high-impact improvements you can make.

Cache-Control – how long a response stays fresh

Cache-Control tells the browser **when a response may be reused without contacting the server**. This header alone can dramatically speed up everyday usage.

Let's start with a minimal, realistic Express handler:

```
1 app.get("/api/board/:id", async (req, res) => {
2   const boardId = req.params.id;
3   const board = await loadBoard(boardId);
4
5   res.set("Cache-Control", "public, max-age=60");
6   res.json(board);
7 });
```

With this, the browser may reuse the board response for **60 seconds** without making another network request.

Detailed breakdown: What does Cache-Control actually include?

Here are the most commonly used directives and what they really mean:

Directive	Meaning
max-age=60	The browser may reuse this cached response for up to 60 seconds. No network request will occur during this period.
public	The response may be stored by any cache – browser, CDN, or proxy.
private	Only the browser may store the response (useful for user-specific data).
no-cache	Browser <i>must</i> revalidate with the server before using the cached response.
no-store	Do not cache at all – not even temporarily.
stale-while-revalidate=30	If the response is older than max-age, return the stale value immediately but fetch a fresh version in the background.
must-revalidate	Once max-age expires, the browser must check with the server before using the cached data.

Using stale-while-revalidate

Let's improve our handler:

```

1 app.get("/api/board/:id", async (req, res) => {
2   const boardId = req.params.id;
3   const board = await loadBoard(boardId);
4
5   res.set(
6     "Cache-Control",
7     "public, max-age=60, stale-while-revalidate=30"
8   );
9   res.json(board);
10 });

```

This tells caches how to handle this response over time:

- For the first **60 seconds**, the response is considered **fresh**, so the cache can serve it directly without contacting the server.
- For the next **30 seconds** (from 60 to 90 seconds), caches that support stale-while-revalidate may **serve the cached response immediately** and **revalidate it in the background** to update it.
- After **90 seconds**, the cached response is too old, and the cache will typically **need to contact the server before serving it again**.

In practice, this behavior is most reliable with CDNs and proxy caches. Browsers may or may not fully support this optimization, so it's better to think of this as a *cache-level* strategy rather than a browser-only feature.

This pattern offers exceptional perceived performance, especially for dashboards or boards users revisit multiple times.



Many frontend apps can safely cache metadata (user info, board structure, settings) for tens of seconds or even minutes. Measured freshness often matters more than perfect real-time accuracy.

ETag – detecting whether content changed

While Cache-Control decides *when* a cached response is allowed, **ETag decides whether the response is still correct**.

An ETag is a small identifier (often a hash) representing the content of a response. The browser stores it and sends it back to the server on the next request.

Here's a full example:

Server-side: Returning an ETag with your response

```
1 import crypto from "crypto";
2
3 function generateETag(data: unknown) {
4   const json = JSON.stringify(data);
5   return crypto.createHash("md5").update(json).digest("hex");
6 }
7
8 app.get("/api/board/:id", async (req, res) => {
9   const boardId = req.params.id;
10  const board = await loadBoard(boardId);
11
12  const etag = generateETag(board);
13  res.set("ETag", etag);
14  res.json(board);
15 });
```

Browser's next request:

```
1 If-None-Match: "a1b2c3d4e5f6"
```

Server logic: respond quickly if no change

```
1 app.get("/api/board/:id", async (req, res) => {
2   const boardId = req.params.id;
3   const board = await loadBoard(boardId);
4
5   const newEtag = generateETag(board);
6   const clientEtag = req.headers["if-none-match"];
7
8   if (clientEtag === newEtag) {
9     return res.status(304).end(); // Not Modified
10  }
11
12  res.set("ETag", newEtag);
13  res.json(board);
14 });
```

Why ETag matters

- The browser avoids downloading the entire board payload again.
- Servers skip generating large JSON bodies when nothing has changed.

- Users on slow networks see dramatic improvements.
- Works perfectly with `Cache-Control: no-cache` (meaning “revalidate before reuse”).
- Works alongside `stale-while-revalidate` for even smoother experiences.



ETag is ideal for large resources like boards, dashboards, or aggregated content. They change occasionally, but not on every request.

Last-Modified — timestamp-based validation

Last-Modified is a simpler alternative to ETag. Instead of hashing the content, you return a timestamp indicating when the resource last changed.

Server-side example

```
1 app.get("/api/board/:id", async (req, res) => {
2   const boardId = req.params.id;
3   const board = await loadBoard(boardId);
4
5   res.set("Last-Modified", board.updatedAt.toUTCString());
6   res.json(board);
7 });
```

Browser request:

```
1 If-Modified-Since: Thu, 23 Jan 2025 10:15:00 GMT
```

Server-side validation:

```
1 app.get("/api/board/:id", async (req, res) => {
2   const boardId = req.params.id;
3   const board = await loadBoard(boardId);
4
5   const clientTimestamp = req.headers["if-modified-since"];
6   const lastUpdated = board.updatedAt.toUTCString();
7
8   if (clientTimestamp === lastUpdated) {
9     return res.status(304).end();
10  }
11
12  res.set("Last-Modified", lastUpdated);
13  res.json(board);
14 });
```

When Last-Modified works well

- Your data already has clear timestamps
- Hashing is unnecessary overhead
- The resource rarely changes within sub-second intervals

ETag vs Last-Modified

ETag is more precise (byte-level comparison). Last-Modified is simpler (timestamp-level comparison).

Both are valid; many APIs include both.

Vary — preventing incorrect caching

Vary tells caches which **request headers** should be considered when deciding whether two requests are the “same.” Without it, caches may incorrectly reuse responses that shouldn’t be shared.

Real-world problem: search queries

Example requests:

```
1 /api/board/1?q=bug
2 /api/board/1?q=feature
```

If your CDN or browser cache doesn't consider `q` as part of the cache key, it may return the wrong results.

Correct implementation with Express:

```
1 app.get("/api/board/:id", async (req, res) => {
2   const boardId = req.params.id;
3   const query = req.query.q;
4
5   res.set("Vary", "X-Search-Query");
6   const results = await searchBoard(boardId, query);
7
8   res.json(results);
9 });
```

Real-world scenario: user-specific data

If your application returns different data per user, you must tell caches that the response depends on the Authorization header.

```
1 res.set("Vary", "Authorization");
```

Without this, a shared CDN might serve one user's board to another – a severe data leak.

Benefits of Vary

Benefit	Explanation
Prevents wrong cached results	Ensures search parameters or filters do not share cached data.

Benefit	Explanation
Enables safe CDN caching	CDNs can cache user-independent responses while safely ignoring sensitive ones.
Better cache hit rates	When used correctly, cache entries are more accurate and reusable.
Improves multi-tenant correctness	Prevents data mixing between different users or organisations.



Vary is not about performance. It is about correctness. It prevents subtle and dangerous bugs in caching layers.

Putting it all together: a realistic request flow

Now that we’ve looked at individual headers in isolation, let’s walk through a complete, realistic request cycle for a board page. This is roughly what happens in a real application once HTTP caching is set up correctly.

Initial board load

When the user opens the board for the first time, the browser sends a straightforward request:

1 GET /api/board/1

The server responds with the board payload, along with a set of caching and validation headers:

```
1 Cache-Control: public, max-age=60, stale-while-revalidate=30
2 ETag: "board-v12"
3 Last-Modified: Thu, 23 Jan 2025 10:15:00 GMT
4 Vary: Authorization
```

At this point, two important things happen:

- The browser stores the response in its cache.
- The browser records the validation metadata (ETag and Last-Modified) so it can efficiently check for changes later.

From the user's perspective, this is just a normal page load. But from the browser's perspective, it now has everything it needs to avoid unnecessary network work.

Subsequent load within the cache window

If the user refreshes the page or navigates back to the board within the next 60 seconds, the browser doesn't need to talk to the server at all.

The response is served directly from the cache and rendered immediately.

This is the fastest possible path: zero network latency, zero server load, and no extra logic in your application code.

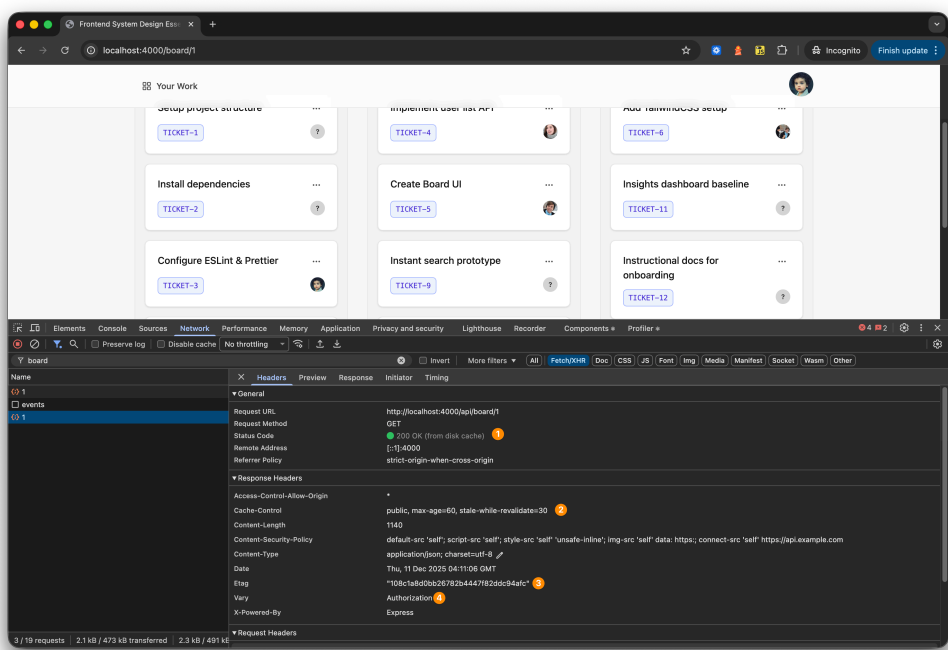


Figure 22. HTTP Headers

Load after the cache expires

After the max-age window has passed, the browser can no longer assume the cached response is fresh. Instead of fetching the entire payload again, it performs a lightweight validation request:

```
1 If-None-Match: "board-v12"
2 If-Modified-Since: Thu, 23 Jan 2025 10:15:00 GMT
```

These headers ask a simple question: *Has anything changed since the last time I fetched this resource?*

If the board is unchanged, the server responds with:

```
1 304 Not Modified
```

No response body is sent. The browser simply reuses the cached data and updates its freshness metadata. This keeps bandwidth usage low while still guaranteeing correctness.

A write operation changes the resource

Now the user edits a card on the board. On the server, this updates the underlying data, and as part of that update, the server also updates the validation headers:

```
1 ETag: "board-v13"  
2 Last-Modified: Thu, 23 Jan 2025 10:18:32 GMT
```

This step is critical. Without updating these values, the browser would have no reliable way to detect that the resource has changed.

The next read reflects the change

On the next request, the browser again performs validation using the previous ETag and Last-Modified values. This time, the validation fails – the server can see that the board has changed.

Instead of returning 304, the server sends back a fresh board payload along with the updated headers. The browser replaces the cached entry, and the user sees the latest state.

All of this happens automatically, without special client-side logic, retry loops, or manual cache invalidation. Performance stays high, data stays correct, and the system scales naturally as traffic grows.

Summary

HTTP caching is one of the highest-impact improvements you can make to your frontend performance, and it doesn't require changing any React code. By returning the right headers, you help the browser reuse data intelligently while still keeping the UI consistent and correct.

Chapter 20 — Handling Runtime Errors in React

Modern frontend applications handle a wide range of failures: network issues, invalid data, expired sessions, background mutations, and more. Testing helps us catch many of these issues early, but it cannot prevent every error — especially **runtime errors** caused by unexpected data shapes or assumptions breaking over time.

Imagine we expect the backend to return:

```
1 {  
2   name: "Juntao",  
3   address: "123 Street, City"  
4 }
```

and one day the response quietly changes to support multiple addresses:

```
1 {  
2   name: "Juntao",  
3   addresses: ["123 Street, City"]  
4 }
```

If our UI still tries to render `user.address`, React will encounter an exception during rendering. And by default, React cannot recover from a rendering error — it unmounts the entire tree, leaving the user with a blank screen.

We obviously don't want that. Before we introduce the solution, let's first connect it to something familiar.

A quick comparison to try/catch

JavaScript already gives us `try / catch` for handling risky operations:

```
1 try {  
2   riskyOperation();  
3 } catch (err) {  
4   console.log("Handled:", err);  
5 }
```

This works because **you** call the function that might fail.

Rendering in React is different. React calls *your* components. So if a component throws while rendering, a try / catch around JSX won't help:

```
1 try {  
2   return <UserSelect />;  
3 } catch {  
4   return <Fallback />;  
5 }
```

React has already taken over control of the call stack – the error never reaches this catch block.



Try-Catch try/catch' is still valuable for imperative code such as validating input, saving a card, or parsing JSON. But it cannot handle rendering errors. For that, React provides Error Boundaries.

With that in mind, let's introduce the tool React gives us for gracefully recovering from UI crashes.

Understanding Error Boundaries

An **Error Boundary** is a React component that catches rendering errors in its child subtree and displays a fallback UI instead of letting the whole app crash. It works very much like a “safety wrapper” around a section of your UI.

Here is a minimal but practical implementation:

```

1  import { Component } from "react";
2  import type { ReactNode, ErrorInfo } from "react";
3
4  export class ErrorBoundary extends Component<Props, State> {
5    constructor(props: Props) {
6      super(props);
7      this.state = { hasError: false, error: null };
8    }
9
10   static getDerivedStateFromError(error: Error): State {
11     // React will call this when a child throws during rendering
12     return { hasError: true, error };
13   }
14
15   componentDidCatch(error: Error, errorInfo: ErrorInfo) {
16     // Useful for logging or sending data to error tracking systems
17     console.error("ErrorBoundary caught an error:", error, errorInfo);
18
19     if (this.props.onError) {
20       this.props.onError(error, errorInfo);
21     }
22   }
23
24   componentDidUpdate(prevProps: Props) {
25     // Automatically reset the boundary when resetKeys change
26     if (this.state.hasError && this.props.resetKeys) {
27       const changed = this.props.resetKeys.some(
28         (key, i) => key !== prevProps.resetKeys?.[i]
29       );
30       if (changed) {
31         this.setState({ hasError: false, error: null });
32       }
33     }
34   }
35
36   render() {
37     if (this.state.hasError) {
38       return this.props.fallback ?? (
39         <SomethingWentWrong
40           onClick={() => this.setState({ hasError: false, error: null })}
41         />
42       );
43     }
44
45     return this.props.children;
46   }
47 }

```

This component uses three lifecycle methods React specifically exposes for error handling:

- **getDerivedStateFromError** marks the boundary as failed so the fallback UI can render.
- **componentDidCatch** lets you log or report the error.
- **componentDidUpdate** allows “resetting” the boundary when context changes (e.g., navigating to a new board).

Now let’s see how to use it.

Wrapping a fragile component

A common place where rendering errors show up is in components driven by untrusted data – for example, a `UserSelect` that depends on backend fields.

Suppose we (intentionally) introduce an error:

```
1 if (page === 0) {
2   throw new Error("oops, page cannot be 0");
3 }
```

When the popover opens, `UserSelect` renders and throws. Without protection, this takes down the entire app.

By wrapping it in an Error Boundary, we isolate the failure to just that part of the UI:

```
1 <ErrorBoundary
2   fallback={
3     <div className="p-4 bg-red-50 rounded-lg border border-red-200">
4       <p className="text-sm text-red-600">Card failed to load</p>
5       <p className="text-xs text-neutral-500 mt-1">ID: {c.id}</p>
6     </div>
7   }
8 >
9   <UserSelect
10    selected={assignee ?? null}
11    handleChange={handleAssignUser}
12  />
13 </ErrorBoundary>
```

Now only the dropdown fails. The card – and the rest of the page – remains interactive.

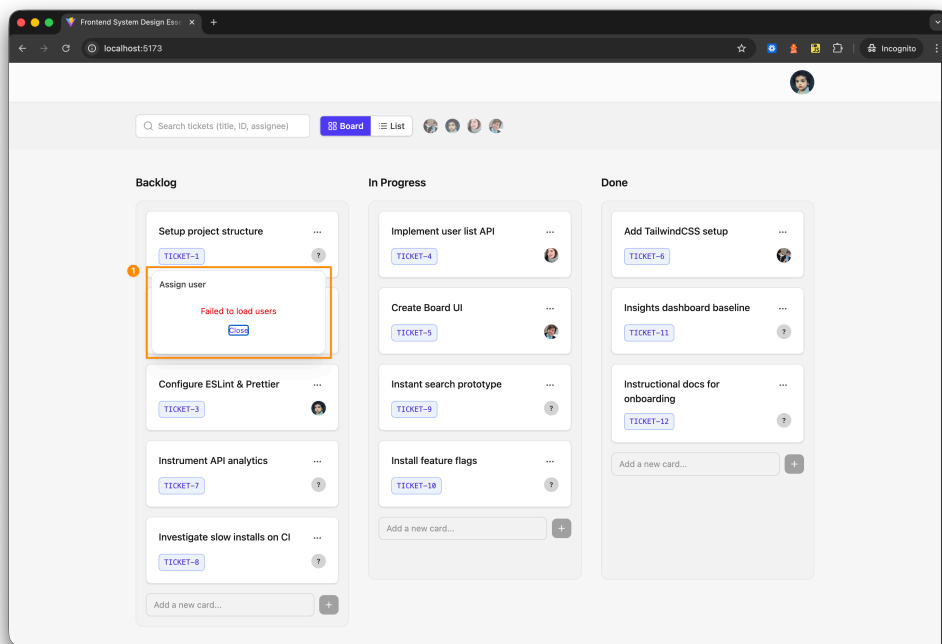


Figure 23. Fallback for UserSelect

This pattern gives the user a much better experience: the part that failed communicates the error directly, without bringing everything else down.

Cascading error boundaries in a column

We can take this idea further. In a board view, one card failing to render shouldn't break the entire column. Wrapping each card individually isolates failures at card-level:

```

1 {cards.map((c) => (
2   <ErrorBoundary
3     key={c.id}
4     fallback={
5       <div className="p-4 bg-red-50 rounded-lg border border-red-200">
6         <p className="text-sm text-red-600">Card failed to load</p>
7         <p className="text-xs text-neutral-500 mt-1">ID: {c.id}</p>
8       </div>
9     }
10   >
11     <Card id={c.id} title={c.title} assignee={c.assignee} />
12   </ErrorBoundary>
13 )})

```

Here the boundary wraps each card independently. This prevents a failure in one card from taking out the entire column.



Design tip Wrap Error Boundaries around components that depend heavily on data outside your control (third-party APIs, user-generated content, plugins). These areas often fail in ways that are hard to predict.

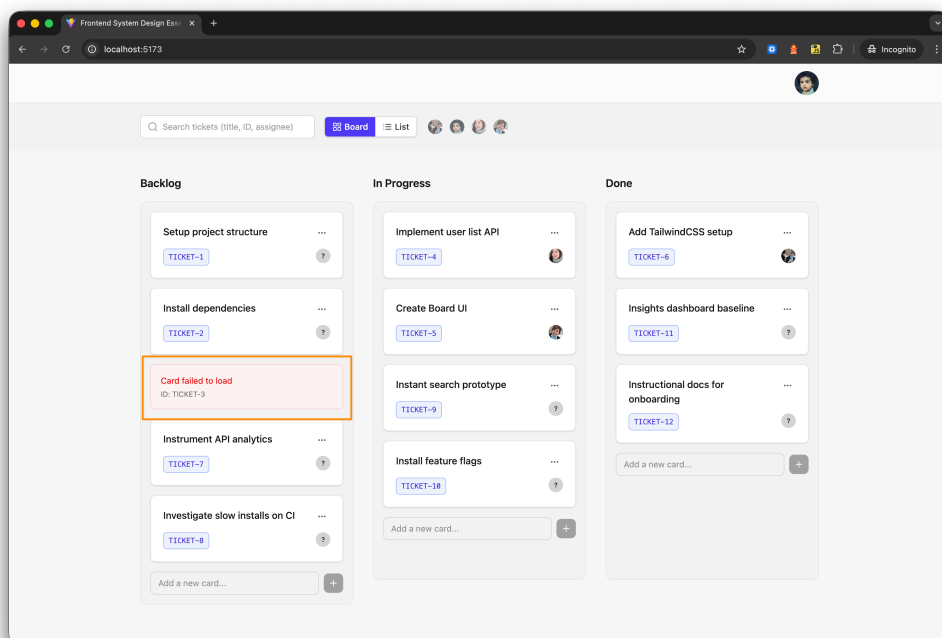


Figure 24. Fallback for UserSelect

Application-level fallback

Finally, you should always have a top-level Error Boundary to catch anything that slips through local boundaries.

```

1  <ErrorBoundary
2    resetKeys={["1"]}
3    fallback={
4      <div className="flex min-h-screen items-center justify-center
5        ↪ bg-neutral-50">
6        <div className="max-w-md text-center p-6">
7          <h2 className="text-xl font-semibold text-red-600 mb-2">
8            Failed to load board
9          </h2>
10         <p className="text-neutral-600 mb-4">
11           The board data couldn't be loaded. This might be due to a network
12           ↪ error or invalid data.
13         </p>
14         <button
15           onClick={() => window.location.reload()}
16           className="px-4 py-2 bg-indigo-600 text-white rounded-md
17             ↪ hover:bg-indigo-700"
18         >
19           Reload Page
20         </button>
21       </div>
22     </div>
23   }
24 >
25   <BoardPage id="1" />
26 </ErrorBoundary>

```

This acts as the “last line of defence.” If something unexpected happens anywhere in the tree, the user still sees a meaningful message instead of a white screen.

Summary

In this chapter we explored how runtime errors can crash React applications, and how Error Boundaries allow us to isolate failures rather than letting them take down the entire page. We compared them with the familiar try / catch flow to clarify why rendering errors behave differently, and we walked

through practical patterns for wrapping fragile components, isolating card-level failures, and defining a global fallback.

Error Boundaries don't prevent errors – but they help you recover gracefully, protect the user experience, and keep the rest of the UI alive even when something goes wrong.

Chapter 21 — Designing for Accessibility

Accessibility is a big topic. Big enough that it could easily justify a book of its own. In this chapter, we'll take a more pragmatic approach.

Rather than trying to cover every rule or edge case, we'll focus on a set of **practical patterns and examples** that show how accessibility fits naturally into frontend system design. The goal is not just to make the UI “pass checks”, but to design systems where accessibility is built in by default – so it doesn't need to be fixed later.

Throughout the chapter, we'll continue using the board application as our running example. You'll see how accessibility concerns influence component structure, interaction models, and even how we design APIs inside the frontend.

Why we need to consider accessibility at all

Accessibility is often treated as a UI polish task: add a few ARIA attributes, improve contrast, run Lighthouse, and move on. In practice, most accessibility problems don't come from missing attributes – they come from **how the system is designed**.

Common failure modes include interactions that only exist as gestures, components that have no clear semantic meaning, state changes that are visible but never announced, or flows that assume everyone uses a mouse.

These are not styling issues. They are modelling issues.

When accessibility is addressed late, teams end up layering fixes on top of unclear abstractions. Keyboard support is bolted on. Screen reader announcements are added ad-hoc. The code becomes harder to understand and harder to change.

When accessibility is considered early, the opposite happens. Interactions become explicit. State transitions become observable. Components gain clearer responsibilities. The system itself becomes easier to reason about.



Accessibility rarely makes a system worse. Most of the time, it exposes design gaps that were already there. Good question – **right now, this section explains the principles well, but it's abstract.** For a system design book, readers benefit a lot from *small, concrete examples* that anchor each idea.

Accessibility-first system design principles

Before looking at tools and implementation details, it helps to establish a few principles that guide accessible system design. These principles show up repeatedly in well-designed systems, and they influence how components, interactions, and APIs are shaped.

Semantic HTML as a foundation

Semantic HTML is the contract between your UI and assistive technologies. When elements are used correctly – buttons for actions, headings for structure, landmarks for navigation – screen readers and keyboards can interpret the interface without guesswork.

For example, consider a card action menu. If the trigger is implemented as a clickable `<div>`, it may look fine visually, but it has no built-in meaning. Screen readers don't announce it as interactive, and keyboard users can't activate it reliably.

Using a real `<button>` immediately gives you:

- keyboard focus and activation by default
- correct roles and announcements for screen readers
- predictable behaviour across browsers

The same idea applies at larger scales. Wrapping a list of cards in a `<section>` with a heading, or representing each card as an `<article>`, gives the document structure that assistive technologies can navigate meaningfully.

This structure also benefits maintainability. Components with clear semantic roles tend to be easier to test, refactor, and reason about, because their purpose is explicit in the markup.

Progressive enhancement

Progressive enhancement starts from a simple idea: the core behaviour of your application should not depend on everything working perfectly.

Instead of baking behaviour directly into complex interactions, you start with a reliable baseline and layer richer features on top.

In the board application, the core action is “move a card to another column.” Drag-and-drop is one way to express that action, but it should not be the only way. By modelling the move as a function (`moveCard`) and exposing it through different UI paths – drag-and-drop for mouse users, context menu actions for keyboard users – the system remains usable even when certain interaction modes are unavailable.

This separation between *what the user is trying to do* and *how the UI enables it* makes accessibility much easier to support. It also makes the system more flexible, because new interaction methods can be added without rewriting core logic.

Progressive enhancement encourages you to design capabilities first, and interactions second.

Keyboard-first interaction

Keyboard support is not a fallback – it is a first-class interaction model.

If an interface works well with a keyboard, it usually works well with screen readers, switch devices, and power users. Designing keyboard interactions early forces you to think carefully about focus, navigation, and activation.

For example, navigating between cards using arrow keys requires you to define an explicit order and decide what “next” and “previous” mean in the system. Handling Enter or Space to activate a card forces you to distinguish between selection and action. Supporting Escape to close menus or dialogs requires you to model exit paths clearly.

These decisions are not just about accessibility. They clarify the interaction model for everyone. Many professional tools feel “fast” and “polished” precisely because their keyboard behaviour is well thought out, not because they added shortcuts later.

Designing with the keyboard in mind early often reveals ambiguities in the UI – and resolving those ambiguities improves the system as a whole.

Accessibility checks as part of the testing architecture

In this project, we don't rely on runtime accessibility checks during development. Instead, we treat accessibility as a **testable system constraint**, enforced through unit tests.

This changes how accessibility is perceived. It is no longer something you “notice while clicking around”. It becomes something the system **asserts and protects**, just like correctness or performance.

Why test accessibility in unit tests

Testing accessibility at the component level scales well for a few reasons. Requirements are explicit and version-controlled. Failures show up in CI, not just in a developer's browser. Regressions are caught immediately when components change. Over time, developers internalise accessible patterns through failing tests.

Most importantly, accessibility becomes part of the definition of correctness for a component.



When accessibility is tested, it stops being optional. The system enforces it.

Setting up axe-jest

We use `jest-axe` (with Vitest) to run accessibility checks directly against rendered components.

The setup lives in the shared test configuration:

```
1 import "@testing-library/jest-dom";
2 import "@testing-library/jest-dom/vitest";
3
4 import { afterEach } from "vitest";
5 import { cleanup } from "@testing-library/react";
6
7 import { toHaveNoViolations } from "jest-axe";
8
9 expect.extend(toHaveNoViolations);
10
11 afterEach(() => {
12   cleanup();
13 });
```

This enables accessibility assertions globally and keeps the test environment predictable.

Writing accessibility assertions

With the setup in place, components can assert their accessibility guarantees directly.

```
1 describe("Card accessibility", () => {
2   it("has no WCAG violations (default state)", async () => {
3     const { container } = renderWithProviders(
4       <Card id="42" title="Refactor drag handle" assignee="Juntao" />
5     );
6
7     const results = await axe(container, {
8       rules: {
9         "color-contrast": { enabled: true },
10      },
11    });
12
13    expect(results).toHaveNoViolations();
14  });
15 });
```

This test expresses a clear contract: rendering a Card in its default state must not violate WCAG rules. If someone later removes a label, breaks semantic structure, introduces invalid ARIA, or reduces contrast, the test fails immediately.

Semantic HTML in real components

Let's move from tests to structure.

A well-designed frontend uses semantic HTML to create meaningful document structure. This isn't just for screen readers – it's the foundation of a maintainable system.

Refactoring the Card component

Before:

```
1 <div className="rounded-lg border...">
2   <div className="flex items-start...">
3     <h3>{title}</h3>
4   </div>
5 </div>
```

After:

```
1 <article
2   className="rounded-lg border..."
3   aria-labelledby={`card-title-${id}`}
4 >
5   <h3 id={`card-title-${id}`} className="text-base font-medium">
6     {title}
7   </h3>
8 </article>
```

By using `<article>` and properly linking the title, screen readers can announce each card meaningfully. The structure also makes it clear, in code, what a “card” represents.

Managing focus after destructive actions

Accessibility also affects how state transitions are handled. When an item is deleted, focus must move somewhere sensible.

```
1  const nextCard = document.querySelector(  
2    `[data-card-id]:not([data-card-id="${id}"])`  
3  );  
4  
5  if (nextCard instanceof HTMLElement) {  
6    nextCard.focus();  
7  }
```

Without this, keyboard and screen reader users can end up in a dead end. Focus management is not a UI detail – it is part of the system’s state logic.

Keyboard navigation as a system concern

Large applications must treat keyboard navigation as a first-class feature.

Arrow keys for navigating lists, Enter and Space for activation, Escape for exiting modal states – these are not random conventions. Users expect them, and assistive technologies rely on them.

Designing keyboard interaction early forces you to make navigation rules explicit, which often leads to cleaner, more predictable behaviour across the app.

A more complex example: accessible drag and drop

Drag and drop is a good example of where accessibility and system design intersect.

In the current implementation:

```
1  git checkout drag-and-drop-c
```

Cards can be moved between columns using the mouse, implemented with `pragmatic-drag-and-drop`. Visually, everything works. Functionally, everything works – for mouse users.

But a keyboard-only user cannot perform the same action.

This is not an attribute problem. It is an interaction model problem.

The real action is not “dragging”. It is **moving a card from one column to another**. Once we model that action explicitly, accessible alternatives become possible.

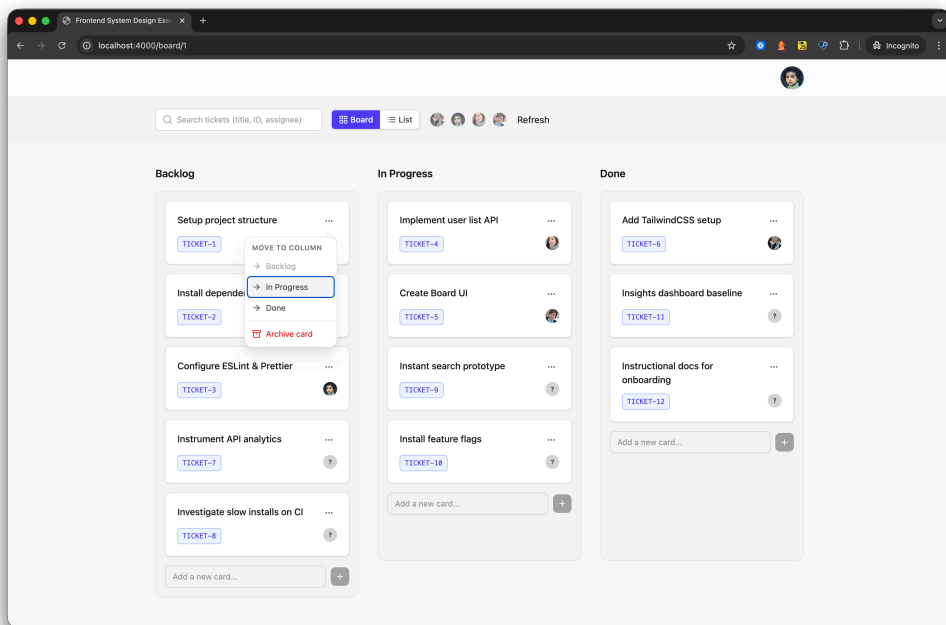


Figure 25. Move card with keyboard

Exposing the action through the context menu

We start by exposing the necessary data and behaviour from the board context:

```
1 const { columns, moveCard } = useBoardContext();
```

Instead of relying on pointer gestures, we surface the action through the card's context menu. Each destination column becomes a real button.

```
1 {columns.map((column) => {
2   const isCurrentColumn = column.id === columnId;
3
4   return (
5     <button
6       key={column.id}
7       type="button"
8       onClick={() => handleMoveToColumn(column.id)}
9       disabled={isCurrentColumn}
10    >
11      {column.title}
12    </button>
13  );
14 }}
```

This design has several benefits. The interaction is keyboard-accessible. Screen readers can announce each option. The logic is shared with drag and drop, not duplicated.

Accessibility here improves the architecture instead of complicating it.

Announcing the new position

Moving a card is a visible state change. Screen reader users need equivalent feedback.

At the board level, we add a live region:

```
1 <div
2   className="sr-only"
3   role="status"
4   aria-live="polite"
5   aria-atomic="true"
6 >
7   {announcement}
8 </div>
```

When a card is moved, we update the announcement:

```
1 moveCard(card.id, column.id, -1);
2
3 setAnnouncement(`Card "${title}" moved to ${column.name}`);
4
5 setTimeout(() => {
6   setAnnouncement("");
7 }, 1000);
```

This ensures that meaningful state changes are communicated clearly, regardless of how the user interacts with the system.



If a change matters visually, it should also be communicated audibly.

My typical accessibility workflow

In practice, accessibility works best when **multiple layers reinforce each other**. No single tool can catch everything, and relying on one check usually creates blind spots.

Over time, I've settled on a workflow where accessibility is checked continuously, from development to validation, without becoming a separate phase or a special task. Each layer has a clear responsibility, and together they form a safety net.

Quick feedback with Lighthouse and Chrome DevTools

The first layer is **fast, visual feedback**.

Lighthouse and the Accessibility panel in Chrome DevTools are useful for answering simple questions early:

- Are there obvious contrast issues?
- Are form elements labelled correctly?
- Are interactive elements exposed with the right roles?
- Does the page structure make sense at a high level?

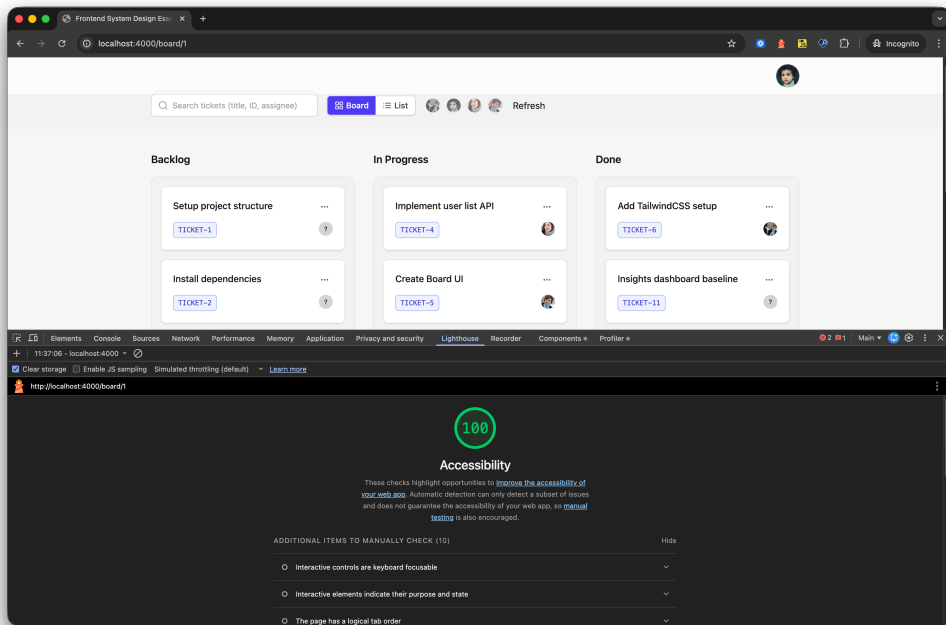


Figure 26. Lighthouse as the first layer check

These tools are not exhaustive, and they don't understand your application's intent. But they are excellent at catching low-hanging issues and surfacing problems you might not notice while focused on feature work.

I treat Lighthouse as a **sanity check**, not a source of truth. If Lighthouse complains, it's usually worth fixing. If Lighthouse is quiet, that doesn't mean the app is accessible – it just means you've cleared the most obvious hurdles.

Enforcing guarantees with axe-based unit tests

The second layer is where accessibility becomes structural.

By running axe checks in unit tests, accessibility turns into a **testable contract**. Components are no longer “probably accessible” – they are either compliant or they fail the build.

This layer is especially valuable because it operates at the component level:

- tests run fast
- failures are precise and actionable
- regressions are caught immediately

- expectations live next to the code they protect

Over time, this changes how teams think. Developers stop asking “should we test accessibility?” and start assuming that accessibility is part of correctness, just like rendering or behaviour.

This is also the layer that scales best as the codebase grows. Once the pattern is established, new components naturally inherit the same expectations.

Preventing regressions with ESLint rules

Linting sits slightly earlier in the feedback loop.

ESLint accessibility rules don’t prove that a component is accessible, but they are very good at preventing **common mistakes** from entering the codebase at all. Things like clickable elements without keyboard handlers, missing labels, or misuse of interactive roles are caught before the code even runs.

This layer works well because it is:

- immediate
- cheap
- hard to ignore

Linting also plays an educational role. Developers learn accessible patterns simply by fixing lint errors, without needing to memorise guidelines or read documentation.

Validating real interactions manually

The final layer is manual testing.

No automated tool truly understands what it *feels like* to use your application. For that, you still need to:

- navigate the app using only a keyboard
- turn on VoiceOver or NVDA and listen to how the UI is announced
- move through real workflows, not isolated components

This is where issues with focus order, confusing announcements, or awkward interaction flows usually surface. These problems are often subtle and context-dependent, which is why automation alone is not enough.

I don't do this constantly, but I do it deliberately – especially after building new interaction patterns or refactoring core flows.

Why this layered approach works

Each layer in this workflow has a clear purpose:

- browser tools catch obvious issues quickly
- unit tests enforce guarantees at the component level
- linting prevents regressions from entering the system
- manual testing validates real user experience

No single layer is sufficient on its own. Together, they create a workflow where accessibility is checked continuously, from different angles, without slowing development down.

Most importantly, accessibility stops being a “special task”. It becomes part of how the system is built and maintained – by default.



Accessibility scales best when it is reinforced by the system, not enforced by memory or goodwill.

Summary

Accessibility is not a feature you bolt on at the end of a project, and it is not something that lives only in UI polish or compliance reports. It is a set of design decisions that shape how your system is structured, how interactions are modelled, and how state changes are communicated.

Chapter 22 — Implementing Performance Monitoring

So far, we've spent a lot of time making the frontend *faster by design*: splitting bundles, lazy-loading routes, optimising assets, and enforcing performance budgets.

But there is one question we haven't answered yet:

How do we know whether users are actually experiencing a fast application?

This is where performance monitoring comes in. Monitoring closes the loop between design-time optimisations and real-world usage. It turns performance from a one-off exercise into a continuously observed system property.

In this chapter, we'll bring together three pieces:

- understanding and controlling bundle size
- enforcing performance budgets as a guardrail
- measuring real user experience using Core Web Vitals

Together, they form a practical approach to production-ready performance.

Why performance monitoring matters

You can have excellent build metrics and still ship a slow experience.

Users don't care how many kilobytes your bundle contains. They care about how quickly content appears, how responsive the interface feels, and whether the layout stays stable while they interact with it.

Performance monitoring shifts the focus from internal metrics to **user-perceived performance**. Instead of guessing whether the app feels fast, you measure it directly – on real devices, under real network conditions, with real users.

This is the difference between “we think it's fast” and “we know how fast it is”.

Understanding bundle size in practice

Before monitoring runtime performance, it's still important to understand what you ship.

Bundle size directly affects download time, parse time, and memory usage – especially on slower devices. But raw numbers aren't very helpful unless you can see *where* they come from.

Visualising the bundle

For the board app, we use a bundle visualiser to inspect the production build.

```
1 import { visualizer } from "rollup-plugin-visualizer";
2
3 export default defineConfig({
4   plugins: [
5     react(),
6     tailwindcss(),
7     visualizer({
8       filename: "./dist/stats.html",
9       open: false,
10      gzipSize: true,
11      brotliSize: true,
12    }),
13  ],
14 });
```

After running `npm run build`, the generated `stats.html` shows a visual breakdown of the bundle: which dependencies are largest, how chunks are split, and where unexpected weight comes from.

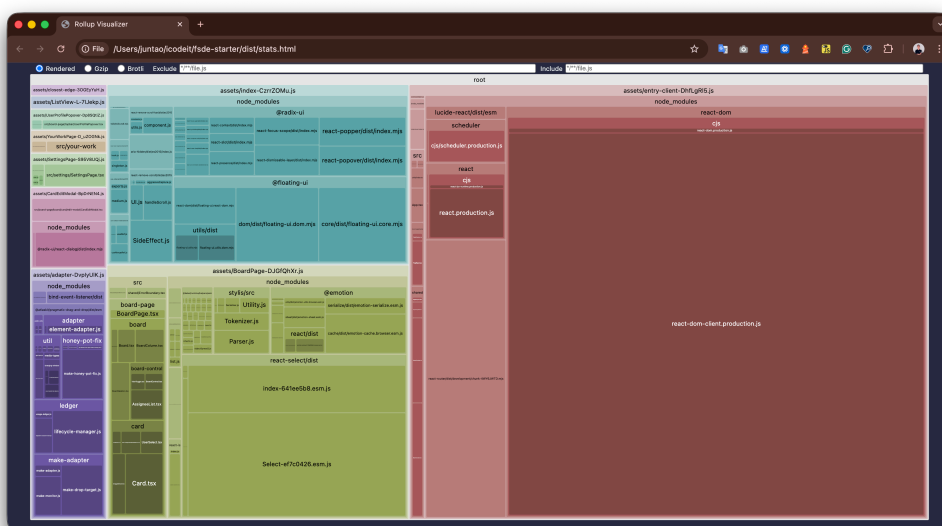


Figure 27. Visualise the bundler

This step isn't about optimisation yet. It's about building intuition. Once you understand what dominates your bundle, later decisions – like lazy loading or dependency replacement – become much more deliberate.

Making performance limits explicit with budgets

Optimisations don't last unless the system protects them.

Performance budgets make expectations explicit. Instead of relying on code reviews or memory, you define concrete limits and fail the build when those limits are exceeded.

Defining a performance budget

A simple budget configuration might look like this:

```

1  {
2    "budgets": {
3      "initial": {
4        "max": "200kb",
5        "description": "Initial JS bundle (gzipped)"
6      },
7      "vendor": {
8        "max": "150kb",
9        "description": "Vendor dependencies (gzipped)"
10     },
11     "total": {
12       "max": "500kb",
13       "description": "Total transfer size (gzipped)"
14     }
15   },
16   "targets": {
17     "LCP": "2.5s",
18     "INP": "200ms",
19     "CLS": "0.1"
20   }
21 }

```

The exact numbers will vary by application. What matters is that the limits exist and are discussed openly. A budget forces trade-offs to be conscious instead of accidental.

Enforcing the budget after build

Budgets only work if they are enforced automatically.

After building the app, we can inspect the output files, gzip them, and compare the results against the configured limits.

```

1  import { readFileSync } from "fs";
2  import { gzipSync } from "zlib";
3  import { globSync } from "glob";
4
5  const budget = JSON.parse(
6    readFileSync("./performance-budget.json", "utf-8")
7  );
8
9  const jsFiles = globSync("dist/assets/**/*.js");
10
11  let totalSize = 0;
12  let vendorSize = 0;
13  let initialSize = 0;

```

```

14
15 jsFiles.forEach((file) => {
16   const content = readFileSync(file);
17   const gzipped = gzipSync(content);
18   const size = gzipped.length;
19
20   totalSize += size;
21
22   const fileName = file.split("/").pop();
23   if (fileName.includes("vendor")) {
24     vendorSize += size;
25   } else if (fileName.includes("index")) {
26     initialSize += size;
27   }
28 });

```

Once the sizes are collected, we validate them against the budget:

```

1  const results = {
2    total: totalSize,
3    vendor: vendorSize,
4    initial: initialSize,
5  };
6
7  let hasError = false;
8
9  Object.entries(budget.budgets).forEach(([key, config]) => {
10   const actual = results[key];
11   if (!actual) return;
12
13   const maxKB = parseFloat(config.max.replace("kb", ""));
14   const actualKB = actual / 1024;
15
16   if (actualKB > maxKB) {
17     console.error(
18       `✖ ${key} bundle exceeded budget: ${actualKB.toFixed(
19         2
20       )} KB > ${maxKB} KB (${config.description})`
21     );
22     hasError = true;
23   } else {
24     console.log(
25       `✔ ${key} bundle OK: ${actualKB.toFixed(2)} KB / ${maxKB} KB`
26     );
27   }
28 });
29
30 if (hasError) {
31   process.exit(1);
32 }

```

Failing the build here is intentional. It turns performance from a warning into a hard constraint.



A performance budget only works if breaking it has consequences.

When you run the bundle check script, it produces a short summary like this:

```
1 $ npm run check-bundle
2
3 ✔ initial bundle OK: 20.55 KB / 200 KB
4 ✔ total bundle OK: 145.34 KB / 500 KB
5 🎉 All performance budgets met.
```

The **initial bundle** is the JavaScript needed for the first render. At **20.55 KB**, it's well below the **200 KB** limit, which helps keep startup fast.

The **total bundle** is the combined size of all JavaScript assets shipped to the browser. At **145.34 KB**, it stays safely under the **500 KB** budget.

Because all budgets are within limits, the script exits successfully. If any limit were exceeded, the build would fail. This makes performance regressions visible and enforceable, not just something you notice after deployment.

Measuring real user experience with Web Vitals

Bundle size and performance budgets tell you what *should* happen. Core Web Vitals tell you what *actually* happens once your app is in users' hands.

Instead of focusing on implementation details, Web Vitals measure how a page **feels** to real users, on real devices, under real network conditions. They are collected directly in the browser and reflect real rendering and interaction timing, not synthetic lab results.

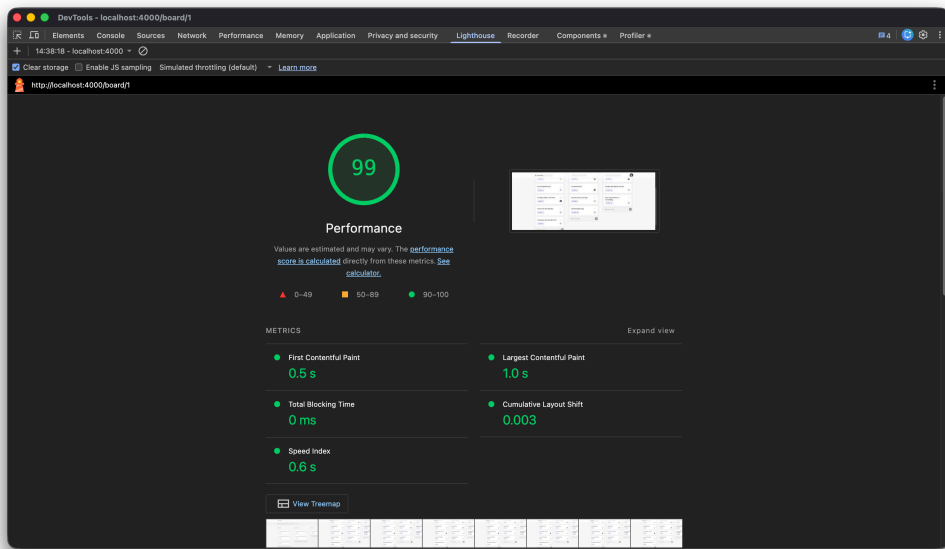


Figure 28. Web Vitals

In our app, we track the following metrics:

- **LCP (Largest Contentful Paint)** – how quickly the main content becomes visible. This captures the user’s first impression of speed.
- **INP (Interaction to Next Paint)** – how responsive the app feels when users interact, such as clicking or typing. This often exposes slow event handlers or expensive renders.
- **CLS (Cumulative Layout Shift)** – how stable the layout is while content loads. Unexpected shifts make the interface feel unreliable.
- **FCP (First Contentful Paint)** – how soon *any* content appears on screen, signaling that the page is loading.
- **TTFB (Time to First Byte)** – how quickly the server responds. This reflects backend performance, caching, and network conditions.

Together, these metrics cover the full lifecycle of a page load – from the first byte coming over the network, to the first pixels appearing, to meaningful content being visible, and finally to how the app responds to user input.

While these metrics are most valuable in production, you don’t have to wait until real users are affected. **Chrome DevTools provides early visibility into Web Vitals** through tools like Lighthouse, the Performance panel, and the Web

Vitals overlay. These help you spot slow paints, long interactions, and layout shifts during development.

DevTools measurements won't perfectly match field data, but they are extremely effective for catching problems early and validating optimizations before shipping.

Collecting Web Vitals

We use the official `web-vitals` package to collect metrics.

```
1 import { onCLS, onINP, onLCP, onFCP, onTTFB } from "web-vitals";
2
3 export function reportWebVitals(onReport) {
4   onCLS(onReport);
5   onINP(onReport);
6   onLCP(onReport);
7   onFCP(onReport);
8   onTTFB(onReport);
9 }
```

During development, simply logging these values is often enough to understand how changes affect performance:

```
1 export function logWebVitals() {
2   if (import.meta.env.DEV) {
3     onINP((metric) => {
4       console.log(
5         `[Web Vitals] ${metric.name}: ${metric.value.toFixed(2)}
6         ↳ (${metric.rating})`
7       );
8     });
9     onLCP((metric) => {
10      console.log(
11        `[Web Vitals] ${metric.name}: ${metric.value.toFixed(2)}
12        ↳ (${metric.rating})`
13      );
14    });
15    onFCP((metric) => {
16      console.log(
17        `[Web Vitals] ${metric.name}: ${metric.value.toFixed(2)}
18        ↳ (${metric.rating})`
19      );
20    });
21  }
22 }
```

In production, the same metrics can be sent to an analytics backend for aggregation and analysis (in `App.tsx`):

```
1 if (import.meta.env.PROD) {
2   reportWebVitals((metric) => {
3     // Send metric to analytics service
4   });
5 }
```

Once this is in place, performance becomes observable over time. You can track regressions, compare releases, and prioritise fixes based on real user impact instead of assumptions.

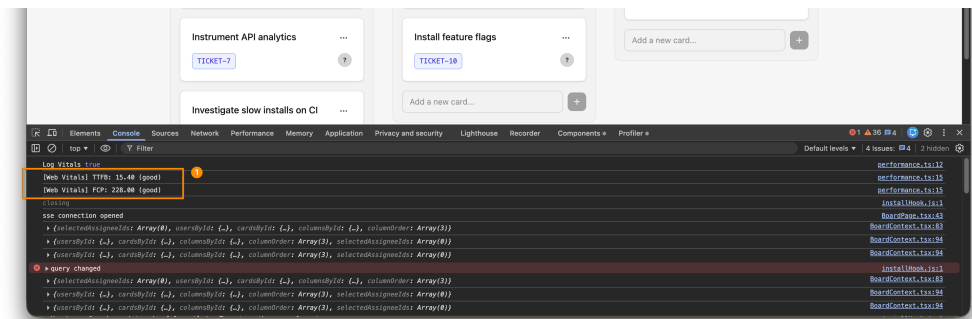


Figure 29. Report web-vitals progmatically

Performance as a continuous system concern

Performance monitoring is not about chasing perfect scores. It's about feedback.

Bundle analysis tells you where cost comes from. Budgets prevent silent growth. Web Vitals show how users experience the system in reality.

Together, these techniques turn performance into something the system actively defends – not something developers have to remember to care about.

Summary

In this chapter, we closed the loop on performance.

We analysed what we ship, enforced limits through budgets, and measured real user experience with Core Web Vitals. More importantly, we treated performance as a system concern – something observable, enforceable, and continuously validated.

A fast application is not one that was optimised once. It's one whose performance characteristics are visible and protected as the system evolves.

That is what makes performance sustainable in production.

The Architectural Roadmap: From Components to Systems

This book was written to help you move from building **isolated UI components** to designing **scalable, reliable, and maintainable frontend systems**.

Rather than presenting abstract theory, every idea is grounded in a single, evolving example: a **practical board application**, similar to a simplified Trello (or Jira). This shared playground allows each concept—data modeling, pagination, rendering strategies, performance, and resilience—to build naturally on what came before.

What follows is a final walkthrough of that journey, so you can step back and see the system as a whole.

Module 1: The System Design Mindset (Chapters 1–4)

We begin by deliberately **zooming out**.

Instead of treating the frontend as a collection of independent components, you learn to see it as a **distributed system**—where a single user action can affect shared state, network requests, and multiple screens.

This module introduces the **CCDAO framework**:

- Collect Information
- Component Structure
- Data Modeling
- API Design
- Optimization Strategies

CCDAO acts as a lightweight mental checklist. It helps you reason about architecture under real constraints, and it also provides a reliable starting point for frontend system design interviews.

Module 2: Building the Data Foundation (Chapters 5–9)

Once the mindset is in place, we focus on **data**.

Here, you learn how to align frontend state with **domain-driven language**, keep UI code declarative, and push business rules toward the backend where they belong.

A major theme in this module is **normalization**. By borrowing ideas from relational databases, we transform deeply nested API responses into flat, predictable structures. This creates a **single source of truth**, simplifies updates, and makes advanced patterns—like optimistic UI and real-time updates—possible later on.

Module 3: Managing Data Flow and Connectivity (Chapters 10–12)

With a stable data model, the focus shifts to **movement**.

You implement request management techniques such as **debouncing**, **cancellation**, and **race condition prevention** using `AbortController`. From there, we explore **pagination strategies**—offset-based and cursor-based—and how each choice shapes the user experience, especially for infinite scrolling and feeds.

This module ends by introducing a real **Express backend**, grounding the frontend in a more realistic full-stack environment and setting the stage for production-level concerns.

Module 4: Rendering and Performance (Chapters 13–15)

At this point, the system is functional—but not yet fast.

You introduce **Server-Side Rendering (SSR)** to improve first paint and SEO, and work through the mechanics of **hydration**. From there, you optimize how and when code is delivered using **code splitting**, **lazy loading**, and **prefetching based on user intent**.

The goal here is not micro-optimization, but **perceived performance**—making the application feel responsive and intentional at every step.

Module 5: Mutations, Real-Time Sync, and Resilience (Chapters 16–22)

The final module focuses on what separates a demo from a **production system**.

You implement **optimistic updates** with proper rollback strategies, then add **real-time synchronization** using Server-Sent Events and WebSockets. The book closes by hardening the system with **HTTP caching**, **error boundaries**, **accessibility-first design**, and **performance monitoring** using Core Web Vitals.

These chapters emphasize resilience: how systems behave when networks fail, data changes unexpectedly, or users interact in ways you didn't predict.

Designing System Step By Step

You can think of this book as a **city planning guide**:

- The **blueprints** define the structure (CCDAO)
- The **underground utilities** establish a stable data foundation
- The **transit lines** move data efficiently across the system
- Traffic flow is optimized through rendering and performance strategies
- Safety and health regulations ensure resilience, accessibility, and long-term stability

A well-designed frontend system, like a well-planned city, doesn't just work on day one—it continues to work as it grows, changes, and comes under pressure.

If this book has helped you think more clearly about that journey, then it has done its job.